# BCS 002-Neural Networks

# Unit-I Back Propagation

- Introduction to Artificial Neural systems
- Perception
- Representation
- Linear Separability
- Learning
- Training algorithm
- The back propagation network
- The generalized data rule
- Practical considerations
- BPN applications.

# Introduction To Neural Networks

- Development of Neural Networks date back to the early 1940s. It experienced an upsurge in popularity in the late 1980s. This was a result of the discovery of new techniques and developments and general advances in computer hardware technology.
- Some NNs are models of biological neural networks and some are not, but historically, much of the inspiration for the field of NNs came from the desire to produce artificial systems capable of sophisticated, perhaps intelligent, computations similar to those that the human brain routinely performs, and thereby possibly to enhance our understanding of the human brain.

Most NNs have some sort of training rule. In other words, NNs learn from examples (as children learn to recognize dogs from examples of dogs) and exhibit some capability for generalization beyond the training data.

Neural computing must not be considered as a competitor to conventional computing. Rather, it should be seen as complementary as the most successful neural solutions have been those which operate in conjunction with existing, traditional techniques.

# Introduction

- What are Neural Networks?
  - Neural networks are a new method of programming computers.
  - They are exceptionally good at performing pattern recognition and other tasks that are very difficult to program using conventional techniques.
  - Programs that employ neural nets are also capable of learning on their own and adapting to changing conditions.

# Neural Networks Techniques

- Computers have to be explicitly programmed
  - Analyze the problem to be solved.
  - Write the code in a programming language.
- Neural networks learn from examples
  - No requirement of an explicit description of the problem.
  - No need for a programmer.

The neural computer adapts itself during a training period, based on examples of similar problems even without a desired solution to each problem. After sufficient training the neural computer is able to relate the problem data to the solutions, inputs to outputs, and it is then able to offer a viable solution to a brand new problem.
Able to generalize or to handle incomplete data.

# Neural Networks vs Computers

- Computers have to be explicitly programmed
  - Analyze the problem to be solved.
  - Write the code in a programming language.
- Neural networks learn from examples
  - No requirement of an explicit description of the problem.
  - No need for a programmer.
  - The neural computer adapts itself during a training period, based on examples of similar problems even without a desired solution to each problem. After sufficient training the neural computer is able to relate the problem data to the solutions, inputs to outputs, and it is then able to offer a viable solution to a brand new problem.
  - Able to generalize or to handle incomplete data.

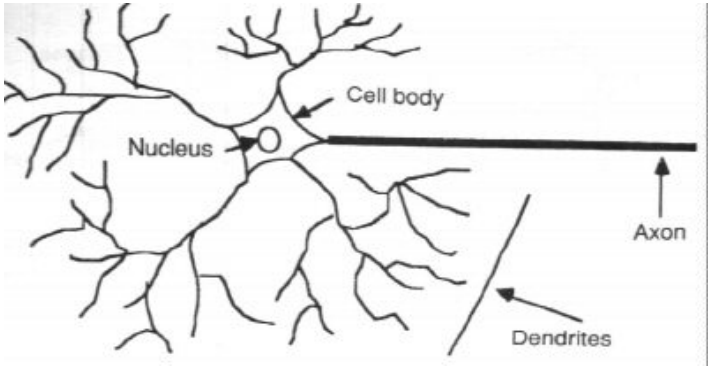
**Neural Networks**

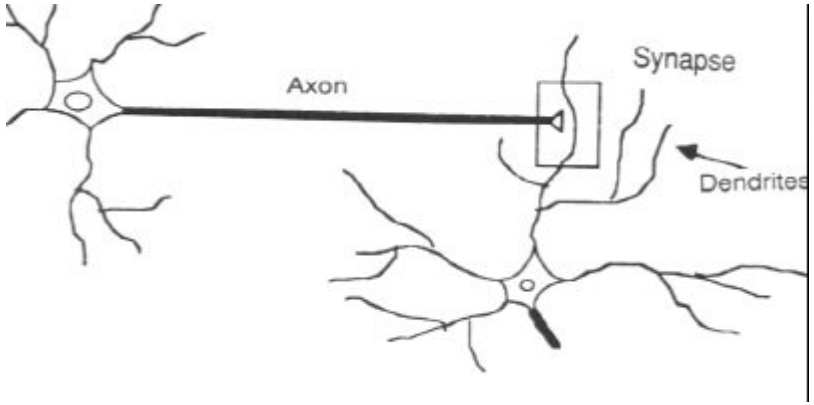
- Inductive Reasoning. Given input and output data (training examples), we construct the rules.
- Computation is collective, asynchronous, and parallel.
- Memory is distributed, internalized, short term and content addressable.
- Fault tolerant, redundancy, and sharing of responsibilities.
- Inexact.
- Dynamic connectivity.
- Applicable if rules are unknown or complicated, or if data are noisy or partial.

# Background

- An Artificial Neural Network (ANN) is an information processing paradigm that is inspired by the biological nervous systems, such as the human brain's information processing mechanism.
- The key element of this paradigm is the novel structure of the information processing system. It is composed of a large number of highly interconnected processing elements (neurons) working in unison to solve specific problems. NNs, like people, learn by example.
- An NN is configured for a specific application, such as pattern recognition or data classification, through a learning process. Learning in biological systems involves adjustments to the synaptic connections that exist between the neurons. This is true of NNs as well.

# How the Human Brain learns

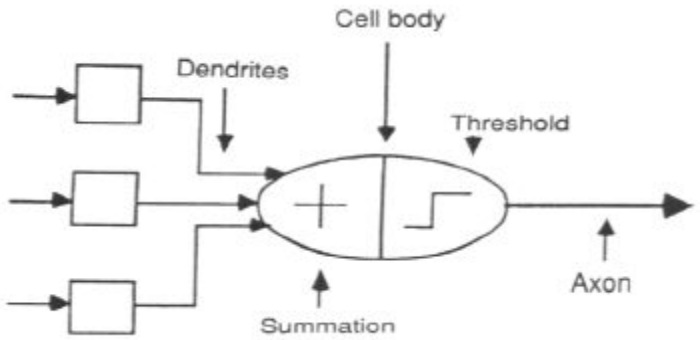



- In the human brain, a typical neuron collects signals from others through a host of fine structures called dendrites.
- The neuron sends out spikes of electrical activity through a long, thin stand known as an axon, which splits into thousands of branches.
- At the end of each branch, a structure called a synapse converts the activity from the axon into electrical effects that inhibit or excite activity in the connected neurons.

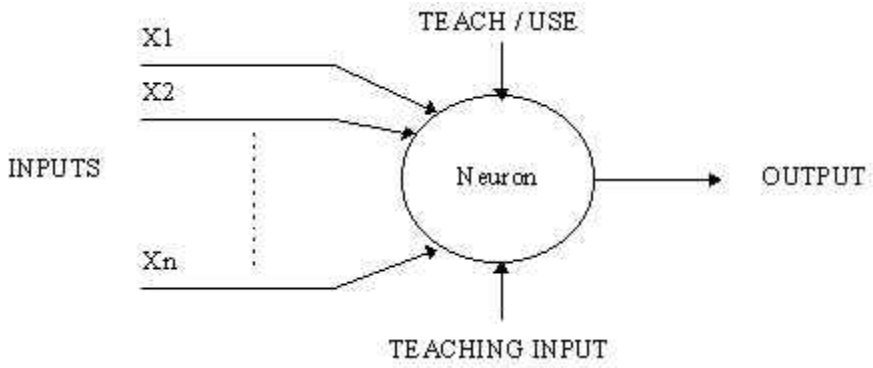
# A Neuron Model

- When a neuron receives excitatory input that is sufficiently large compared with its inhibitory input, it sends a spike of electrical activity down its axon. Learning occurs by changing the effectiveness of the synapses so that the influence of one neuron on another changes.




- We conduct these neural networks by first trying to deduce the essential features of neurons and their interconnections.
- We then typically program a computer to simulate these features.

# A Simple Neuron




- An artificial neuron is a device with many inputs and one output.
- The neuron has two modes of operation;
- the training mode and
- the using mode.

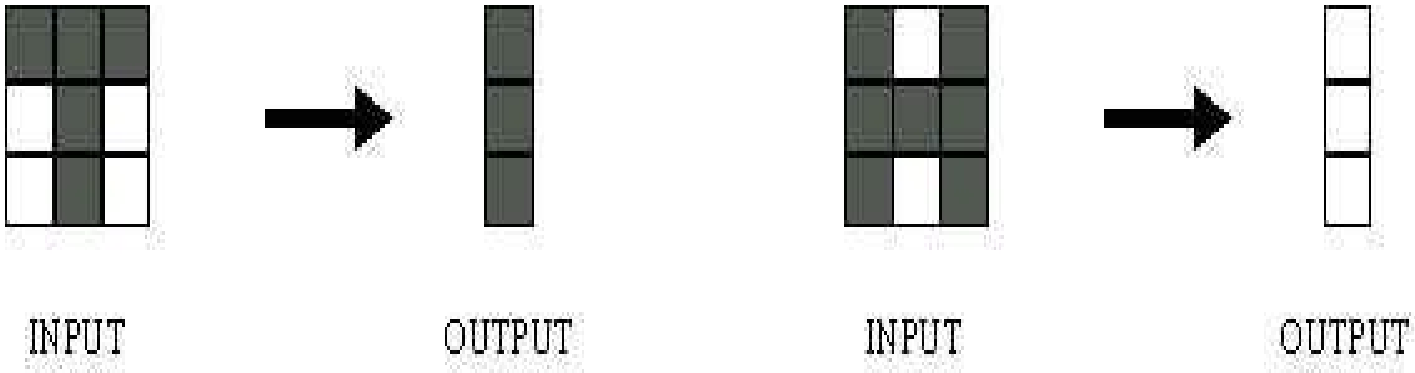
# A Simple Neuron (Cont.)

- In the training mode, the neuron can be trained to fire (or not), for particular input patterns.
- In the using mode, when a taught input pattern is detected at the input, its associated output becomes the current output. If the input pattern does not belong in the taught list of input patterns, the firing rule is used to determine whether to fire or not.
- The firing rule is an important concept in neural networks and accounts for their high flexibility. A firing rule determines how one calculates whether a neuron should fire for any input pattern. It relates to all the input patterns, not only the ones on which the node was trained on previously.

# Pattern Recognition

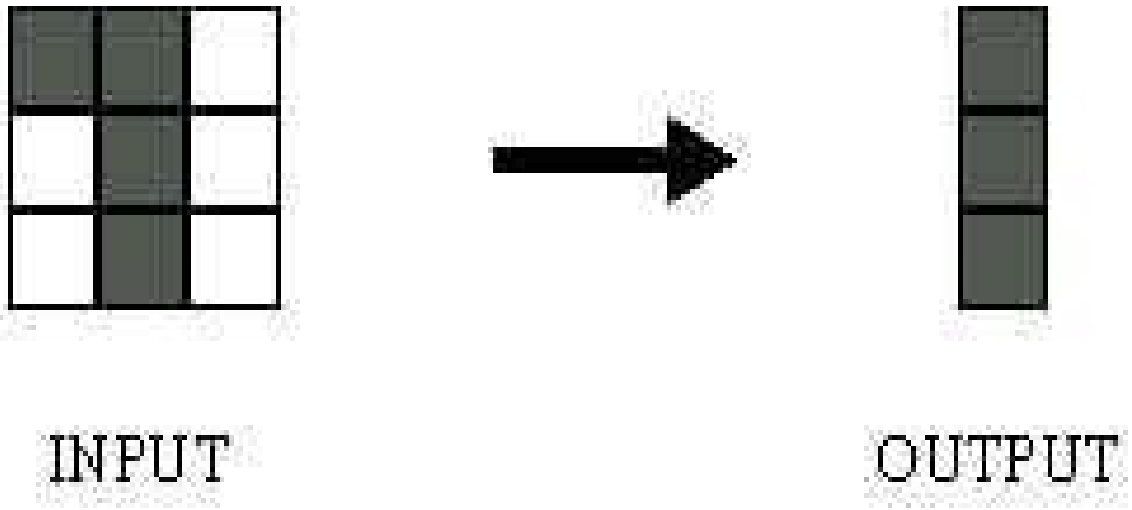
- An important application of neural networks is pattern recognition. Pattern recognition can be implemented by using a feed-forward neural network that has been trained accordingly.
- During training, the network is trained to associate outputs with input patterns. When the network is used, it identifies the input pattern and tries to output the associated output pattern.
- The power of neural networks comes to life when a pattern that has no output associated with it, is given as an input. In this case, the network gives the output that corresponds to a taught input pattern that is least different from the given pattern.

# Pattern Recognition (cont.)



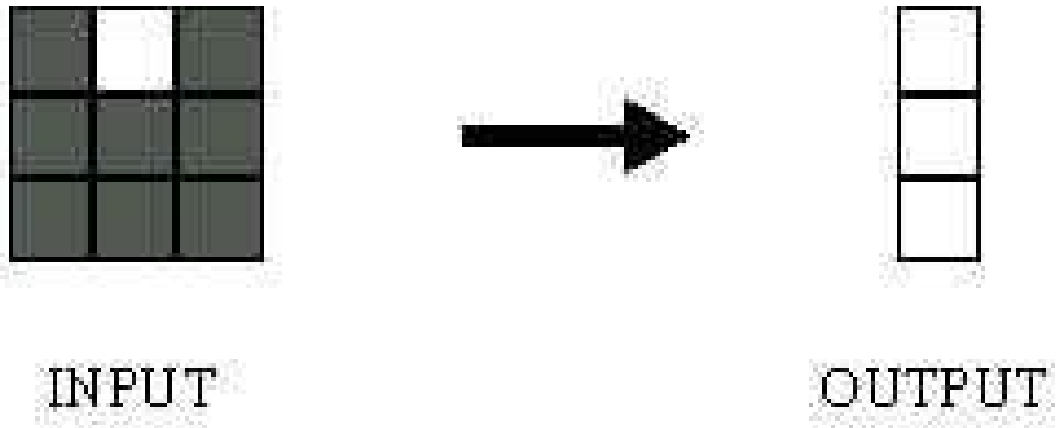

- Suppose a network is trained to recognize the patterns T and H. The associated patterns are all black and all white respectively as shown above.

# Pattern Recognition (cont.)



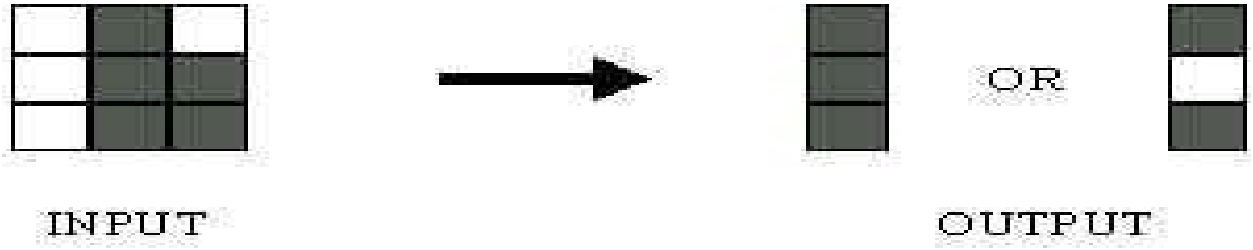

Since the input pattern looks more like a ‘T’, when the network classifies it, it sees the input closely resembling ‘T’ and outputs the pattern that represents a ‘T’.

# Pattern Recognition (cont.)



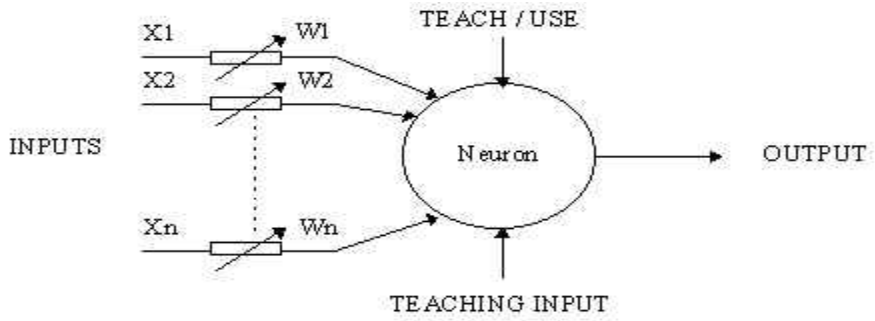

The input pattern here closely resembles ‘H’ with a slight difference. The network in this case classifies it as an ‘H’ and outputs the pattern representing an ‘H’.

# Pattern Recognition (cont.)




- Here the top row is 2 errors away from a ‘T’ and 3 errors away from an H. So the top output is a black.
- The middle row is 1 error away from both T and H, so the output is random.
- The bottom row is 1 error away from T and 2 away from H. Therefore the output is black.
- Since the input resembles a ‘T’ more than an ‘H’ the output of the network is in favor of a ‘T’.

# A Complicated Perceptron




- A more sophisticated Neuron is know as the McCulloch and Pitts model (MCP).
- The difference is that in the MCP model, the inputs are weighted and the effect that each input has at decision making, is dependent on the weight of the particular input.
- The weight of the input is a number which is multiplied with the input to give the weighted input.

# A Complicated Perceptron (cont.)

- The weighted inputs are then added together and if they exceed a pre-set threshold value, the perceptron / neuron fires.
- Otherwise it will not fire and the inputs tied to that perceptron will not have any effect on the decision making.
- In mathematical terms, the neuron fires if and only if;

$$X1W1 + X2W2 + X3W3 + ... > T$$

# A Complicated Perceptron

- The MCP neuron has the ability to adapt to a particular situation by changing its weights and/or threshold.
- Various algorithms exist that cause the neuron to 'adapt'; the most used ones are the Delta rule and the back error propagation.

# Different types of Neural Networks

- **Feed-forward networks**
  - Feed-forward NNs allow signals to travel one way only; from input to output. There is no feedback (loops) i.e. the output of any layer does not affect that same layer.
  - Feed-forward NNs tend to be straight forward networks that associate inputs with outputs. They are extensively used in pattern recognition.
  - This type of organization is also referred to as bottom-up or top-down.

# Contd….

- **Feedback networks**
  - Feedback networks can have signals traveling in both directions by introducing loops in the network.
  - Feedback networks are dynamic; their 'state' is changing continuously until they reach an equilibrium point. They remain at the equilibrium point until the input changes and a new equilibrium needs to be found.
  - Feedback architectures are also referred to as interactive or recurrent, although the latter term is often used to denote feedback connections in single-layer organizations.

# Diagram of an NN

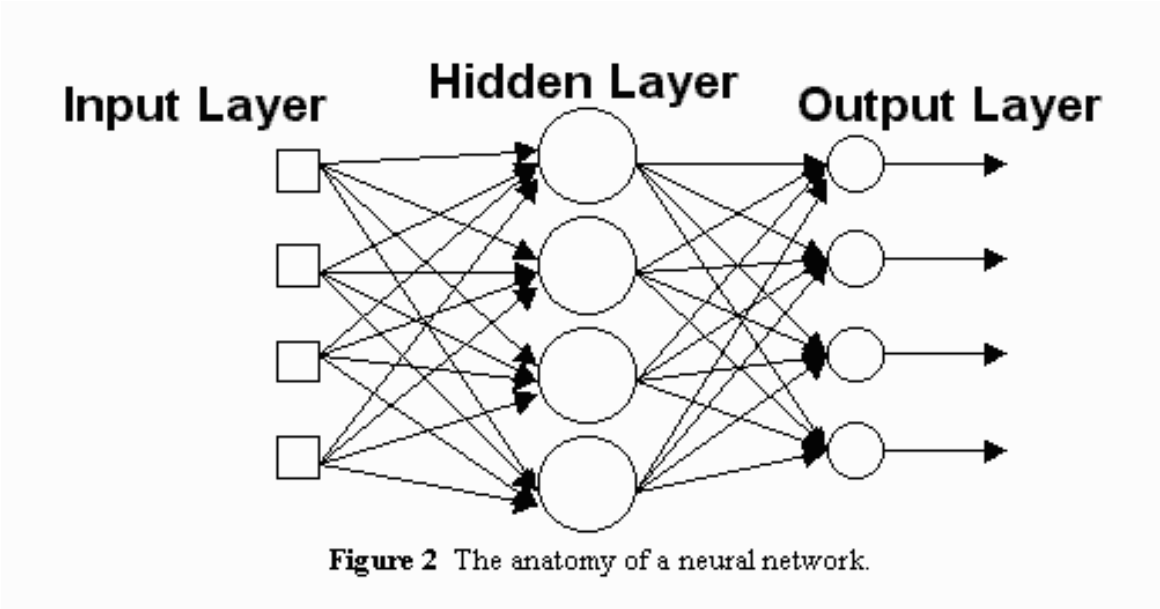



**Figure 2** The anatomy of a neural network.

Fig: A simple Neural Network

# Network Layers

- Input Layer - The activity of the input units represents the raw information that is fed into the network.
- Hidden Layer - The activity of each hidden unit is determined by the activities of the input units and the weights on the connections between the input and the hidden units.
- Output Layer - The behavior of the output units depends on the activity of the hidden units and the weights between the hidden and output units.

# Contd…

- This simple type of network is interesting because the hidden units are free to construct their own representations of the input.
- The weights between the input and hidden units determine when each hidden unit is active, and so by modifying these weights, a hidden unit can choose what it represents.

# Network Structure

- The number of layers and of neurons depend on the specific task. In practice this issue is solved by trial and error.
- Two types of adaptive algorithms can be used:
  - start from a large network and successively remove some neurons and links until network performance degrades.
  - begin with a small network and introduce new neurons until performance is satisfactory.

# Perceptrons

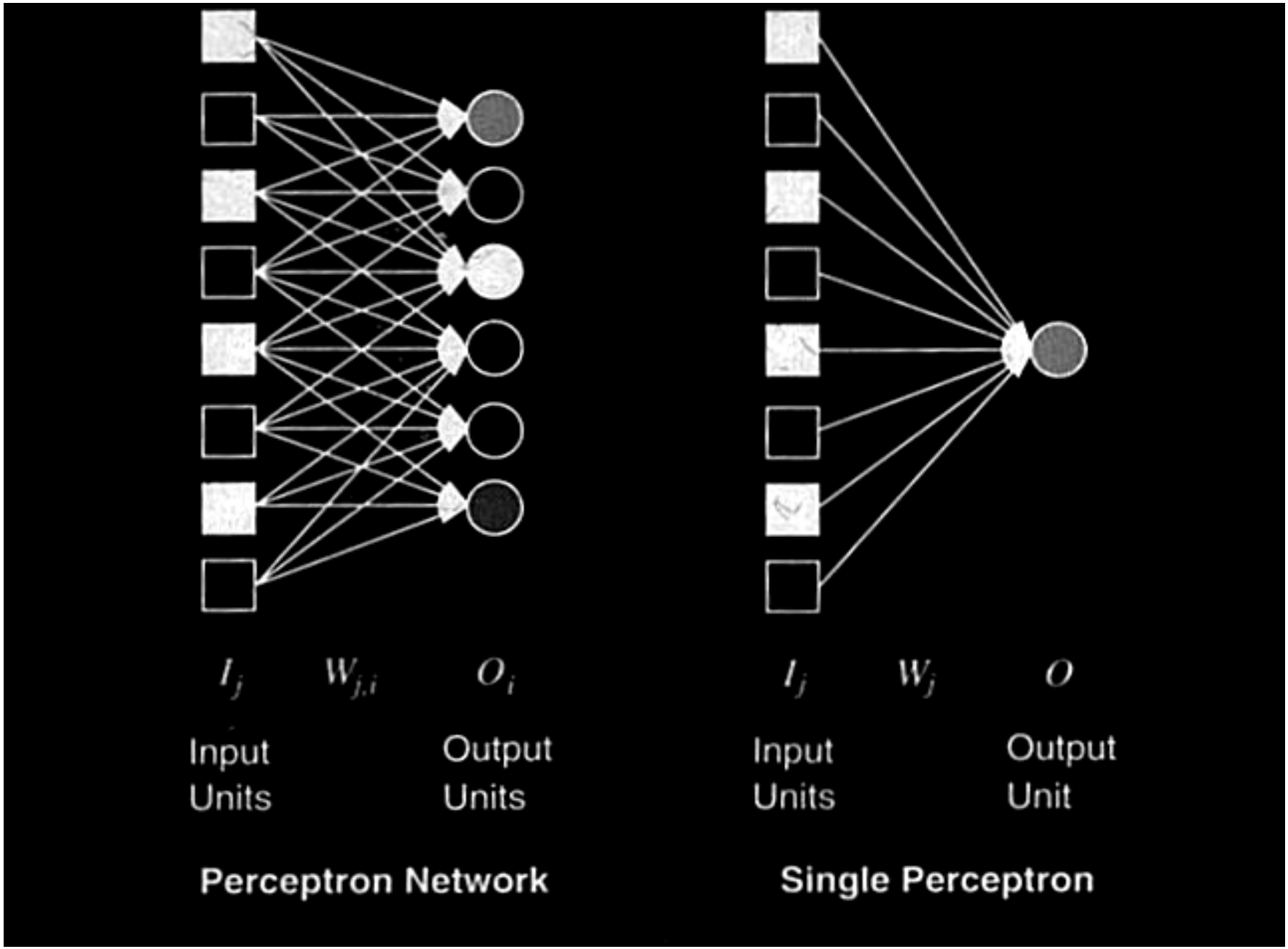
- First studied in the late 1950s.

- Also known as Layered Feed-Forward Networks.

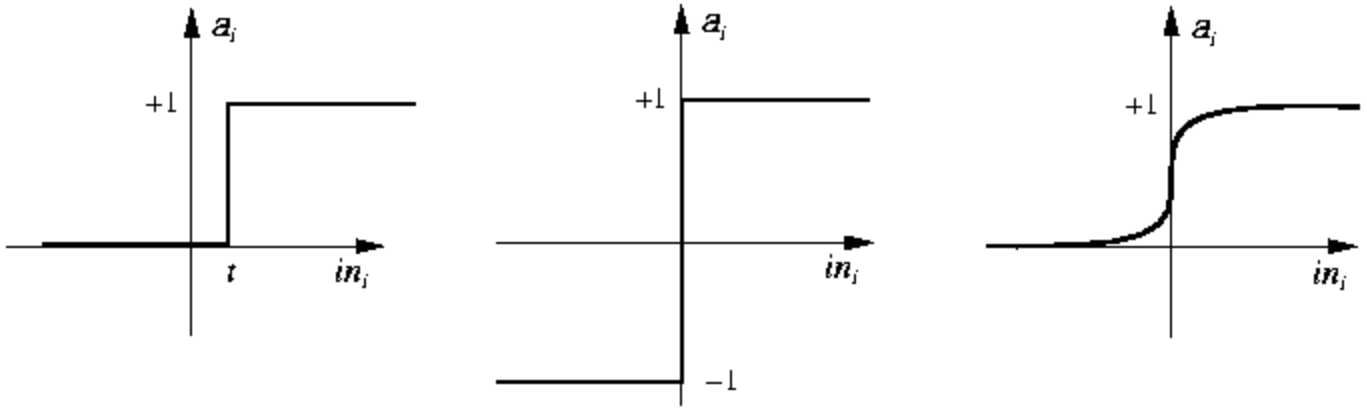
- The only efficient learning element at that time was for single-layered networks.

- Today, used as a synonym for a single-layer, feed-forward network.

# Perceptrons



# Sigmoid Perceptron




(a) Step function (b) Sign function (c) Sigmoid function

# Perceptron learning rule

- Teacher specifies the desired output for a given input
- Network calculates what it thinks the output should be
- Network changes its weights in proportion to the error

between the desired & calculated results

$\Delta w_{i,j} = \alpha * [teacher_i - output_i] * input_j$

- where:
- $\alpha$ is the learning rate;
- $teacher_i - output_i$ is the error term;
- and $input_j$ is the input activation

$w_{i,j} = w_{i,j} + \Delta w_{i,j}$ Delta rule

# Adjusting perceptron weights

- $\Delta w_{i,j} = \alpha * [teacher_i - output_i] * input_j$
- $miss_i$ is $(teacher_i - output_i)$

| | miss<0 | miss=0 | miss>0 |
|---|---|---|---|
| input < 0 | alpha | 0 | -alpha |
| input = 0 | 0 | 0 | 0 |
| input > 0 | -alpha | 0 | alpha |

- Adjust each wi,j based on inputj and missi
- The above table shows adaptation.
- Incremental learning.

# Node biases

- A node’s output is a weighted function of its inputs

- What is a bias?
- How can we learn the bias value?

- Answer: treat them like just another weight

# Training biases (Θ)

- A node's output:
  - 1 if $w_1x_1 + w_2x_2 + \ldots + w_nx_n >= \Theta$ ← bias
  - 0 otherwise
- Rewrite
  - $w_1x_1 + w_2x_2 + \ldots + w_nx_n - \Theta >= 0$
  - $w_1x_1 + w_2x_2 + \ldots + w_nx_n + \Theta(-1) >= 0$

- Hence, the bias is just another weight whose activation is always -1

- Just add one more input unit to the network topology

# Perceptron convergence theorem

- If a set of <input, output> pairs are learnable (representable), the delta rule will find the necessary weights
  - in a finite number of steps
  - independent of initial weights

- However, a single layer perceptron can only learn linearly separable concepts
  - it works **iff** gradient descent works

# Linear separability

- Consider a perceptron
- Its output is
  - 1, if $W_1X_1 + W_2X_2 > \Theta$
  - 0, otherwise
- In terms of feature space
  - hence, it can only classify examples if a line (hyperplane more generally) can separate the positive examples from the negative examples

# What can Perceptrons Represent ?

- Some complex Boolean function can be represented.

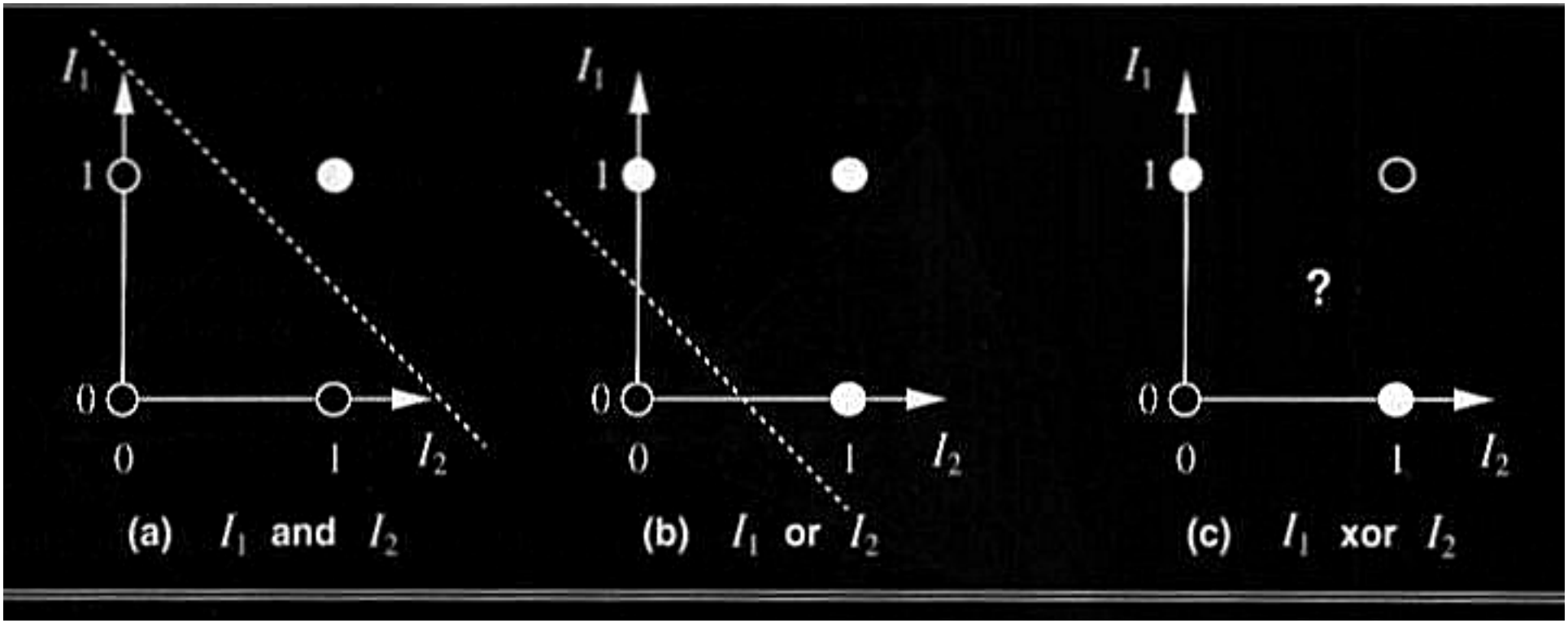
**For example:**

Majority function - will be covered in this lecture.

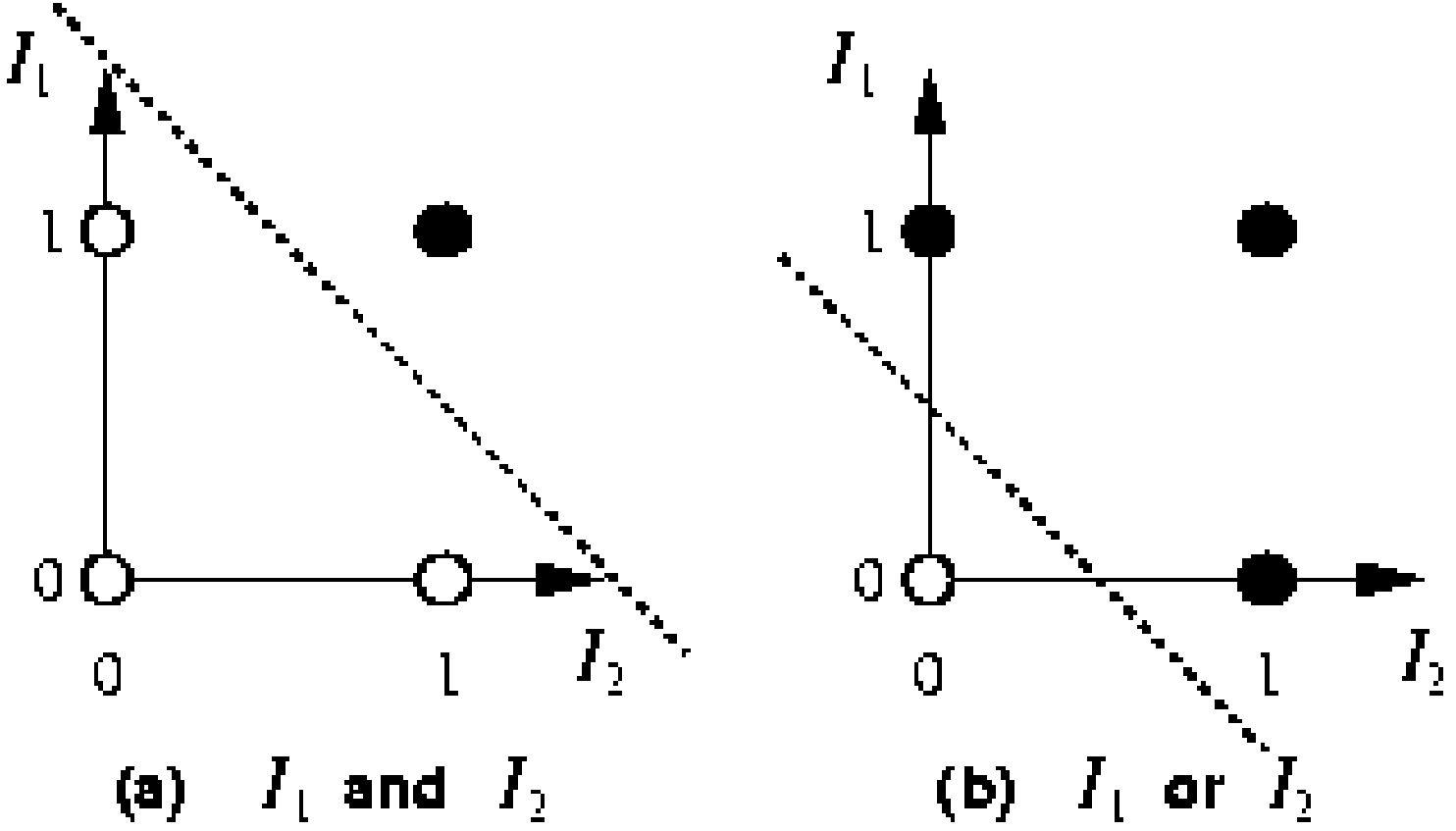
- Perceptrons are limited in the Boolean functions they can represent.

# The Separability Problem and EXOR trouble

Linear Separability in Perceptrons

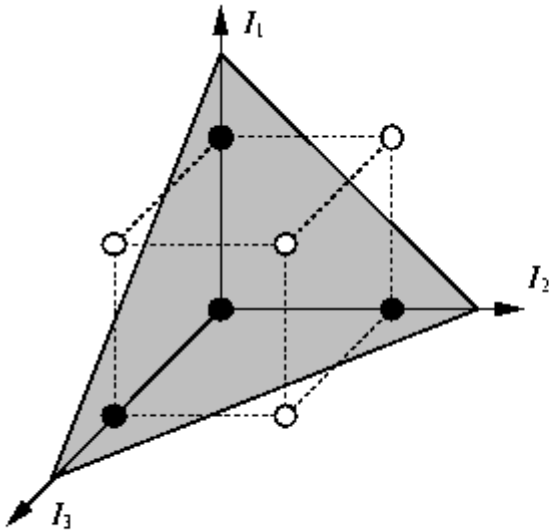


# AND and OR linear Separators

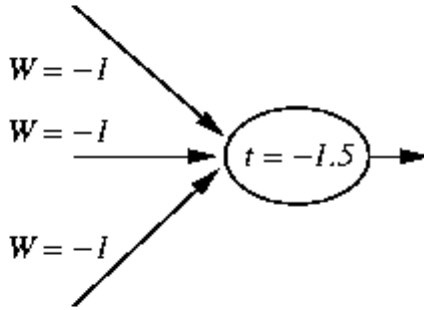


# Separation in n-1 dimensions

majority




(a) Separating plane




(b) Weights and threshold

Example of 3Dimensional space

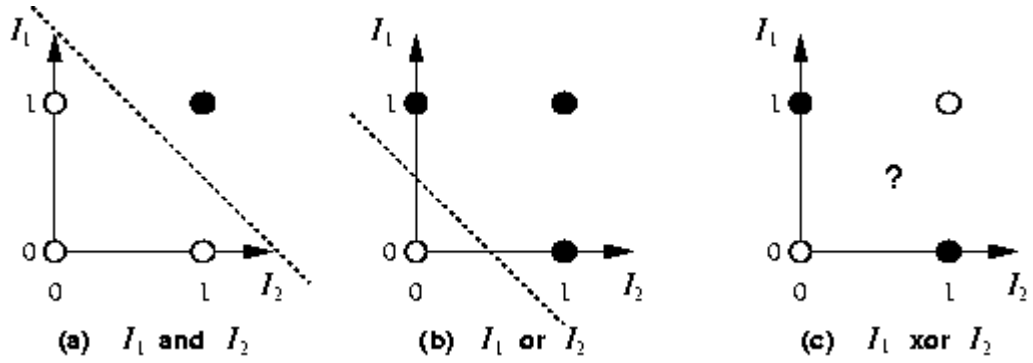
# Perceptrons & XOR

- XOR function

| Input1 | Input2 | Output |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

  - no way to draw a line to separate the positive from negative examples

# How do we compute XOR?



# Learning Linearly Separable Functions (1)

What can these functions learn ?

Bad news:

- There are not many linearly separable functions.

Good news:

- There is a perceptron algorithm that will learn any linearly separable function, given enough training examples.

# Learning Linearly Separable Functions (2)

Most neural network learning algorithms, including the perceptrons learning method, follow the current-best-hypothesis (CBH) scheme.

# Learning Linearly Separable Functions

-Initial network has a randomly assigned weights.
- Learning is done by making small adjustments in the weights to reduce the difference between the observed and predicted values.
-Main difference from the logical algorithms is the need to repeat the update phase several times in order to achieve convergence.
-Updating process is divided into epochs.
-Each epoch updates all the weights of the process.

# Weights

- In general, initial weights are randomly chosen, with typical values between -1.0 and 1.0 or -0.5 and 0.5.
- There are two types of NNs. The first type is known as
  - Fixed Networks – where the weights are fixed
  - Adaptive Networks – where the weights are changed to reduce prediction error.

# Size of Training Data

- Rule of thumb:
  - the number of training examples should be at least five to ten times the number of weights of the network.

- Other rule:

$$N > \frac{|W|}{(1-a)}$$

|W|= number of weights

a = expected accuracy on test set

# Training Basics

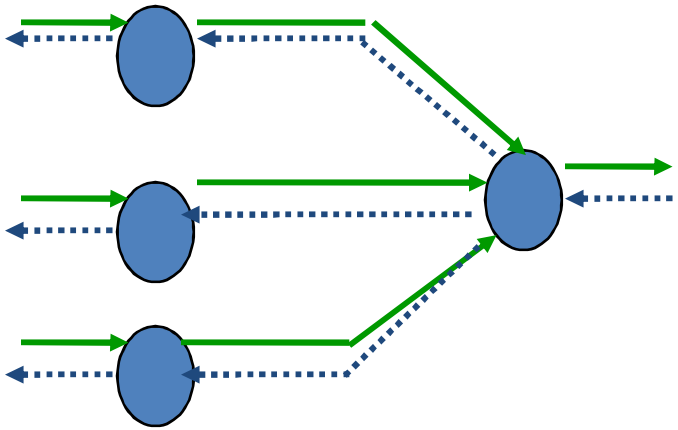
- The most basic method of training a neural network is trial and error.
- If the network isn't behaving the way it should, change the weighting of a random link by a random amount. If the accuracy of the network declines, undo the change and make a different one.
- It takes time, but the trial and error method does produce results.

# Training: Back prop algorithm

- The Backprop algorithm searches for weight values that minimize the total error of the network over the set of training examples (training set).
- Backprop consists of the repeated application of the following two passes:
  - **Forward pass**: in this step the network is activated on one example and the error of (each neuron of) the output layer is computed.
  - **Backward pass**: in this step the network error is used for updating the weights. Starting at the output layer, the error is propagated backwards through the network, layer by layer. This is done by recursively computing the local gradient of each neuron.

# Back Propagation

- Back-propagation training algorithm



*Network activation Forward Step*

*Error propagation Backward Step*

- Backprop adjusts the weights of the NN in order to minimize the network total mean squared error.

# Applications of Back Propagation Network

1. **Classification**

   In this classification problem, the goal is to identify whether a certain "data point" belongs to Class 1, 2, or 3 (see above). Random points are assigned to a certain class, and the neural network is trained to find the pattern. When training is complete, it will use what it has learned to accurately classify new points.

2. **Function Approximation**

   In this problem, the network tries to approximate the value of a certain function.

It is fed with noisy data, and the goal is to find the true pattern. After training, the network successfully estimates the value of the gaussian function (below).

**3. Time-series Prediction**

In this problem, the goal is to design a neural network to predict a value based on a given time-series data (i.e. stock market prediction based on given trends). To approach this problem, the inputs to the neural network have to be refactored in chunks, and the resulting output will be the next data item directly following that chunk

# The Learning Process

- The memorization of patterns and the subsequent response of the network can be categorized into two general paradigms:
  - Associative mapping
  - Regularity detection

# Associative Mapping

- **Associative mapping** a type of NN in which the network learns to produce a particular pattern on the set of input units whenever another particular pattern is applied on the set of input units.
- This allows the network to complete a pattern given parts of a pattern that is similar to a previously learned pattern.

# Regularity Detection

- Regularity detection is a type of NN in which units learn to respond to particular properties of the input patterns. Whereas in associative mapping the network stores the relationships among patterns.
- In regularity detection the response of each unit has a particular 'meaning'. This means that the activation of each unit corresponds to different input attributes.

# The Learning Process (cont.)

- Every neural network possesses knowledge which is contained in the values of the connection weights.
- Modifying the knowledge stored in the network as a function of experience implies a learning rule for changing the values of the weights.

# The Learning Process (cont.)

- Recall: Adaptive networks are NNs that allow the change of weights in its connections.
- The learning methods can be classified in two categories:
  - Supervised Learning
  - Unsupervised Learning

# Supervised Learning

- Supervised learning which incorporates an external teacher, so that each output unit is told what its desired response to input signals ought to be.
- An important issue concerning supervised learning is the problem of error convergence, ie the minimization of error between the desired and computed unit values.
- The aim is to determine a set of weights which minimizes the error. One well-known method, which is common to many learning paradigms is the least mean square (LMS) convergence.

# Supervised Learning

- In this sort of learning, the human teacher's experience is used to tell the NN which outputs are correct and which are not.
- This does not mean that a human teacher needs to be present at all times, only the correct classifications gathered from the human teacher on a domain needs to be present.
- The network then learns from its error, that is, it changes its weight to reduce its prediction error.

# Unsupervised Learning

- Unsupervised learning uses no external teacher and is based upon only local information. It is also referred to as self-organization, in the sense that it self-organizes data presented to the network and detects their emergent collective properties.
- The network is then used to construct clusters of similar patterns.
- This is particularly useful is domains were a instances are checked to match previous scenarios. For example, detecting credit card fraud.

# Neural Network in Use

Since neural networks are best at identifying patterns or trends in data, they are well suited for prediction or forecasting needs including:

- sales forecasting
- industrial process control
- customer research
- data validation
- risk management

# Disadvantage of Neural Network

- The individual relations between the input variables and the output variables are not developed by engineering judgment so that the model tends to be a black box or input/output table without analytical basis.
- The sample size has to be large.
- Requires lot of trial and error so training can be time consuming.

# Applications

- **classification**

  in marketing: consumer spending pattern classification

  In defence: radar and sonar image classification

  In agriculture & fishing: fruit and catch grading

  In medicine: ultrasound and electrocardiogram image classification, EEGs, medical diagnosis

**Recognition and Identification**

In general computing and telecommunications: speech, vision and handwriting recognition

In finance: signature verification and bank note verification assessment

In engineering: product inspection monitoring and control

In defence: target tracking

In security: motion detection, surveillance image analysis and fingerprint matching forecasting and prediction

In finance: foreign exchange rate and stock market forecasting

In agriculture: crop yield forecasting

In marketing: sales forecasting

# UNIT-II
# STATISTICAL METHODS

- Hopfield nets
- Cauchy training
- Simulated annealing
- The Boltzmann machine
- Associative memory
- Bidirectional associative memory -applications.

# Hopfield Nets

- A Hopfield net is composed of binary threshold units with recurrent connections between them. Recurrent networks of non-linear units are generally very hard to analyze. They can behave in many different ways:
  - Settle to a stable state,Oscillate,Follow chaotic trajectories that cannot be predicted far into the future.
- But Hopfield realized that if the connections are symmetric, there is a global energy function
  - Each “configuration” of the network has an energy.
  - The binary threshold decision rule causes the network to settle to an energy minimum.

# The energy function

- The global energy is the sum of many contributions. Each contribution depends on one connection weight and the binary states of two neurons:

$$E = -\sum_i s_i b_i \; - \sum_{i<j} s_i s_j w_{ij}$$

- The simple quadratic energy function makes it easy to compute how the state of one neuron affects the global energy:

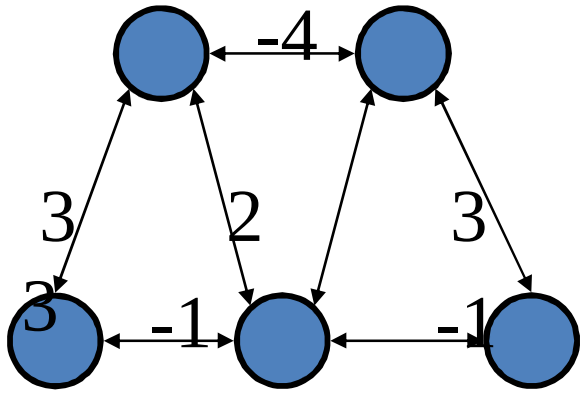
$$E(s_i = 0) - E(s_i = 1) \quad = \quad b_i + \sum_j s_j w_{ij}$$

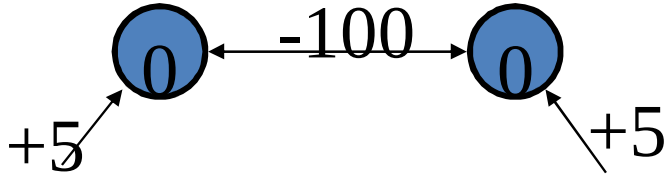
# Settling to an energy minimum

- Pick the units one at a time and flip their states if it reduces the global energy.

  Find the minima in this net

- If units make simultaneous decisions the energy could go up.



# How to make use of this type of computation

- Hopfield proposed that memories could be energy minima of a neural net.
- The binary threshold decision rule can then be used to "clean up" incomplete or corrupted memories.
  - This gives a content-addressable memory in which an item can be accessed by just knowing part of its content (like google)
  - It is robust against hardware damage.

# Storing memories

- If we use activities of 1 and -1, we can store a state vector by incrementing the weight between any two units by the product of their activities.
  - Treat biases as weights from a permanently on unit

$$\Delta w_{ij} = s_i s_j$$

- With states of 0 and 1 the rule is slightly more complicated.

$$\Delta w_{ij} = 4\,(s_i - \tfrac{1}{2})(s_j - \tfrac{1}{2})$$

# Spurious minima

- Each time we memorize a configuration, we hope to create a new energy minimum.

  But what if two nearby minima merge to create a minimum at an intermediate location?

  This limits the capacity of a Hopfield net.
- Using Hopfield's storage rule the capacity of a totally connected net with N units is only 0.15N memories.
  - This does not make efficient use of the bits required to store the weights in the network.

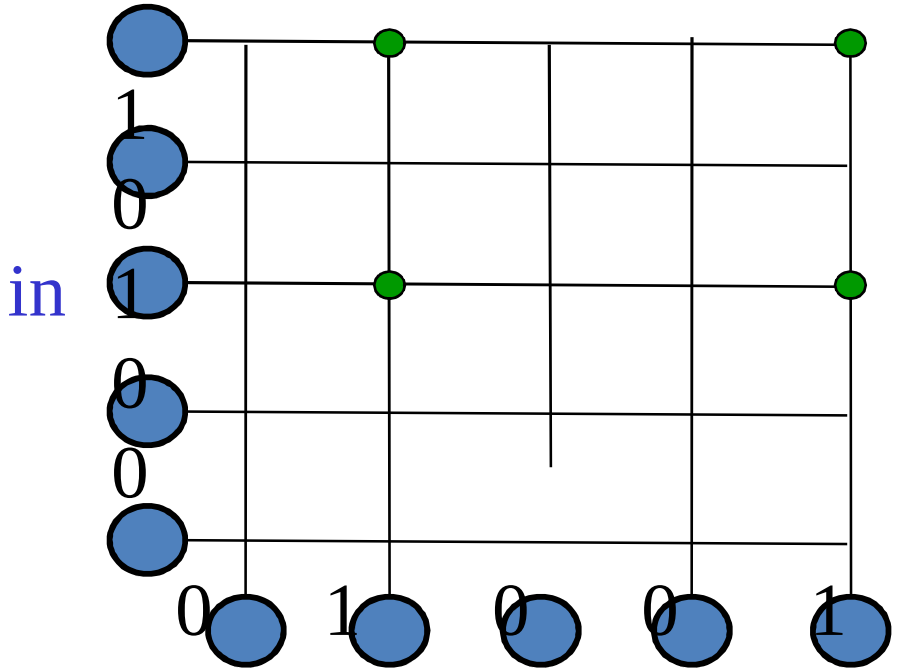
# Avoiding spurious minima by unlearning

- Hopfield, Feinstein and Palmer suggested the following strategy:
  - Let the net settle from a random initial state and then do unlearning.
  - This will get rid of deep , spurious minima and increase memory capacity.
- Crick and Mitchison proposed unlearning as a model of what dreams are for.
  - That's why you don't remember them

    (Unless you wake up during the dream)
- But how much unlearning should we do?
  - And can we analyze what unlearning achieves?

# Willshaw nets

- We can improve efficiency by using sparse vectors and only allowing one bit per weight.
  - Turn on a synapse when input and output units are both active.
- For retrieval, set the output threshold equal to the number of active input units
  - This makes false positives improbable



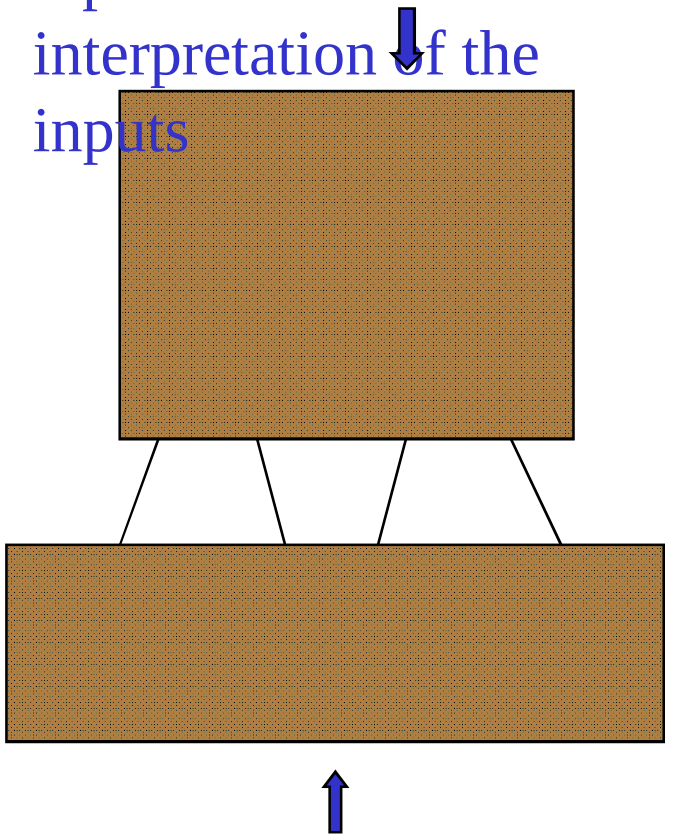

output units with dynamic thresholds

# An iterative storage method

- Instead of trying to store vectors in one shot as Hopfield does, cycle through the training set many times.
  - use the perceptron convergence procedure to train each unit to have the correct state given the states of all the other units in that vector.
  - This uses the capacity of the weights efficiently.

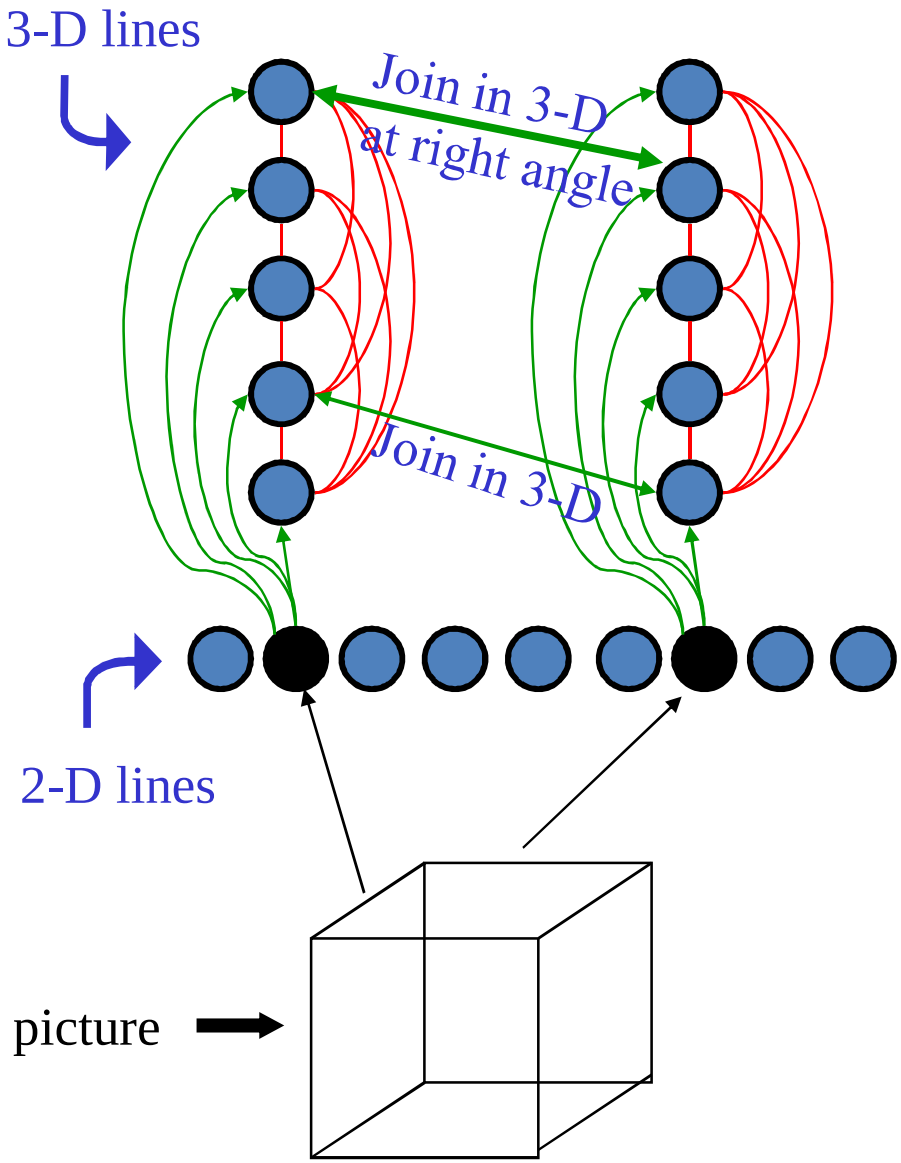
# Another computational role for Hopfield nets

- Instead of using the net to store memories, use it to construct interpretations of sensory input.
  - The input is represented by the visible units.
  - The interpretation is represented by the states of the hidden units.
  - The badness of the interpretation is represented by the energy
- This raises two difficult issues:
  - How do we escape from poor local minima to get good interpretations?
  - How do we learn the weights on connections to the hidden units?



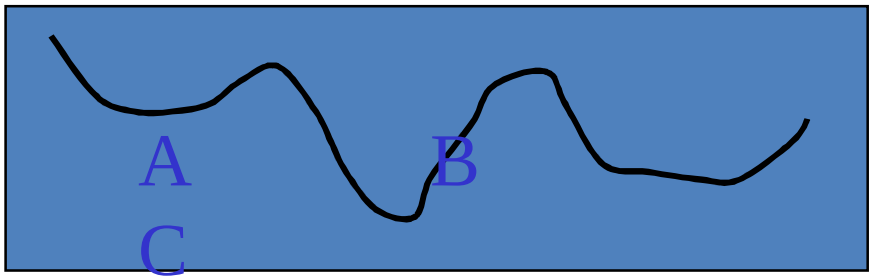
# An example: Interpreting a line drawing

- Use one “2-D line” unit for each possible line in the picture.
  - Any particular picture will only activate a very small subset of the line units.
- Use one “3-D line” unit for each possible 3-D line in the scene.
  - Each 2-D line unit could be the projection of many possible 3-D lines. Make these 3-D lines compete.
- Make 3-D lines support each other if they join in 3-D. Make them **strongly support** each other if they join at right angles.



# Noisy networks find better energy minima

- A Hopfield net always makes decisions that reduce the energy.
  - This makes it impossible to escape from local minima.
- We can use random noise to escape from poor minima.
  - Start with a lot of noise so its easy to cross energy barriers.
  - Slowly reduce the noise so that the system ends up in a deep minimum. This is “simulated annealing”.



# Stochastic units

- Replace the binary threshold units by binary stochastic units that make biased random decisions.
  - The "temperature" controls the amount of noise
  - Decreasing all the energy gaps between configurations is equivalent to raising the noise level.

$$p(s_i{=}1) \;=\; \frac{1}{1+e^{-\sum_j s_j w_{ij}/T}} \;=\; \frac{1}{1+e^{-\Delta E_i/T}}$$

temperatu
re

$$\textit{Energy gap} \;=\; \Delta E_i \;=\; E(s_i{=}0) - E(s_i{=}1)$$

# The annealing trade-off

- At high temperature the transition probabilities for uphill jumps are much greater.

$$p(\textit{pick higher energy state}) = \frac{1}{1+e^{\Delta E/T}}$$

Energy increase

- At low temperature the equilibrium probabilities of good states are much better than the equilibrium probabilities of bad ones.

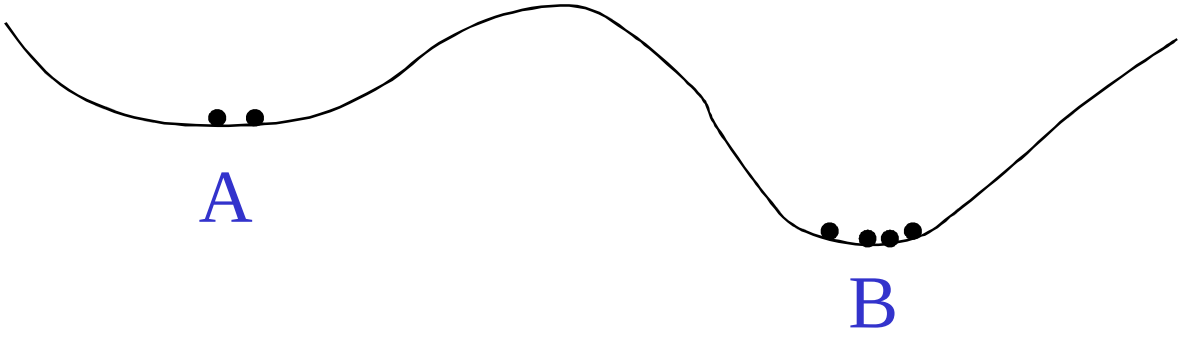
$$\frac{P_A}{P_B} = e^{-\frac{(E_A - E_B)}{T}}$$

# How temperature affects transition probabilities

High temperature transition probabilities

$p(A \rightarrow B) = 0.2$

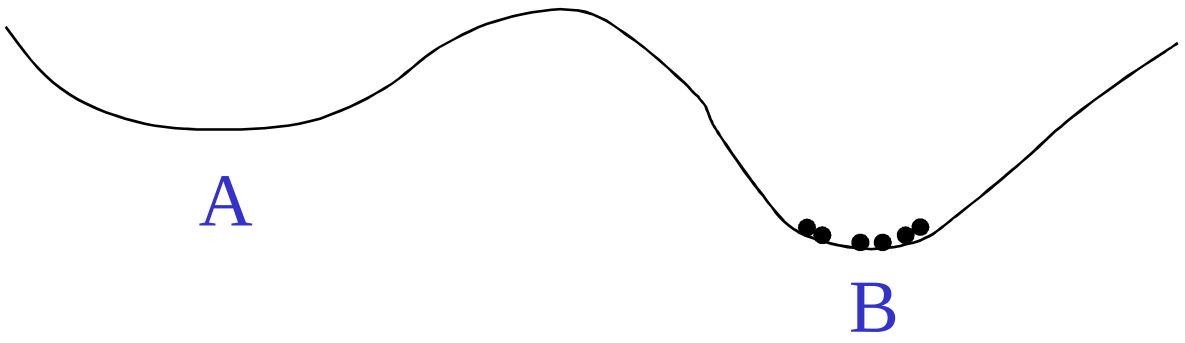
$p(A \leftarrow B) = 0.1$




Low temperature transition probabilities

$p(A \rightarrow B) = 0.001$

$p(A \leftarrow B) = 0.000001$



# Thermal equilibrium

- Thermal equilibrium is a difficult concept!
  - It does not mean that the system has settled down into the lowest energy configuration.
  - The thing that settles down is the probability distribution over configurations.
- The best way to think about it is to imagine a huge ensemble of systems that all have exactly the same energy function.
  - After running the systems stochastically in the right way, we eventually reach a situation where the number of systems in each configuration remains constant even though any given system keeps moving between configurations

# Simulated Annealing

*Step 1: Initialize* – Start with a random initial placement. Initialize a very high “temperature”.

*Step 2: Move* – Perturb the placement through a defined move.

*Step 3: Calculate score* – calculate the change in the score due to the move made.

*Step 4: Choose* – Depending on the change in score, accept or reject the move. The prob of acceptance depending on the current “temperature”.

*Step 5: Update and repeat*– Update the temperature value by lowering the temperature. Go back to Step 2.

The process is done until “Freezing Point” is reached.

# Algorithm for placement

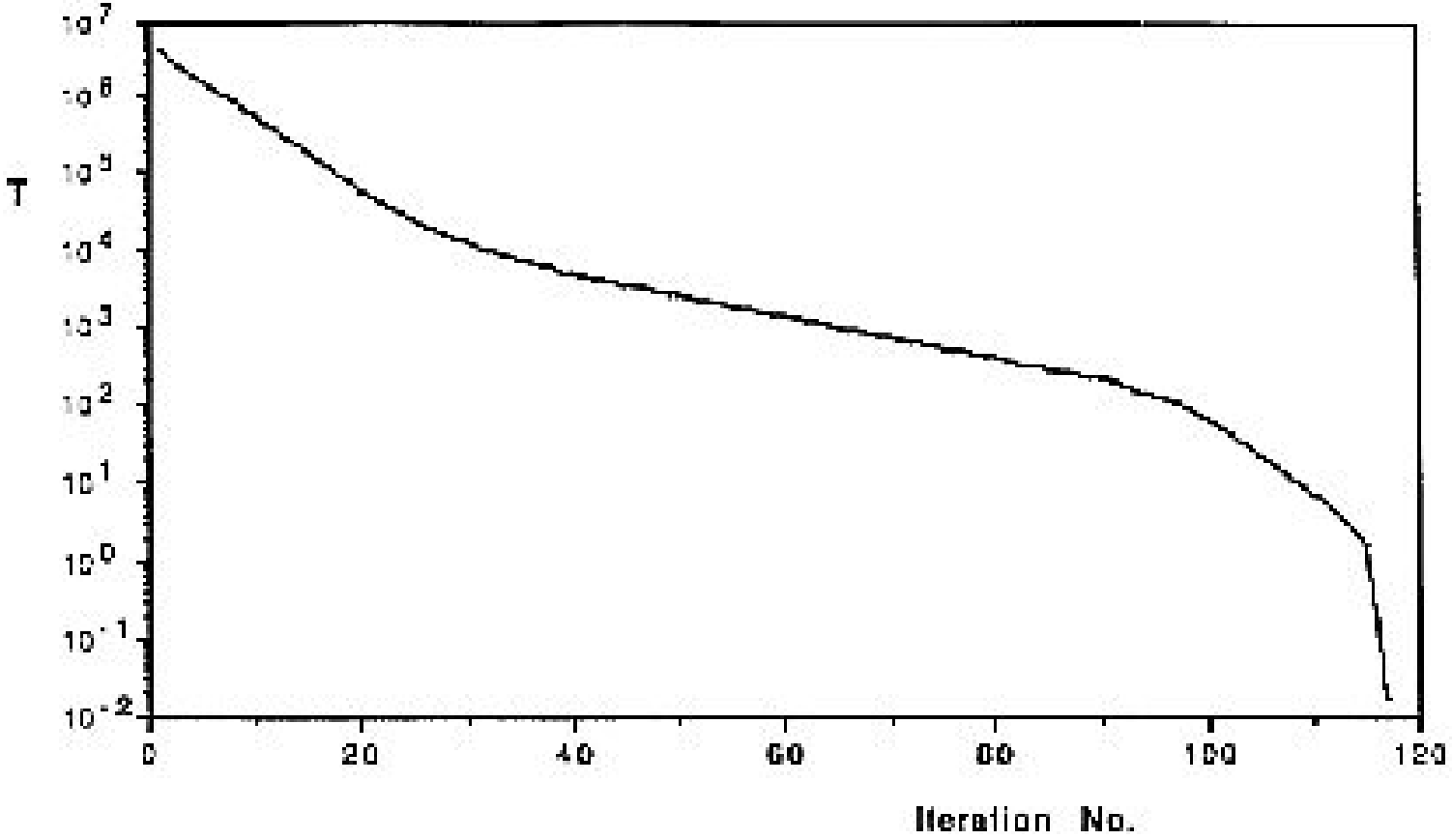
**Algorithm** SIMULATED-ANNEALING
**Begin**
    *temp* = INIT-TEMP;
    *place* = INIT-PLACEMENT;
    **while** (*temp* > FINAL-TEMP) **do**
        **while** (*inner_loop_criterion* = FALSE) **do**
            *new_place* = PERTURB(*place*);
            $\Delta C$ = COST(*new_place*) - COST(*place*);
            **if** ($\Delta C < 0$) **then**
                *place = new_place*;
            **else if** (RANDOM(0,1) > $e^{-(\Delta C/temp)}$) **then**
                *place = new_place*;
        *temp* = SCHEDULE(*temp*);
**End.**

# Parameters

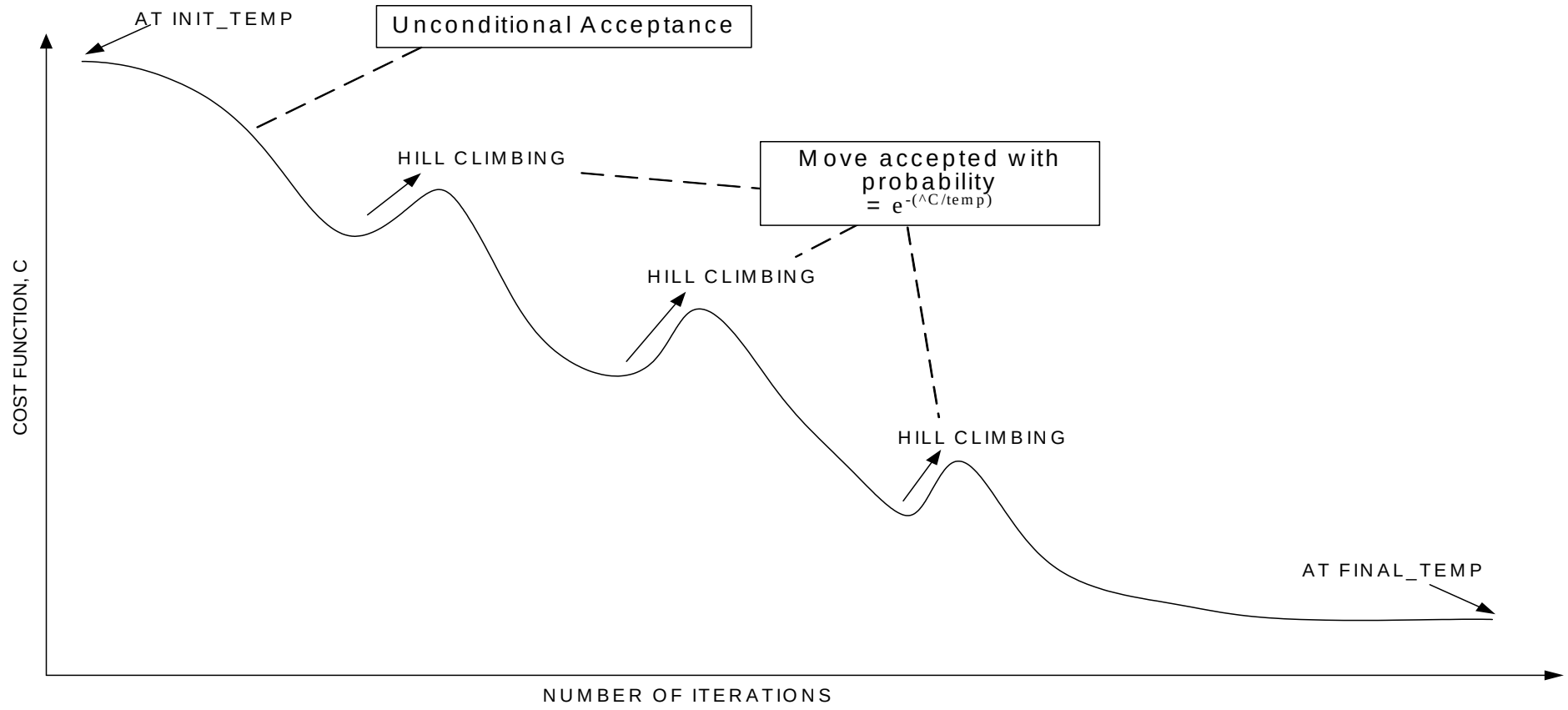
- INIT-TEMP = 4000000;
- INIT-PLACEMENT = Random;
- PERTURB(*place*)

  1. Displacement of a block to a new position.
  2. Interchange blocks.
  3. Orientation change for a block.
- SCHEDULE.

# Cooling schedule



# Convergence of simulated annealing



# Algorithm for partitioning

**Algorithm** SA
**Begin**
    $t = t_0$;
    *cur_part = ini_part;*
    *cur_score* = SCORE*(cur_part);*
    **repeat**
        **repeat**
            *comp1* = SELECT*(part1);*
            *comp2* = SELECT*(part2);*
            *trial_part* = EXCHANGE*(comp1, comp2, cur_part);*
            *trial_score* = SCORE*(trial_part);*
            $\delta s$ = *trial_score – cur_score;*
            **if** ($\delta s < 0$) **then**
                *cur_score = trial_score;*
                *cur_part* = MOVE*(comp1, comp2);*
            **else**
                *r* = RANDOM(0,1);
                **if** ($r < e^{-(\delta s/t)}$) **then**
                    *cur_score = trial_score;*
                    *cur_part* = MOVE*(comp1, comp2);*
        **until** (equilibrium at t is reached)
        $t = \alpha t$ $(0 < \alpha < 1)$
    **until** (freezing point is reached)
**End.**

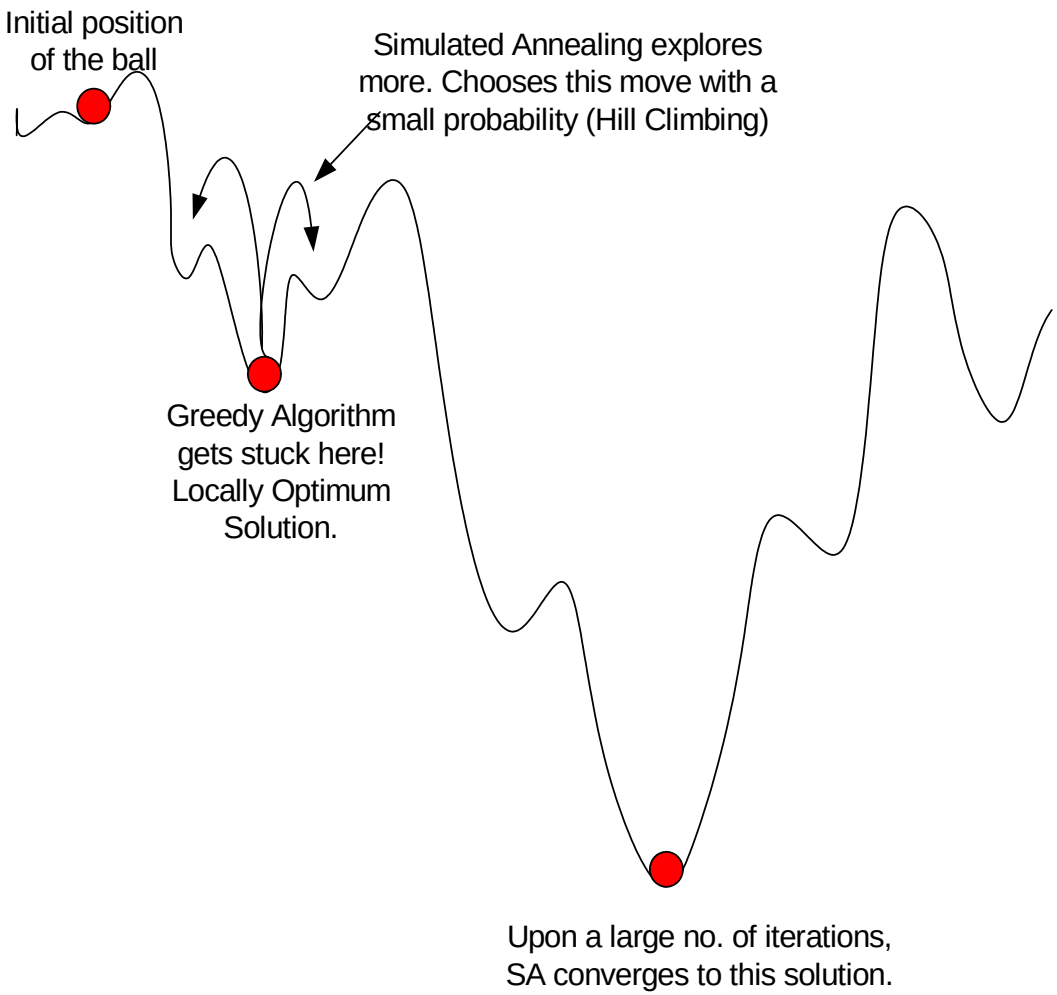
# Qualitative Analysis

- Randomized local search.
- Is simulated annealing greedy?
- Controlled greed.
- Once-a-while exploration.
- Is a greedy algorithm better? Where is the difference?
- The ball-on-terrain example.

# Ball on terrain example – Simulated Annealing vs Greedy Algorithms

The ball is initially placed at a random position on the terrain. From the current position, the ball should be fired such that it can only move one step left or right.What algorithm should we follow for the ball to finally settle at the lowest point on the terrain?

# Ball on terrain example – SA vs Greedy Algorithms



# Applications

- Circuit partitioning and placement.
- Strategy scheduling for capital products with complex product structure.
- Umpire scheduling in US Open Tennis tournament!
- Event-based learning situations.

# Jigsaw puzzles – Intuitive usage of Simulated Annealing

- Given a jigsaw puzzle such that one has to obtain the final shape using all pieces together.
- Starting with a random configuration, the human brain unconditionally chooses certain moves that tend to the solution.
- However, certain moves that may or may not lead to the solution are accepted or rejected with a certain small probability.
- The final shape is obtained as a result of a large number of iterations.

# Boltzmann Machine

- Let **x** denote the state of the Boltzmann machine, with its component $x_i$ denoting the state of neuron i. The state **x** represents a realization of the random vector **X**. The synaptic connection from neuron i to neuron j is denoted by $w_{ji}$, with $w_{ji} = w_{ij}$ for all (i,j) and $w_{ii} = 0$ for all i.

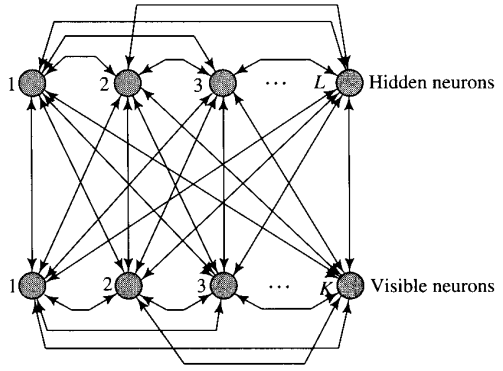



**FIGURE 11.4** Architectural graph of Botzmann machine; *K* is the number of visible neurons and *L* is the number of hidden neurons.

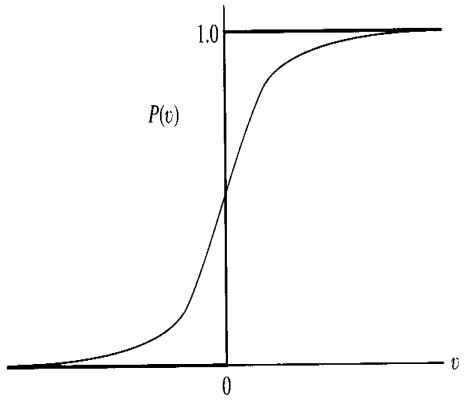



**FIGURE 11.5** Sigmoid-shaped function $\varphi(v)$.

$E(\mathbf{x}) = -\frac{1}{2}\sum_i \sum_j w_{ji}\, x_i x_j \;,\; i \neq j$

$P(\mathbf{X} = \mathbf{x}) = 1/Z \exp(-E(\mathbf{x})/T)$

$= \varphi(x/T \sum_i w_{ji}\, x_i)$ where $\varphi(.)$ is a

sigmoid function of its arguments.

Gibbs sampling and simulated annealing are used.

# Boltzmann Machine

- The goal of Boltzmann learning is to maximize the likelihood or log-likelihood function in accordance with the maximum-likelihood principle.
- *Positve phase*. In this phase the network operates in its clamped condition.
- *Negative phase*. In this second phase, the network is allowed to run freely, and therefore with no environmental input.
- The log-likelihood function $L(\mathbf{w}) = \log \prod_{x\alpha \in T} P(\mathbf{X}_\alpha = \mathbf{x}_\alpha)$ $L(\mathbf{w}) = \sum_{\mathbf{x}\alpha \in T} (\log \sum_{\mathbf{x}\beta} \exp(-E(\mathbf{x})/T) - \log \sum_{\mathbf{x}} \exp(-E(\mathbf{x})/T) )$

# Boltzmann Machine

- Differentiating L(**w**) with respect to $w_{ji}$ and introducing $\rho^{+}_{ji}$ and $\rho^{-}_{ji}$ .$\Delta w_{ji} = \varepsilon \partial L(w) / \partial w_{ji} = \eta (\rho^{+}_{ji} - \rho^{-}_{ji})$ where $\eta$ is a learning-rate parameter $\eta = \varepsilon/T$.
- From a learning point of view, the two terms that constitute the Boltzmann learning rule have opposite meaning: $\rho^{+}_{ji}$ corresponding to the clamped condition of the network is a Hebbian learning rule; $\rho^{-}_{ji}$ corresponding to the free-running condition of the network is unlearning (forgetting) term.

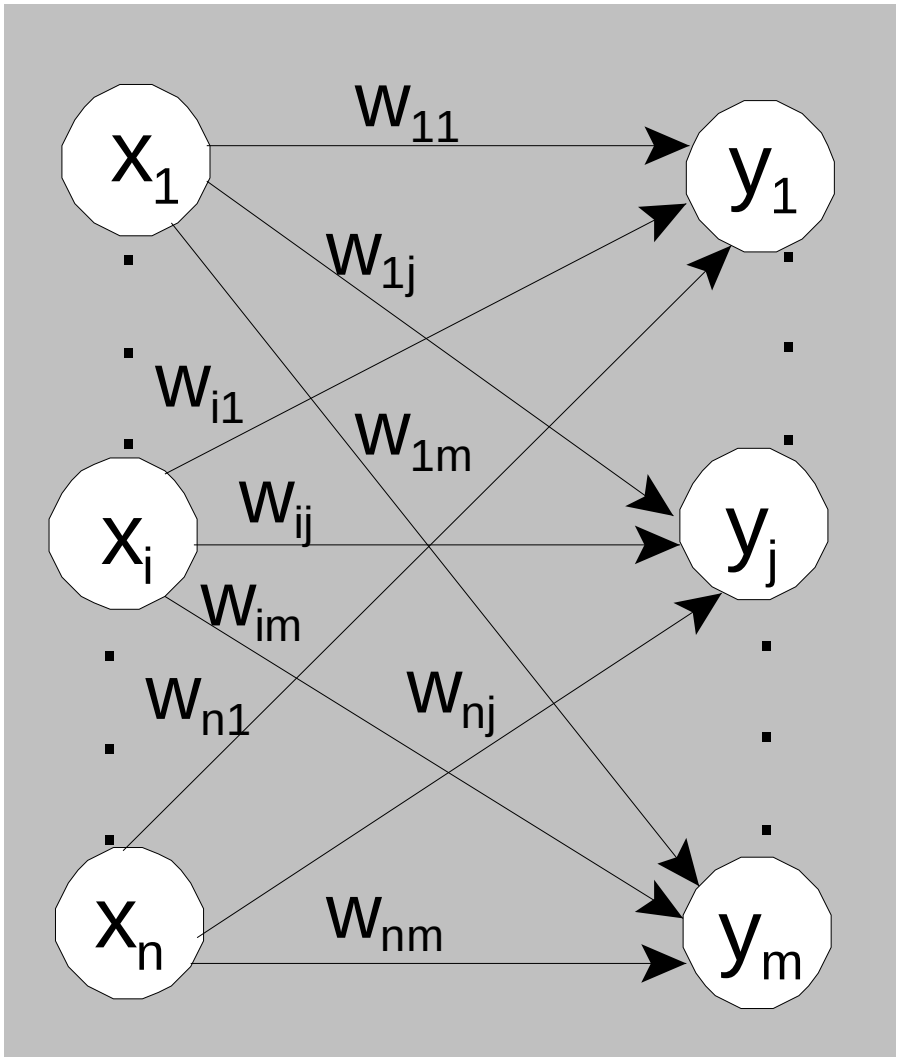
# Associative Memory

Used for memory retrieval, returning one pattern given another. There are three types of associative memory

1Heteroassociative: Mapping from X to Y s.t. if an arbitrary vector is closer to Xi than to any other Xj, the vector Yi associated with Xi is returned.

2Autoassociative: Same as above except that Xi = Yi for all exemplar pairs. Useful in retrieving a full pattern from a degraded one

# Heteroassociative Memory Architecture



# Heteroassociative Memory

- The inputs and output vectors *s* and *t* are different.
- The Hebb rule is used as a learning algorithm or calculate the weight matrix by summing the outer products of each input-output pair.
- The heteroassociative application algorithm is used to test the algorithm.

# The Hebb Algorithm

- Initialize weights to zero, $w_{ij}$ =0, where i = 1, …, n and j = 1, …, m.
- For each training case s:t repeat:
  - $x_i = s_i$ , where i=1,...,n
  - $y_i = t_j$, where j = 1, .., m
  - Adjust weights $w_{ij}(new) = w_{ij}(old) + x_i y_j$, where i = 1, .., n and j = 1, .., m

# Autoassociative Memory

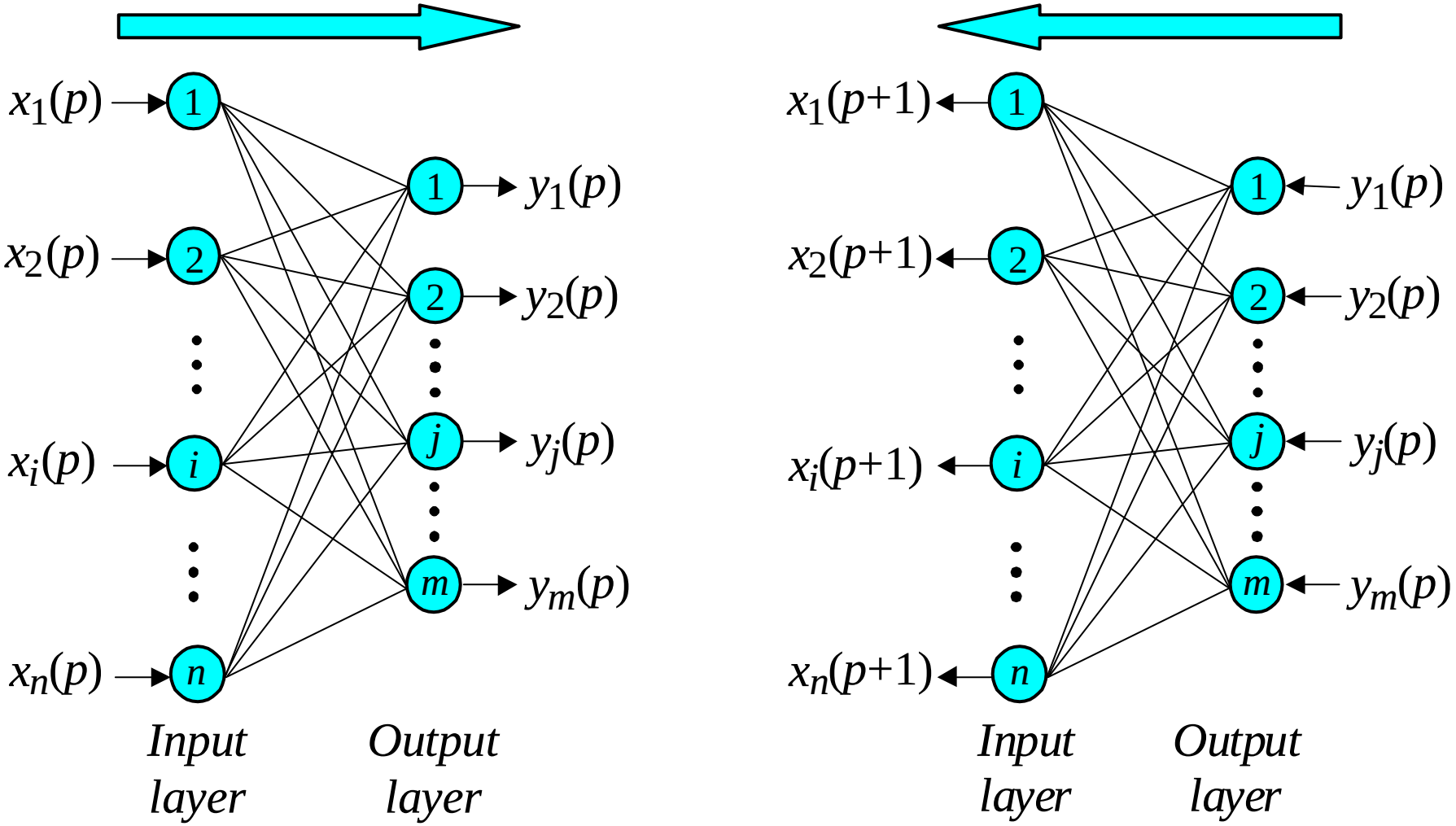
- The inputs and output vectors *s* and *t* are the same.
- The Hebb rule is used as a learning algorithm or calculate the weight matrix by summing the outer products of each input-output pair.
- The autoassociative application algorithm is used to test the algorithm.

# Bidirectional associative memory (BAM)

- The Hopfield network represents an autoassociative type of memory - it can retrieve a corrupted or incomplete memory but cannot associate this memory with another different memory.
- Human memory is essentially associative. One thing may remind us of another, and that of another, and so on. We use a chain of mental associations to recover a lost memory. If we forget where we left an umbrella, we try to recall where we last had it, what we were doing, and who we were talking to. We attempt to establish a chain of associations, and thereby to restore a lost memory.

- To associate one memory with another, we need a recurrent neural network capable of accepting an input pattern on one set of neurons and producing a related, but different, output pattern on another set of neurons.
- Bidirectional associative memory (BAM), first proposed by Bart Kosko, is a heteroassociative network. It associates patterns from one set, set A, to patterns from another set, set B, and vice versa. Like a Hopfield network, the BAM can generalise and also produce correct outputs despite corrupted or incomplete inputs.

# BAM operation




(*a*) Forward direction.

(*b*) Backward direction.

The basic idea behind the BAM is to store pattern pairs so that when n-dimensional vector X from set A is presented as input, the BAM recalls m-dimensional vector Y from set B, but when Y is presented as input, the BAM recalls X.

- To develop the BAM, we need to create a correlation matrix for each pattern pair we want to store. The correlation matrix is the matrix product of the input vector X, and the transpose of the output vector YT. The BAM weight matrix is the sum of all correlation matrices that is,

$$\mathbf{W} = \sum_{m=1}^{M} \mathbf{X}_m \, \mathbf{Y}_m^T$$

where M is the number of pattern pairs to be stored in the BAM.

# Stability and storage capacity of the BAM

- The BAM is unconditionally stable. This means that any set of associations can be learned without risk of instability.
- The maximum number of associations to be stored in the BAM should not exceed the number of neurons in the smaller layer.
- The more serious problem with the BAM is incorrect convergence. The BAM may not always produce the closest association. In fact, a stable association may be only slightly related to the initial input vector.

# UNIT-III
# COUNTER PROPAGATION NETWORK & SELF ORGANISATION MAPS

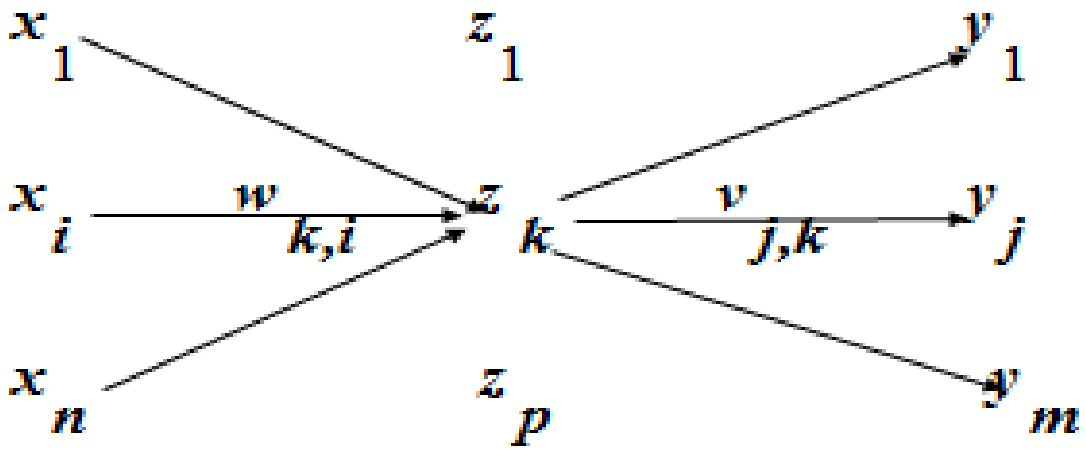
- CRN building blocks
- CPN data processing
- SQM data processing
- Applications

# Counter Propagation Network

- The counterpropagation network (CPN) is a fast-learning combination of unsupervised and supervised learning.
- Although this network uses linear neurons, it can learn nonlinear functions by means of a hidden layer of competitive units.
- Moreover, the network is able to learn a function and its inverse at the same time.
- However, to simplify things, we will only consider the feedforward mechanism of the CPN.

# Counter propagation network (CPN)

- Basic idea of CPN
  - Purpose: fast and coarse approximation of vector mapping $y = \phi(x)$
    - not to map any given x to its $\phi(x)$ with given precision,
    - input vectors x are divided into clusters/classes.
    - each cluster of x has one output y, which is (hopefully) the average of $\phi(x)$ for all x in that class.
  - Architecture: Simple case: **FORWARD ONLY CPN**,




**from input to hidden (class)**

**from hidden (class) to output**

Counter Propagation Network

- are multilayer networks based on combination of input, clustering and output layers
- can be used to compress data, to approximate functions, or to associate patterns
- approximate its training input vectors pair by adoptively constructing a lookup table

# Introduction to Counterpropagation (cont.)

- training has two stages
  - Clustering
  - Output weight updating

- There are two types of it
  - Full
  - Forward only

# Full Counterpropagation

- Produces an approximation x*:y* based on
  - input of an x vector
  - input of a y vector only
  - input of an x:y ,possibly with some distorted or missing elements in either or both vectors.

# Full Counterpropagation (cont.)

- Phase 1
  - The units in the cluster layer compete. The learning rule for weight updates on the winning cluster unit is (only the winning unit is allowed to learn)

$$\begin{cases} w_{iJ}^{new} = w_{iJ}^{old} + \alpha\left(x_i - w_{iJ}^{old}\right) & i = 1,2,..., n \\ u_{kJ}^{new} = w_{kJ}^{old} + \beta\left(y_k - u_{kJ}^{old}\right) & k = 1,2,..., m \end{cases}$$

(This is standard Kohonen learning)

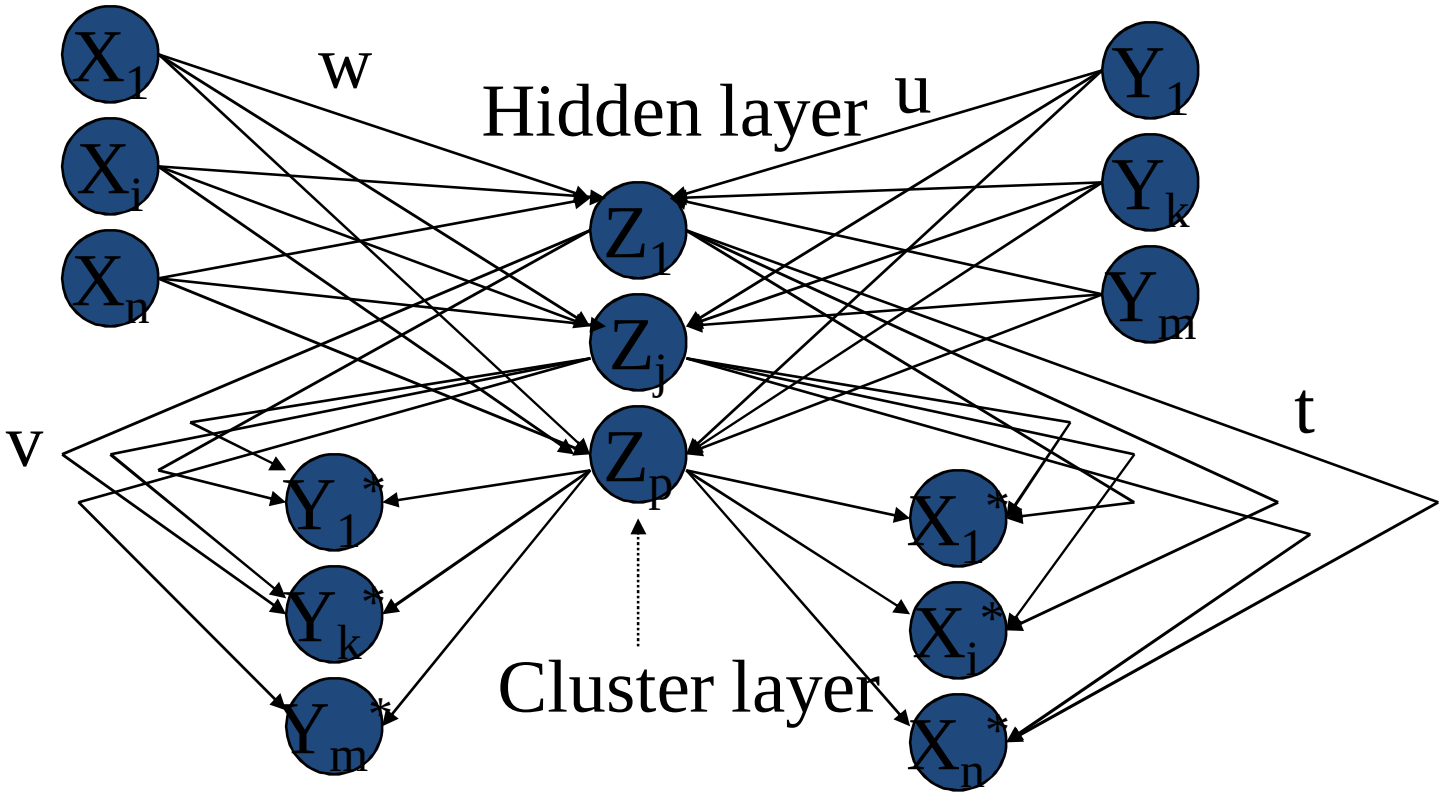
# Full Counterpropagation (cont.)

- Phase 2
  - The weights from the winning cluster unit J to the output units are adjusted so that the vector of activations of the units in the Y output layer, y*, is an approximation to the input vector y; x*, is an approximation to the input vector x. The weight updates for the units in the Y output and X output layers are

$$\begin{cases} v_{Jk}^{new} = v_{Jk}^{old} + a\left(y_k - v_{Jk}^{old}\right) & k = 1,2,...,\ m \\ t_{Ji}^{new} = t_{Ji}^{old} + b\left(x_i - t_{Ji}^{old}\right) & i = 1,2,...,\ n \end{cases}$$

(This is known as Grossberg learning)

# Architecture of Full Counterpropagation



# Full Counterpropagation Algorithm

$\mathrm{x}$ : input training vector : $\mathrm{x} = (x_1, ..., x_i, ..., x_n)$

$\mathrm{y}$ : target output corresponding to input $\mathrm{x}$ : $\mathrm{y} = (y_1, ..., y_k, ..., y_m)$

$z_j$ : activation of cluster layer unit $\mathrm{Z}_j$

$x^*$ : computed approximation to vector $\mathrm{x}$

$y^*$ : computed approximation to vector $\mathrm{y}$

$w_{ij}$ : weight from $X$ input layer, unit $X_i$, to cluster layer, unit $\mathrm{Z}_j$

$u_{kj}$ : weight from $Y$ input layer, unit $Y_k$, to cluster layer, unit $\mathrm{Z}_j$

$v_{jk}$ : weight from cluster layer, unit $Z_j$, to $Y$ output layer, unit $Y_k^*$

$t_{jk}$ : weight from cluster layer, unit $Z_j$, to $X$ output layer, unit $X_i^*$

$\alpha, \beta$ : learning rates for weights into cluster layer (Kohonen learning)

$a, b$ : learning rates for weight out from cluster layer (Grossberg learning)

# Full Counterpropagation Algorithm (phase 1)

- Step 1. Initialize weights, learning rates, etc.
- Step 2. While stopping condition for Phase 1 is false, do Step 3-8
- Step 3. For each training input pair **x**:**y**, do Step 4-6
- Step 4. Set *X* input layer activations to vector **x** ;set *Y* input layer activations to vector **y**.
- Step 5. Find winning cluster unit; call its index *J*
- Step 6. Update weights for unit *ZJ*:
- Step 7. Reduce learning rate $\alpha$ and $\beta$.
- Step 8. Test stopping condition for Phase 1 training

# Full Counterpropagation algorithm (phase 2)

- Step 9. While stopping condition for Phase 2 is false, do Step 10-16
- (Note: $\alpha$ and $\beta$ are small, constant values during phase 2)
- Step 10. For each training input pair **x**:**y**, do Step 11-14
- Step 11. Set *X* input layer activations to vector **x** ;set *Y* input layer activations to vector **y**.
- Step 12. Find winning cluster unit; call its index *J*
- Step 13. Update weights for unit *ZJ*:

# Full Counterpropagation Algorithm (phase 2)(cont.)

- Step 14. Update weights from unit *ZJ* to the output layers
- Step 15. Reduce learning rate *a* and *b*.
- Step 16. Test stopping condition for Phase 2 training.

## Which cluster is the winner?

- dot product (find the cluster with the largest net input)

$$net_j = \sum_i x_i w_{ij} + \sum_k y_k u_{kj}$$

- Euclidean distance (find the cluster with smallest square distance from the input)

$$D_j = \sum_i \left(x_i - w_{ij}\right)^2 + \sum_k \left(y_k - u_{kj}\right)^2$$

# Full Counterpropagation Application

- The application for counterpropagation is as follows:

  – Step0: initialize weights.
  – step1: for each input pair x:y, do step 2-4.
  – Step2: set X input layer activation to vector x
    set Y input layer activation to vector Y;

# Full Counterpropagation Application (cont.)

- Step3: find cluster unit Z, that is closest to the input pair

- Step4: compute approximations to x and y:

  - $X^*_i = t_{ji}$

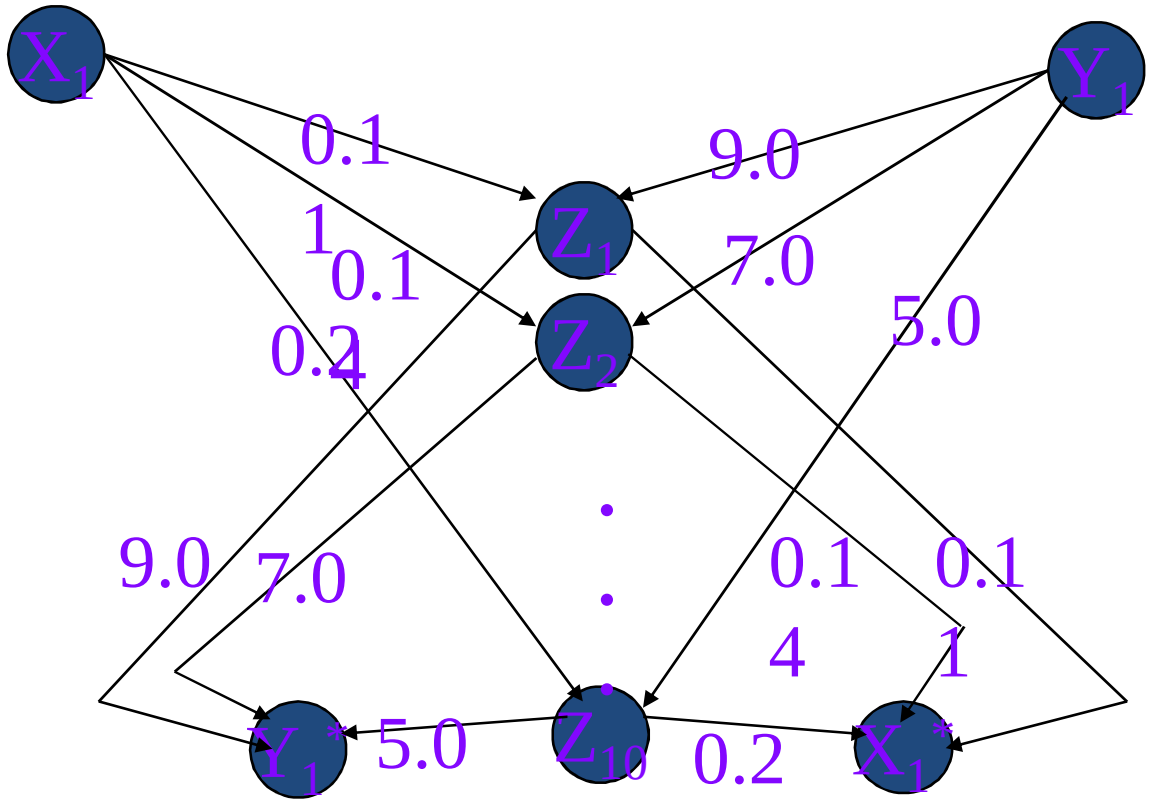
  - $Y^*_k = u_{jk}$

# Full counterpropagation example

- Function approximation of y=1/x
- After training phase we have

| Cluster unit | v | w |
|---|---|---|
| z1 | 0.11 | 9.0 |
| z2 | 0.14 | 7.0 |
| z3 | 0.20 | 5.0 |
| z4 | 0.30 | 3.3 |
| z5 | 0.60 | 1.6 |
| z6 | 1.60 | 0.6 |
| z7 | 3.30 | 0.3 |
| z8 | 5.00 | 0.2 |
| z9 | 7.00 | 0.14 |
| z10 | 9.00 | 0.11 |

# Full counterpropagation example (cont.)



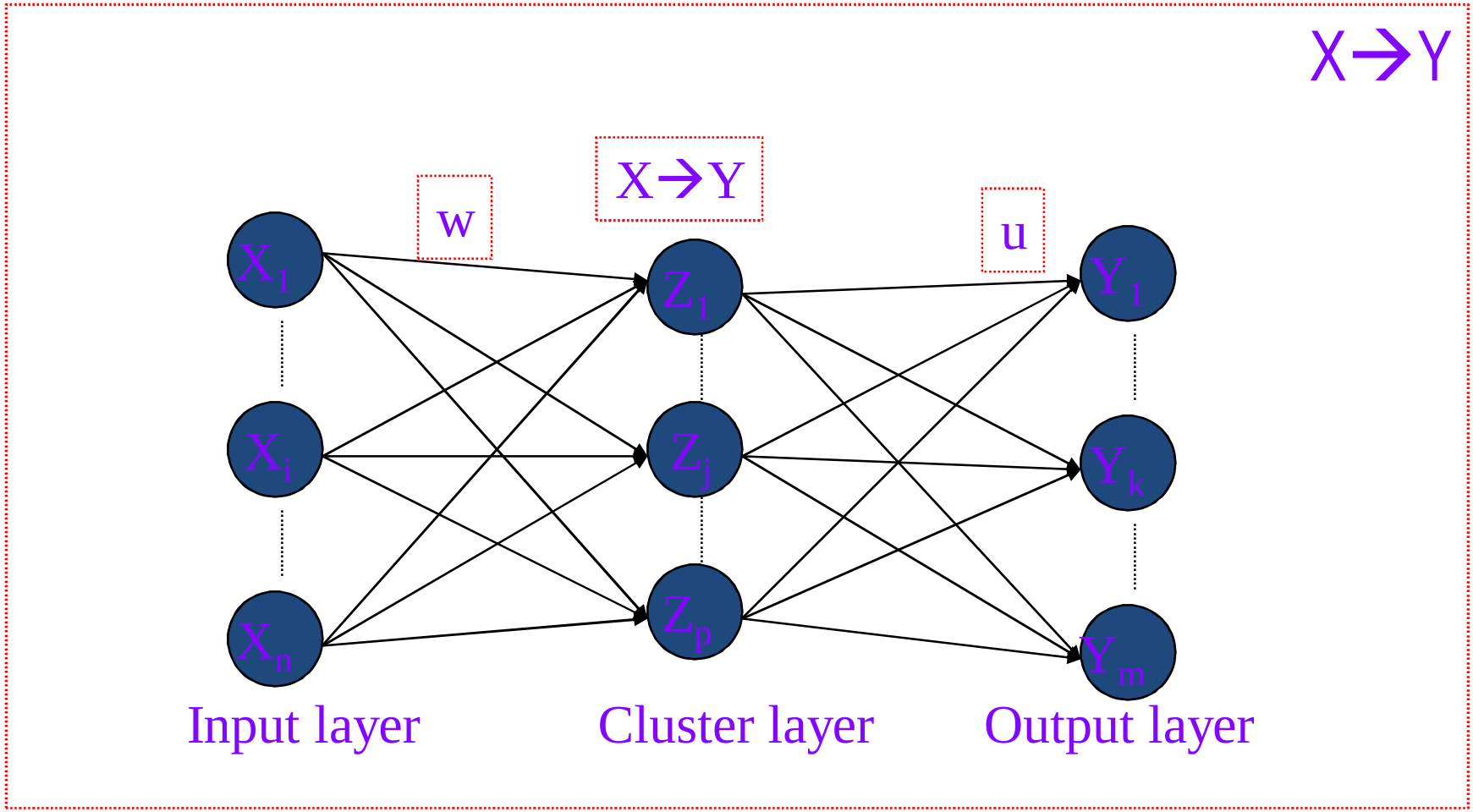
# Full counterpropagation example (cont.)

- To approximate value for y for x=0.12
- As we don't know any thing about y compute D just by means of x
- $D1=(.12-.11)^2=.0001$ 
  D2=.0004
  D3=.064
  D4=.032
  D5=.23
  D6=2.2
  D7=10.1
  D8=23.8
  D9=47.3
  D10=81

# Forward Only Counterpropagation

- Is a simplified version of  the full counterpropagation
- Are intended to approximate y=f(x) function that is not necessarily invertible
- It may be used if the mapping from x to y is well defined, but the mapping from y to x is not.

# Forward Only Counterpropagation Architecture



## Forward Only Counterpropagation Algorithm

- Step 1. Initialize weights, learning rates, etc.
- Step 2. While stopping condition for Phase 1 is false, do Step 3-8
- Step 3. For each training input **x**, do Step 4-6
- Step 4. Set *X* input layer activations to vector **x**
- Step 5. Find winning cluster unit; call its index j
- Step 6. Update weights for unit *ZJ*:

$$w_{iJ}^{new} = w_{iJ}^{old} + \alpha \left( x_i - w_{iJ}^{old} \right), \quad i = 1,2,..., \quad n$$

- Step 7. Reduce learning rate $\alpha$
- Step 8. Test stopping condition for Phase 1 training.

- Step 9. While stopping condition for Phase 2 is false, do Step 10-16
- Step 10. For each training input pair **x**:**y**, do Step 11-14
- Step 11. Set *X* input layer activations to vector **x** ;
- set *Y* input layer activations to vector **y**.
- Step 12. Find winning cluster unit; call its index *J*
- Step 13. Update weights for unit *ZJ* ($\alpha$ is small)

$$u_{Jk}^{new} = u_{Jk}^{old} + a\left(y_k - u_{Jk}^{old}\right), k = 1,2,...,m.$$

- Step 14. Update weights from unit *ZJ* to the output layers

$$w_{iJ}^{new} = w_{iJ}^{old} + \alpha\left(x_i - w_{iJ}^{old}\right), \; i = 1,2,..., \; n$$

- Step 15. Reduce learning rate *a*.
- Step 16. Test stopping condition for Phase 2 training.

# Forward Only Counterpropagation Application

- Step0: initialize weights (by training in previous subsection).
- Step1: present input vector x.
- Step2: find unit J closest to vector x.
- Step3: set activation output units:

$$y_k = u_{jk}$$

# Forward only counterpropagation example

- Function approximation of y=1/x
- After training phase we have

| Cluster unit | w | u |
|---|---|---|
| z1 | 0.5 | 5.5 |
| z2 | 1.5 | 0.75 |
| z3 | 2.5 | 0.4 |
| z4 | . | . |
| z5 | . | . |
| z6 | . | . |
| z7 | . | . |
| z8 | . | . |
| z9 | . | . |
| z10 | 9.5 | 0.1 |

- Learning in two phases:
  - training sample ($x$, $d$ ) where $z_k$ is the desired precise mapping
  - Phase1: weights $w_k$ coming into hidden nodes are trained by ***competitive learning*** to become the representative vector of a cluster of input vectors $x$: (use *only* $x$, the input part of ($x$, $d$ )) $d = \phi(x)$

Phase 2: weights going out of hidden nodes are trained by ***delta rule*** to be an average output of where $x$ is an input vector that causes to win (use *both* $x$ *and* $d$).

# Notes

- A combination of both unsupervised learning (for $w_k$ in phase 1) and supervised learning (for $v_k$ in phase 2).
- After phase 1, clusters are formed among sample input $\boldsymbol{x}$ , each $w_k$ is a representative of a cluster (average).
- After phase 2, each cluster *k* maps to an output vector y, which is the average of $\{\phi(x): x \in cluster_k\}$
- View phase 2 learning as following delta rule

$$v_{j,k^*} = v_{j,k^*} + \eta(d_j - v_{j,k^*}) \text{ where } d_j - v_{j,k^*} \propto -\frac{\partial E}{\partial v_{j,k^*}}, \text{ because}$$

$$\frac{\partial E}{\partial v_{j,k^*}} = \frac{\partial}{\partial v_{j,k^*}} \sum (d_j - v_{j,k^*} z_{k^*})^2 = -2(d_j - v_{j,k^*} z_{k^*})$$

- It can be shown that, when $t \to \infty$,

$$w_k(t) \to \langle x \rangle \text{ and } v_k(t) \to \langle \phi(x) \rangle$$

where $\langle x \rangle$ is the mean of all training samples that make $k^*$ win

# Notes

- A combination of both unsupervised learning (for $w_k$ in phase 1) and supervised learning (for $v_k$ in phase 2).
- After phase 1, clusters are formed among sample input $\boldsymbol{x}$ , each $w_k$ is a representative of a cluster (average).
- After phase 2, each cluster *k* maps to an output vector y, which is the average of $\{\phi(x): x \in cluster_k\}$
- View phase 2 learning as following delta rule

$$v_{j,k^*} = v_{j,k^*} + \eta(d_j - v_{j,k^*}) \text{ where } d_j - v_{j,k^*} \propto -\frac{\partial E}{\partial v_{j,k^*}}, \text{ because}$$

$$\frac{\partial E}{\partial v_{j,k^*}} = \frac{\partial}{\partial v_{j,k^*}} \sum (d_j - v_{j,k^*} z_{k^*})^2 = -2(d_j - v_{j,k^*} z_{k^*})$$

- It can be shown that, when $t \to \infty$,

$$w_k(t) \to \langle x \rangle \text{ and } v_k(t) \to \langle \phi(x) \rangle$$

where $\langle x \rangle$ is the mean of all training samples that make $k^*$ win

Show only on $w_{k^*}$ (proof of $v_{k^*}$ is similar.) Weight update rule can be rewriteenas $w_{k^*,i}(t+1) = (1-\eta)w_{k^*,i}(t) + \eta\, x_i(t+1)$

$$
\begin{aligned}
w_{k^*,i}(t+1) &= (1-\eta)w_{k^*,i}(t) + \eta\, x_i(t+1) \\
&= (1-\eta)((1-\eta)w_{k^*,i}(t-1) + \eta\, x_i(t)) + \eta\, x_i(t+1) \\
&= (1-\eta)^2 w_{k^*,i}(t-1) + (1-\eta)\eta\, x_i(t) + \eta\, x_i(t+1) \\
&\cdots \\
&\approx \eta\left[x_i(t+1) + x_i(t)(1-\eta) + x_i(t-1)(1-\eta)^2 + \ldots x_i(1)(1-\eta)^t\right]
\end{aligned}
$$

If $\boldsymbol{x}$ are drawn randomly from the training set, then

$$
\begin{aligned}
E[w_{ik^*,i}(t+1)] &= E[\eta(x_i(t+1) + x_i(t)(1-\eta) + \ldots x_1(1-\eta)^t] \\
&= \eta[E(x(t+1)) + (1-\eta)E(x(t)) + \ldots (1-\eta)^t E(x_i(1))] \\
&= \eta\langle x\rangle[1 + (1-\eta) + \ldots (1-\eta)^t] \\
&\Rightarrow \eta\langle x\rangle \frac{1}{1-(1-\eta)} = \langle x\rangle
\end{aligned}
$$

- After training, the network works like a look-up of math table.
  - For any input ***x,*** find a region where ***x*** falls (represented by the wining ***z*** node);
  - use the region as the index to look-up the table for the function value.
  - CPN works in multi-dimensional input space
  - More cluster nodes (***z***), more accurate mapping.
  - Training is much faster than BP
  - May have linear separability problem

# Full CPN

- If both $y = \phi(x)$ and its inverse function $x = \phi^{-1}(y)$ exist
  we can establish bi-directional approximation
- Two pairs of weights matrices:
  ***W***(*x* to *z*) and ***V***(*z* to *y*) for approx. map *x* to $y = \phi(x)$
  ***U***(*y* to *z*) and ***T***(*z* to *x*) for approx. map *y* to $x = \phi^{-1}(y)$
- When training sample (*x*, *y*) is applied $(x \text{ on } X \text{ and } y \text{ on } Y)$,
  they can jointly determine the winner $z_{k^*}$ or separately for
  $z_{k^*(x)}$ and $z_{k^*(y)}$

# Forward only counterpropagation example

- Function approximation of y=1/x
- After training phase we have

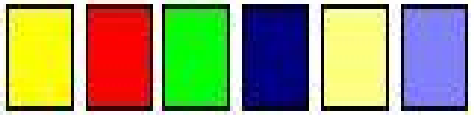
| Cluster unit | w | u |
|---|---|---|
| z1 | 0.5 | 5.5 |
| z2 | 1.5 | 0.75 |
| z3 | 2.5 | 0.4 |
| z4 | . | . |
| z5 | . | . |
| z6 | . | . |
| z7 | . | . |
| z8 | . | . |
| z9 | . | . |
| z10 | 9.5 | 0.1 |

# Introduction of SOM contd…

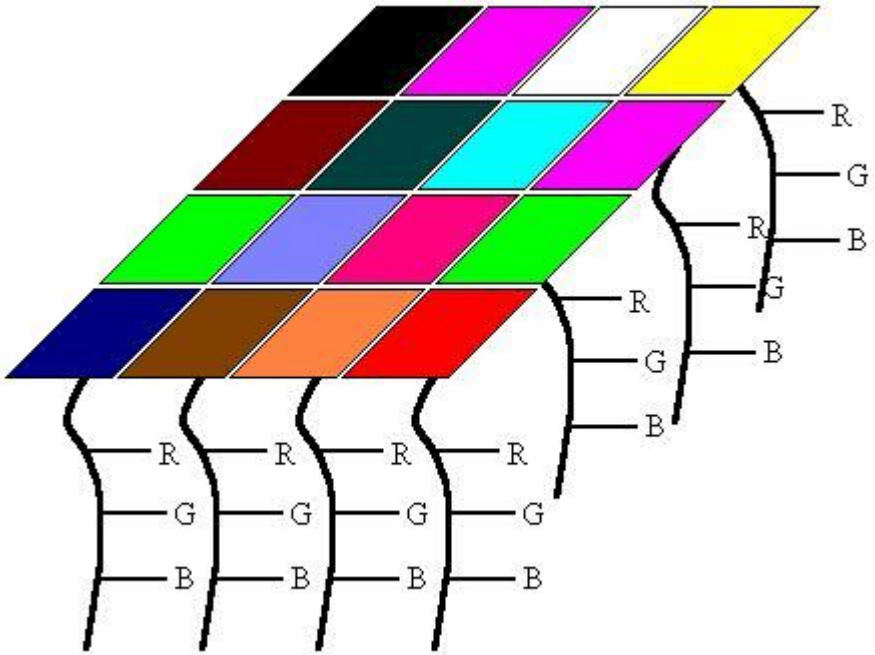
- Maintains the topology of the dataset
- Training occurs via competition between the neurons
- Impossible to assign network nodes to specific input classes in advance
- Can be used for detecting similarity and degrees of similarity
- It is assumed that input pattern fall into sufficiently large distinct groupings
- Random weight vector initialization

# Components of SOM

- Sample data



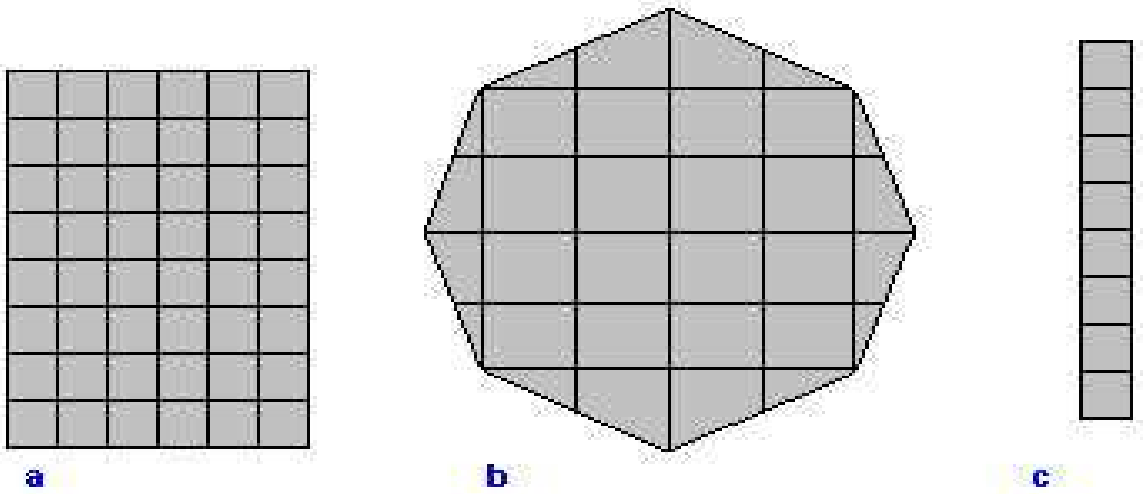
- Weights




- Output nodes

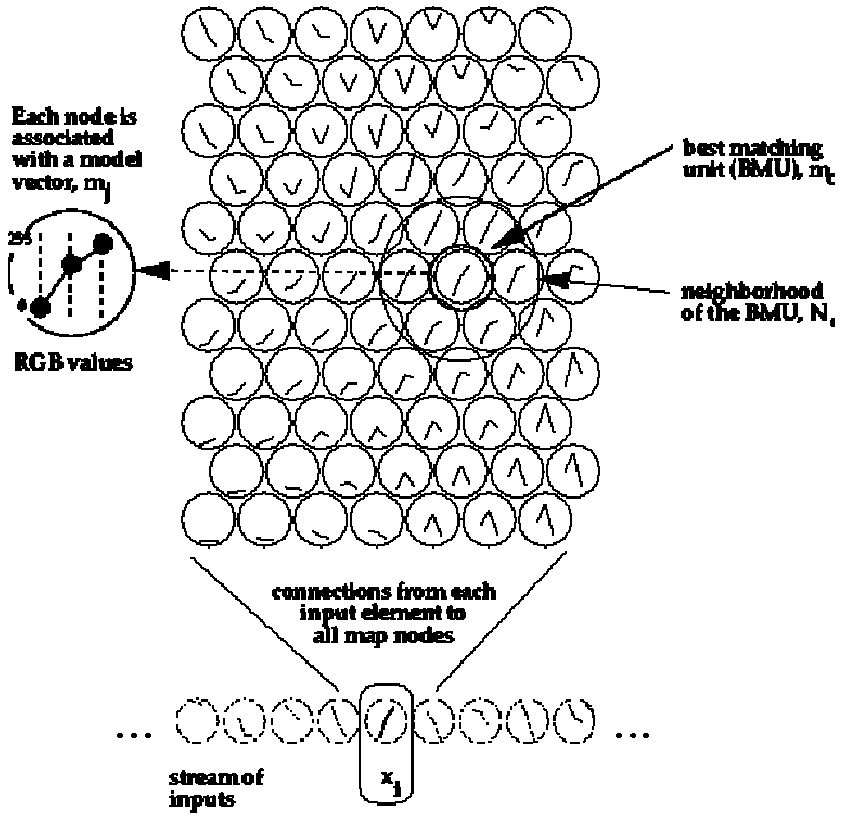
# Structure of the map

- 2-dimensional or 1-dimensional grid




- Each grid point represents a output node
- The grid is initialized with random vectors

# Training Algorithm



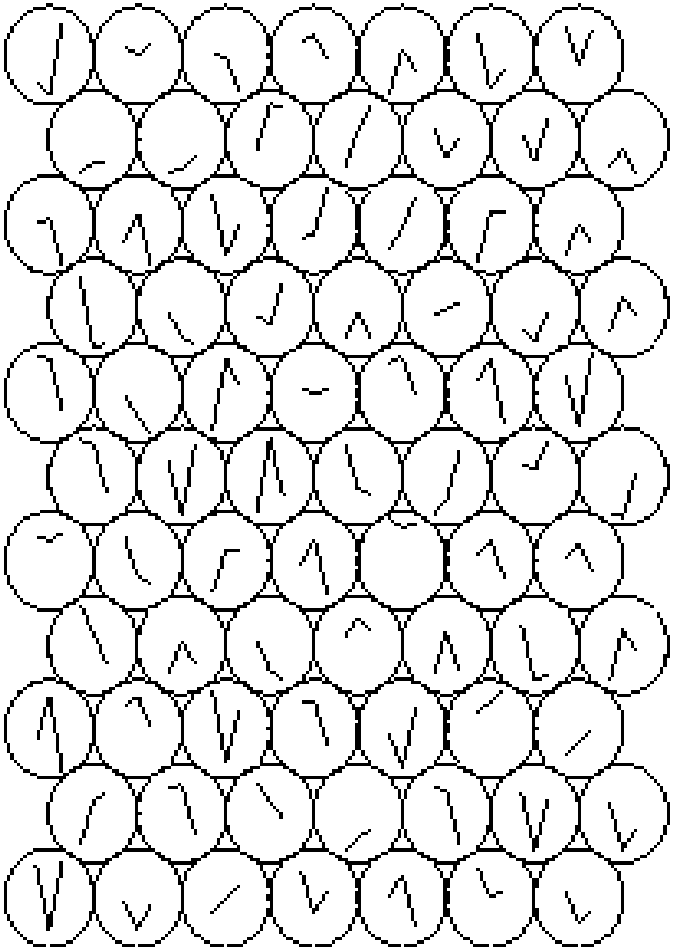

- Initialize Map
- For t from 0 to 1
  - Select a sample
  - Get best matching unit
  - Scale neighbors
  - Increase t a small amount

  End for

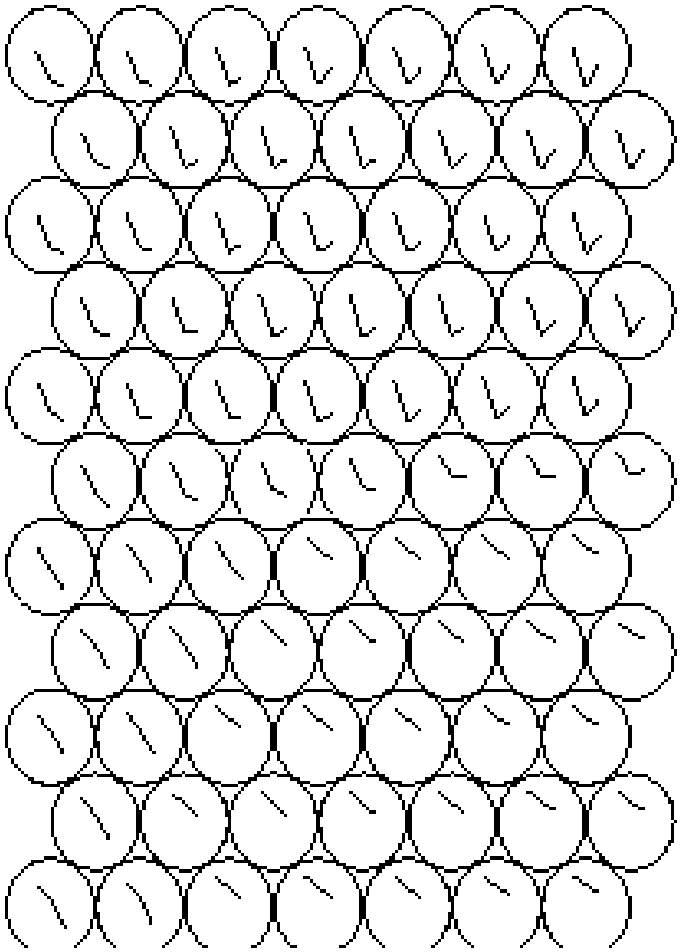
$$m_i(t+1) = m_i(t) + \alpha(t)[x(t) - m_i(t)]$$

$$\forall i \in N_c(t)$$

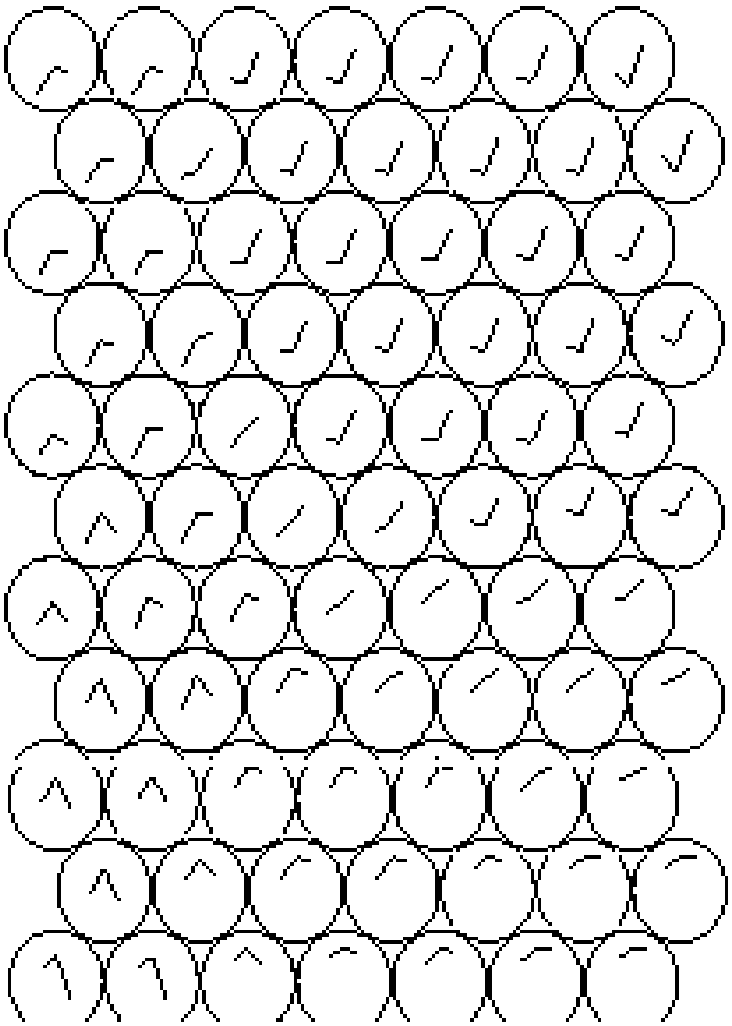
random



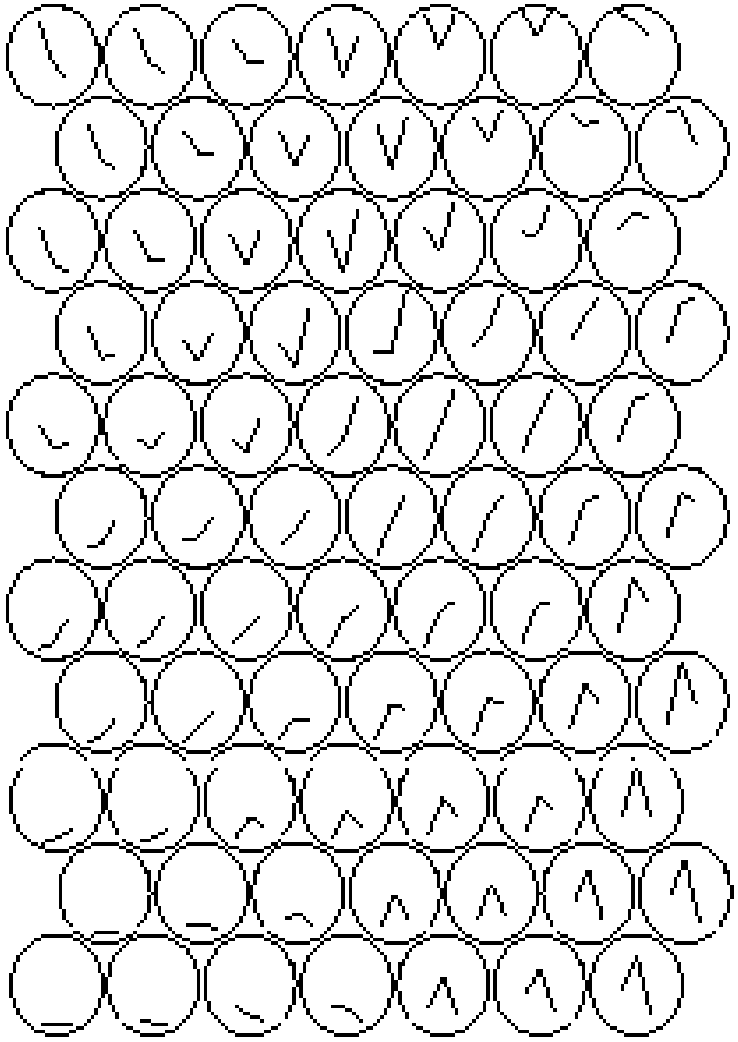
100



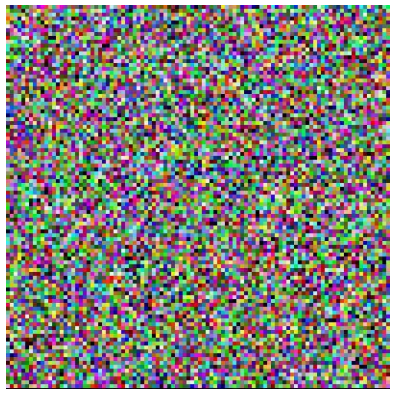
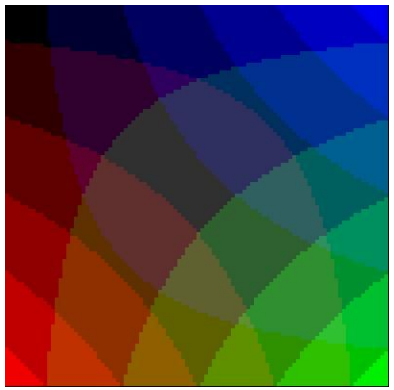
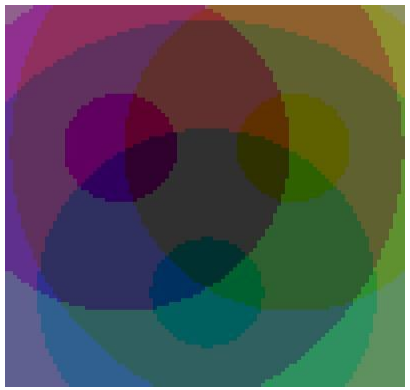
1000



10000



# Initializing the weights

- SOMs are computationally very expensive.
- Good Initialization
  - Less iterations
  - Quality of Map

## Get Best Matching Unit

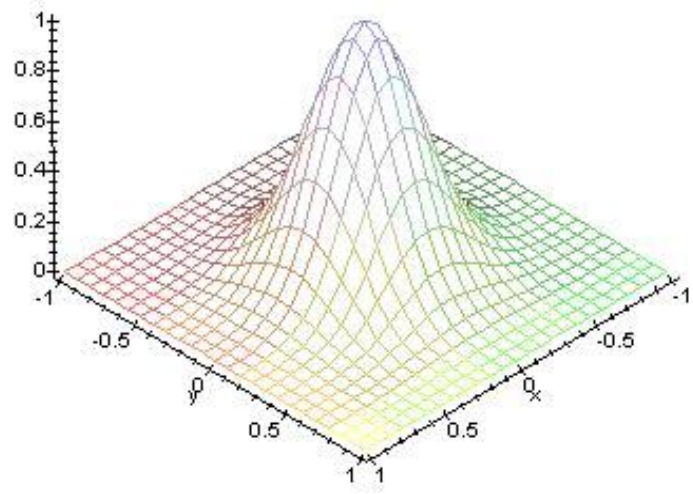
- Any method for vector distance i. e.
  - Nearest neighbor
  - Farthest neighbor
  - Distance between means
  - Distance between medians
- Most common method is Euclidean distance.

$$\sqrt{\sum_{i=0}^{n} x_i^2}$$

- More than one contestant, choose randomly

# Scale Neighbors

- Determining Neighbors
  - –Neighborhood size
    - •Decreases over time
  - –Effect on neighbors
- Learning

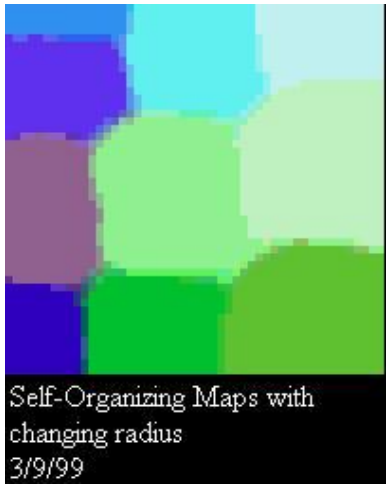


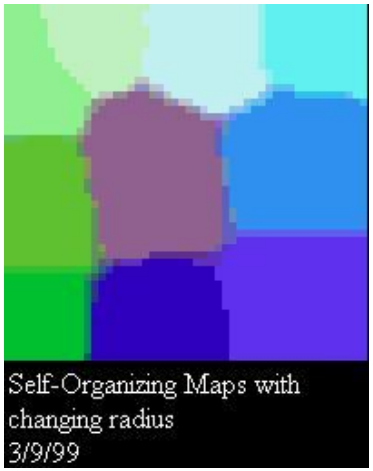

$$\forall i \in N_c(t),$$
$$m_i(t+1) = m_i(t) + \alpha(t)[x(t) - m_i(t)]$$
otherwise,
$$m_i(t+1) = m_i(t)$$

# Necessary conditions

- Amount of training data
- Change of weights should be
  - In excited neighborhood
  - Proportional to activation received

- Advantages
  - Very easy to understand
  - Works well
- Disadvantages
  - computationally expensive
  - every SOM is different



# Proof of convergence

- Complete proof only for one dimension.
  - Very trivial
- Almost all partial proofs are based on
  - Markov chains
- Difficulties :
  - No definition for "A correctly ordered configuration"
  - **Proved result** : It is not possible to associate a "Global decreasing potential function" with this algorithm.

# WebSOM (overview)

- Millions of Documents to be Searched
- Keywords or Key phrases used for searching
- DATA is clustered
  - According to similarity
  - Context
- It is kind of a Similarity graph of DATA
- For proper storage raw text documents must be encoded for mapping.

# Feature Vectors / Encoding

- Can simply be histograms of words of the Document.
    - (Histogram may be the input vector but that makes it a very large Input vector, so there is a need of some kind of reduction)
- Reduction
  - Reduction by random mapping
  - Weighted word histogram (based on word frequency)
  - By Latent Semantic Analysis

# WebSOM

- Architecture
  - Word category Map
  - Document category Map
- Modes of Operation
  - Supervised
    - (some information about the class is given, for e.g. in the collection of Newsgroup articles maybe the name of news group is supplied)
  - Unsupervised
    - (no information provided)

# Word Category Map

- Preprocessing
  - Remove unimportant data (like images, signatures)
  - Remove articles prepositions etc.
  - Words occurring less than some fixed no. of times are to be termed as ***don't care !***
  - Replace synonymous words

# Averaging Method

- Word code vector
  - –Each word represented by a unique vector (with dimension n ~ 100)
  - –Values may be random
- Context Vector
  - –For word at position i word vector is x(i)

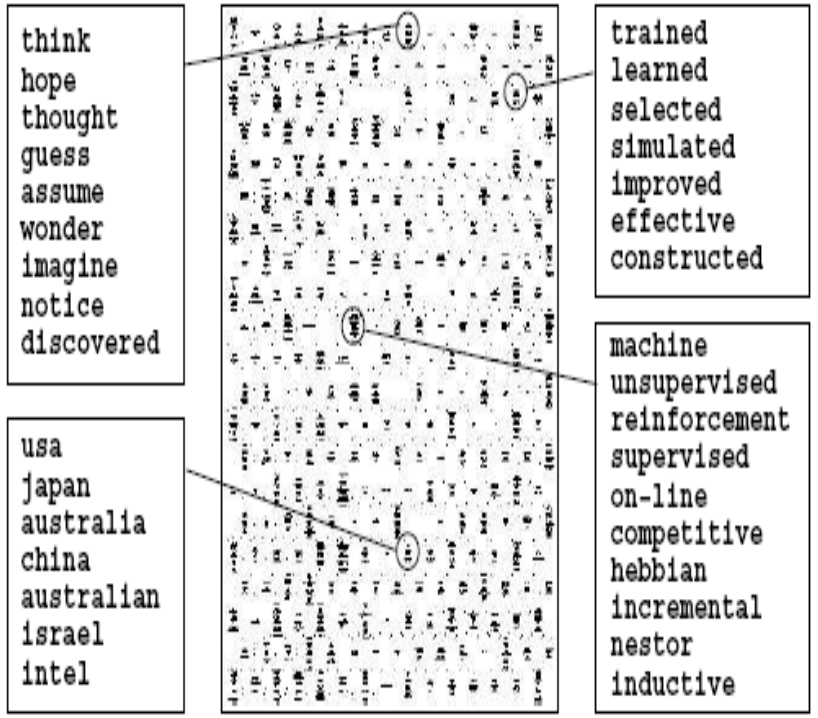
$$X(i) = \begin{bmatrix} \mathrm{E}\{x_{i-1}|x_i\} \\ \varepsilon x_i \\ \mathrm{E}\{x_{i+1}|x_i\} \end{bmatrix}$$

**where:**

  - – E() = Estimate of expected value of x over text corpus
  - – $\varepsilon$ = small scalar number

# (contd.)

- Training: taking words with different x(i)’s
- Input X(i)’s again .
- At the best matching node write the corresponding word .
- Similar context words come at same node

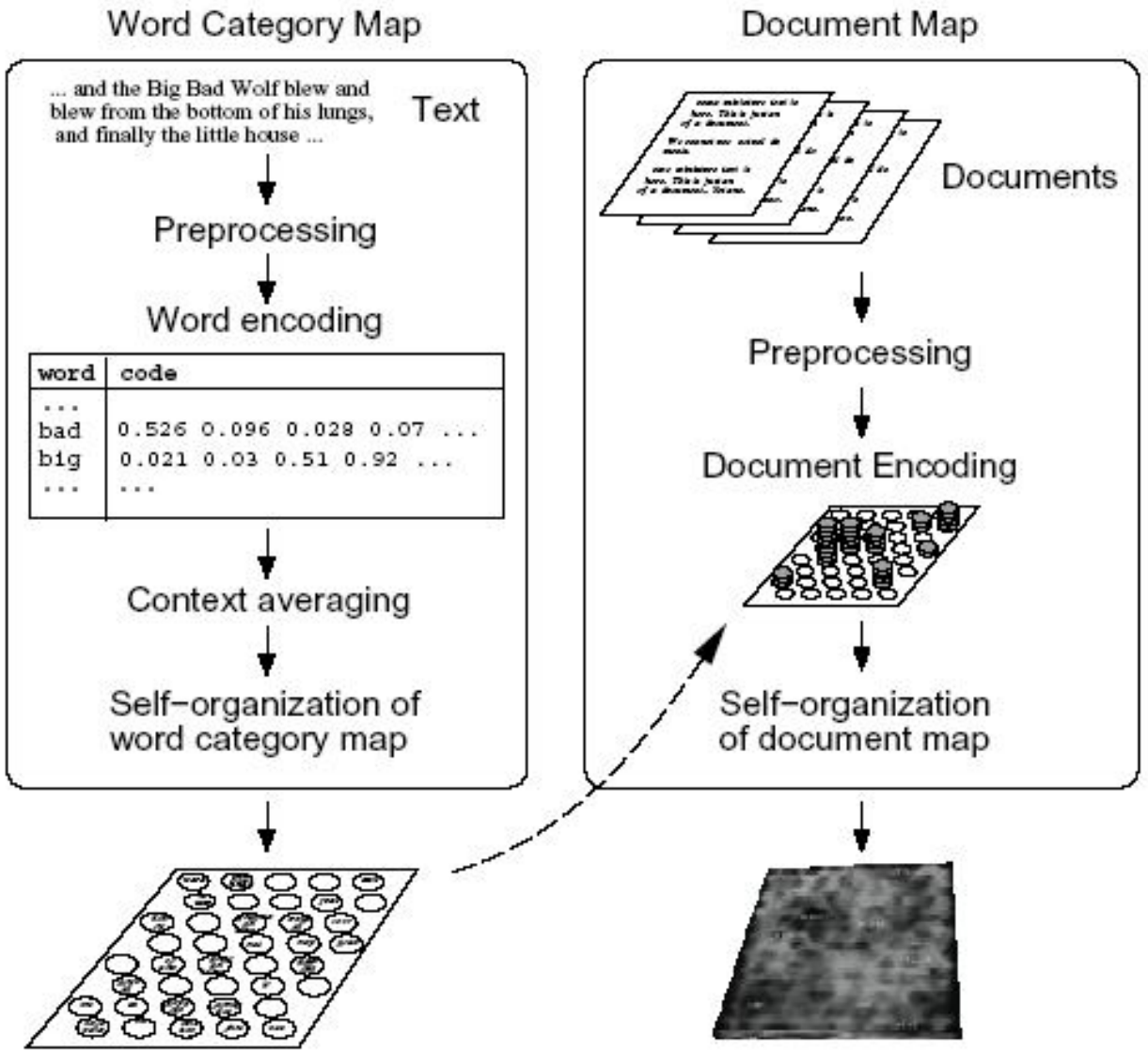



Example

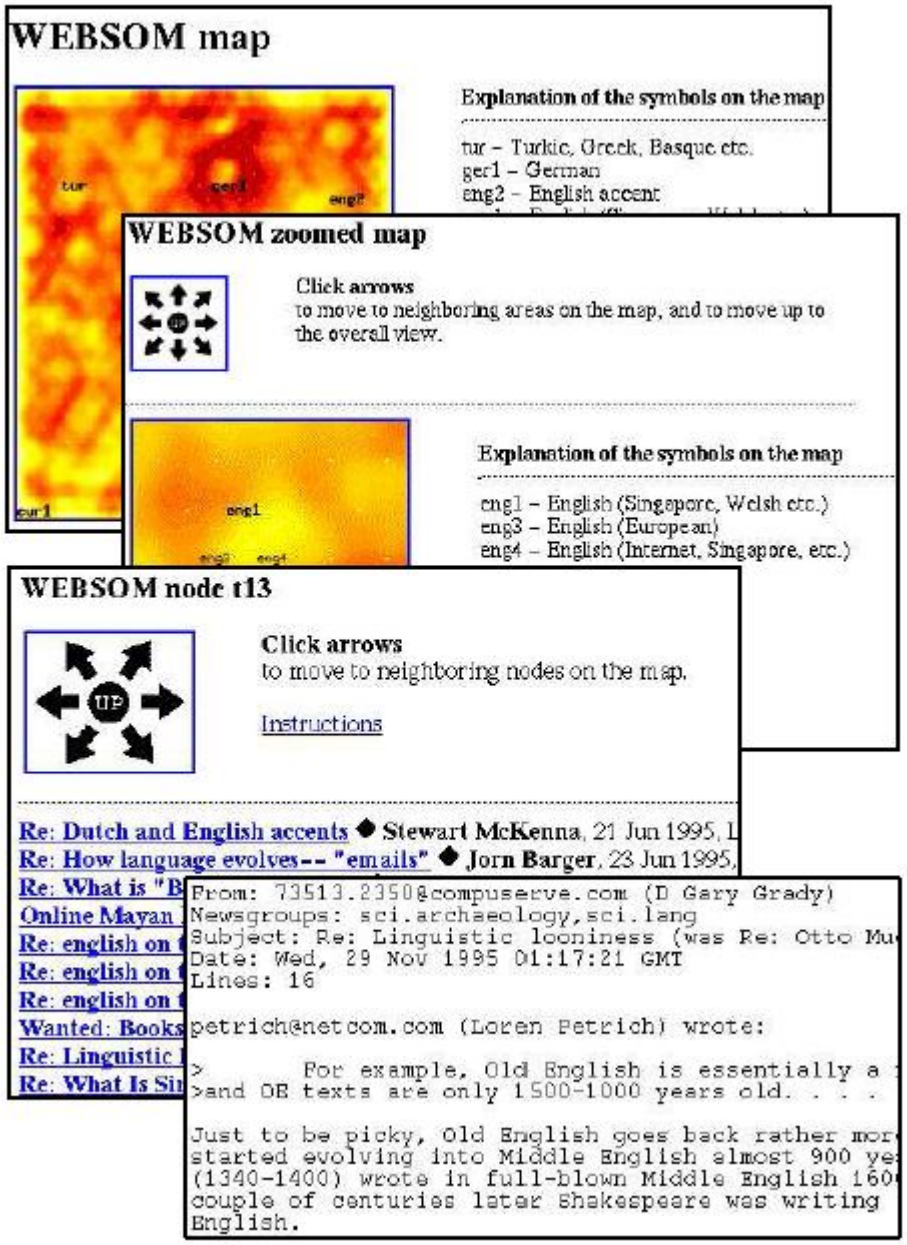
# Document Category Map

- Encoded by mapping text word by word onto the WCM.
- A histogram is formed based on the hits on WCM.
- Use this histogram as fingerprint for DCM.

# Summary:

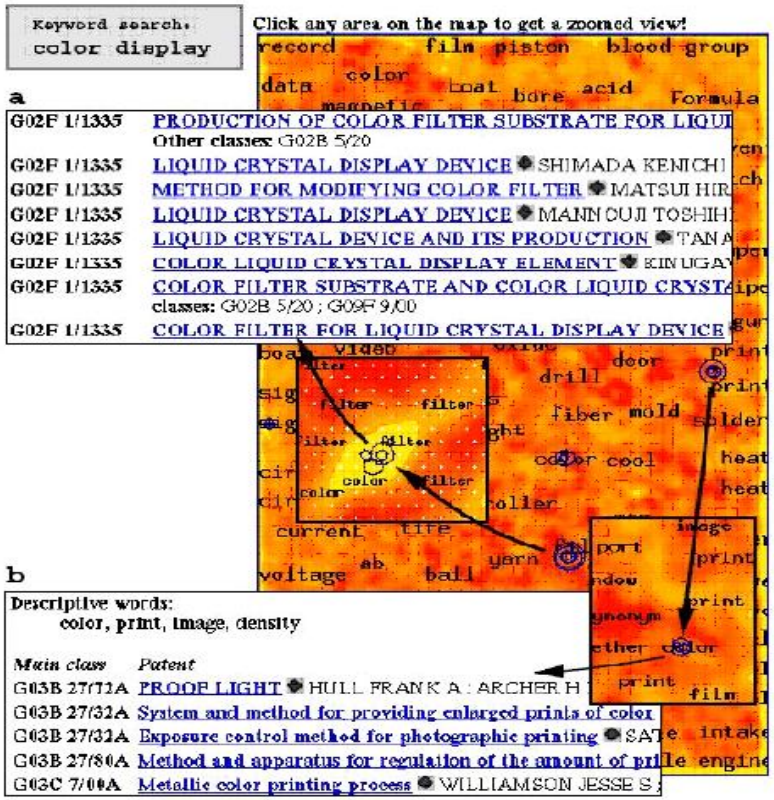

Word Category Map
... and the Big Bad Wolf blew and blew from the bottom of his lungs, and finally the little house ...
Text
Preprocessing
Word encoding
word
code
...
bad
0.526 0.096 0.028 0.07 ...
big
0.021 0.03 0.51 0.92 ...
...
...
Context averaging
Self-organization of word category map
Document Map
Documents
Preprocessing
Document Encoding
Self-organization of document map

# Demo


WEBSOM map
Explanation of the symbols on the map
tur – Turkic, Greek, Basque etc.
ger1 – German
eng2 – English accent
WEBSOM zoomed map
Click arrows
to move to neighboring areas on the map, and to move up to the overall view.
Explanation of the symbols on the map
eng1 – English (Singapore, Welsh etc.)
eng3 – English (European)
eng4 – English (Internet, Singapore, etc.)
WEBSOM node t13
Click arrows
to move to neighboring nodes on the map.
Instructions
Re: Dutch and English accents ◆ Stewart McKenna, 21 Jun 1995,
Re: How language evolves-- "emails" ◆ Jorn Barger, 23 Jun 1995,
Online Mayan
Wanted: Books
Re: Linguistic
Re: What Is Si
From: 73513.2350@compuserve.com (D Gary Grady)
Newsgroups: sci.archaeology,sci.lang
Subject: Re: Linguistic looniness (was Re: Otto Mu
Date: Wed, 29 Nov 1995 01:17:21 GMT
Lines: 16
petrich@netcom.com (Loren Petrich) wrote:
> For example, Old English is essentially a
>and OE texts are only 1500-1000 years old. . . .
Just to be picky, Old English goes back rather mor
started evolving into Middle English almost 900 ye
(1340-1400) wrote in full-blown Middle English 160
couple of centuries later Shakespeare was writing
English.

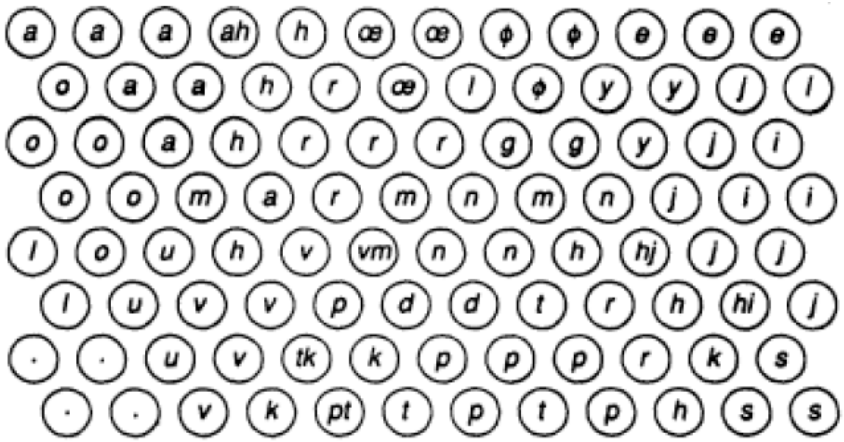
# Current Applications

- WEBSOM: Organization of a Massive Document Collection



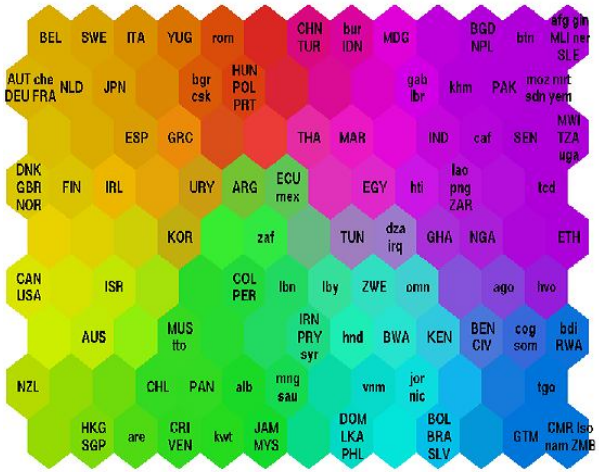
# Current Applications (continued)

- Phonetic Typewriter



# Current Applications (contd…)

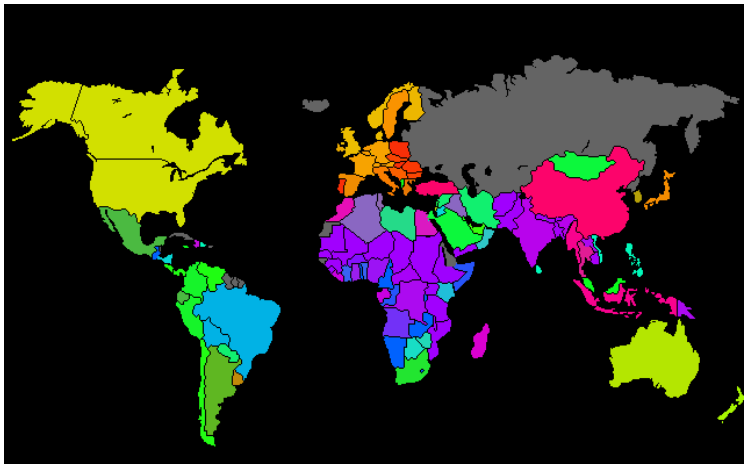
- Classifying World Poverty






The Country Names

| | | | | | |
|---|---|---|---|---|---|
| AFG | Afghanistan | GTM | Guatemala | NZL | New Zealand |
| AGO | Angola | HKG | Hong Kong | OAN | Taiwan, China |
| ALB | Albania | HND | Honduras | OMN | Oman |
| ARE | United Arab Emirates | HTI | Haiti | PAK | Pakistan |
| ARG | Argentina | HUN | Hungary | PAN | Panama |
| AUS | Australia | HVO | Burkina Faso | PER | Peru |
| AUT | Austria | IDN | Indonesia | PHL | Philippines |
| BDI | Burundi | IND | India | PNG | Papua New Guinea |
| BEL | Belgium | IRL | Ireland | POL | Poland |
| BEN | Benin | IRN | Iran, Islamic Rep. | PRT | Portugal |
| BGD | Bangladesh | IRQ | Iraq | PRY | Paraguay |
| BGR | Bulgaria | ISR | Israel | ROM | Romania |
| BOL | Bolivia | ITA | Italy | RWA | Rwanda |
| BRA | Brazil | JAM | Jamaica | SAU | Saudi Arabia |
| BTN | Bhutan | JOR | Jordan | SDN | Sudan |
| BUR | Myanmar | JPN | Japan | SEN | Senegal |
| BWA | Botswana | KEN | Kenya | SGP | Singapore |
| CAF | Central African Rep. | KHM | Cambodia | SLE | Sierra Leone |
| CAN | Canada | KOR | Korea, Rep. | SLV | El Salvador |
| CHE | Switzerland | KWT | Kuwait | SOM | Somalia |
| CHL | Chile | LAO | Lao PDR | SWE | Sweden |
| CHN | China | LBN | Lebanon | SYR | Syrian Arab Rep. |
| CIV | Cote d'Ivoire | LBR | Liberia | TCD | Chad |
| CMR | Cameroon | LBY | Libya | TGO | Togo |
| COG | Congo | LKA | Sri Lanka | THA | Thailand |
| COL | Colombia | LSO | Lesotho | TTO | Trinidad and Tobago |
| CRI | Costa Rica | MAR | Morocco | TUN | Tunisia |
| CSK | Czechoslovakia | MDG | Madagascar | TUR | Turkey |
| DEU | Germany | MEX | Mexico | TZA | Tanzania |
| DNK | Denmark | MLI | Mali | UGA | Uganda |
| DOM | Dominican Rep. | MNG | Mongolia | URY | Uruguay |
| DZA | Algeria | MOZ | Mozambique | USA | United States |
| ECU | Ecuador | MRT | Mauritania | VEN | Venezuela |
| EGY | Egypt, Arab Rep. | MUS | Mauritius | VNM | Viet Nam |
| ESP | Spain | MWI | Malawi | YEM | Yemen, Rep. |
| ETH | Ethiopia | MYS | Malaysia | YUG | Yugoslavia |
| FIN | Finland | NAM | Namibia | ZAF | South Africa |
| FRA | France | NER | Niger | ZAR | Zaire |
| GAB | Gabon | NGA | Nigeria | ZMB | Zambia |
| GBR | United Kingdom | NIC | Nicaragua | ZWE | Zimbabwe |
| GHA | Ghana | NLD | Netherlands | | |
| GIN | Guinea | NOR | Norway | | |
| GRC | Greece | NPL | Nepal | | |



# UNIT-IV
# ART AND SPATIO TEMPORAL PATTERN CLASSIFICATION

- ART network description - ART1 -ART2- Application.
- The formal avalanche
- Architecture of spatio temporal networks
- The sequential competitive avalanche field
- Applications of STNs.

# Adaptive Resonance Theory

Adaptive Resonance Theory (ART) aims to solve the "Stability – Plasticity Dilemma":

How can a system be adaptive enough to handle significant events while stable enough to handle irrelevant events?

Essentially, ART (Adaptive Resonance Theory) models incorporate new data by checking for similarity between this new data and data already learned; "memory". If there is a close enough match, the new data is learned. Otherwise, this new data is stored as a "new memory".

# Adaptive Resonance Theory

- The main operation of ART classification can be divided into the following phases −
- **Recognition phase** − The input vector is compared with the classification presented at every node in the output layer. The output of the neuron becomes “1” if it best matches with the classification applied, otherwise it becomes “0”.

**Comparison phase** − In this phase, a comparison of the input vector to the comparison layer vector is done. The condition for reset is that the degree of similarity would be less than vigilance parameter.

**Search phase** − In this phase, the network will search for reset as well as the match done in the above phases. Hence, if there would be no reset and the match is quite good, then the classification is over. Otherwise, the process would be repeated and the other stored pattern must be sent to find the correct match.

Variations:

ART1 – Designed for discrete input.

ART2 – Designed for continuous input.

ARTMAP – Combines two ART models to form a supervised learning model.

# Adaptive Resonance Theory-Operating Principle

- Adaptive Resonance Theory (ART) networks, as the name suggests, is always open to new learning (adaptive) without losing the old patterns (resonance).
- Basically, ART network is a vector classifier which accepts an input vector and classifies it into one of the categories depending upon which of the stored pattern it resembles the most.

# Architecture of ART1

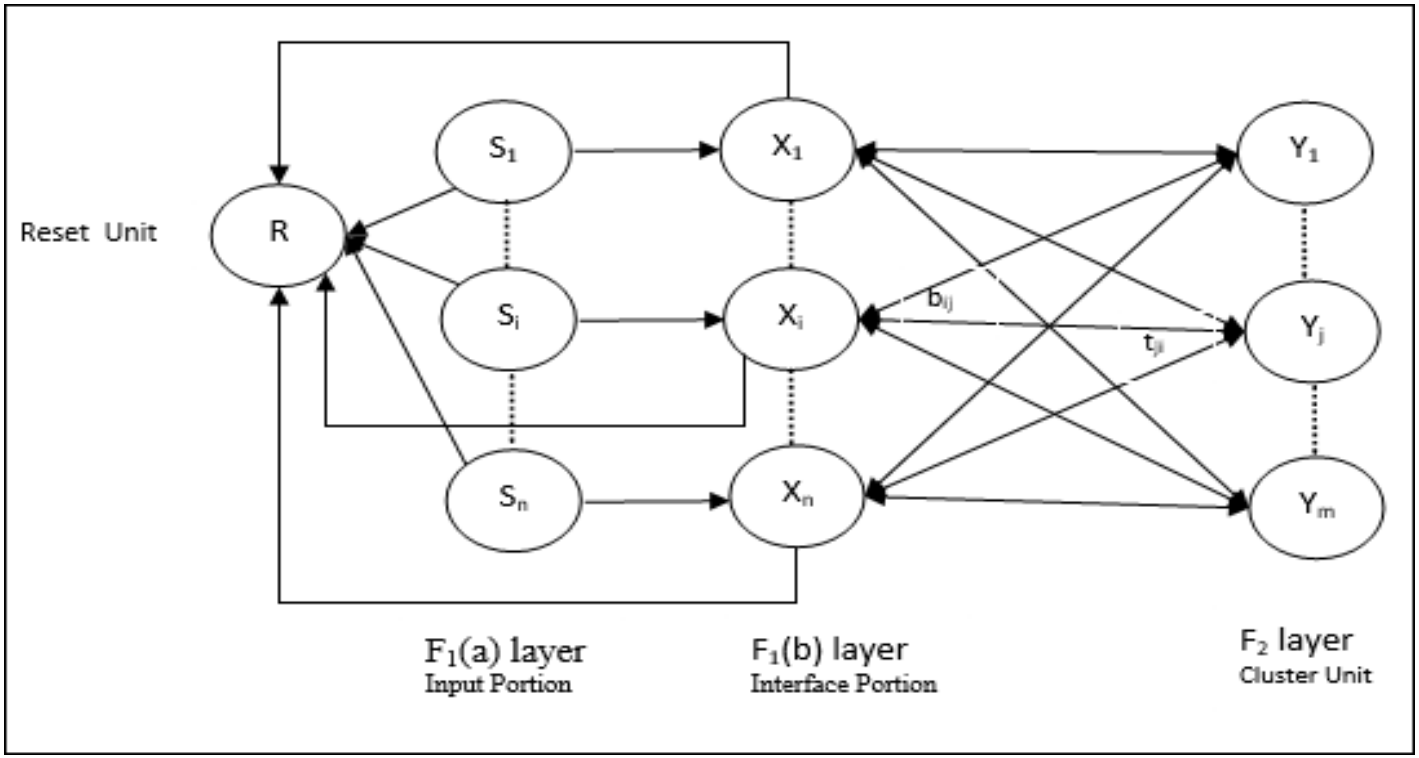
- It consists of the following two units −
- **Computational Unit** − It is made up of the following
- **Input unit ($F_1$ layer)** − It further has the following two portions −
  - **$F_1$(a) layer (Input portion)** − In ART1, there would be no processing in this portion rather than having the input vectors only. It is connected to $F_1$(b) layer (interface portion).
  - **$F_1$(b) layer (Interface portion)** − This portion combines the signal from the input portion with that of $F_2$ layer. $F_1$(b) layer is connected to $F_2$ layer through bottom up weights $\mathbf{b_{ij}}$ and $F_2$layer is connected to $F_1$(b) layer through top down weights $\mathbf{t_{ji}}$.

# Architecture of ART1

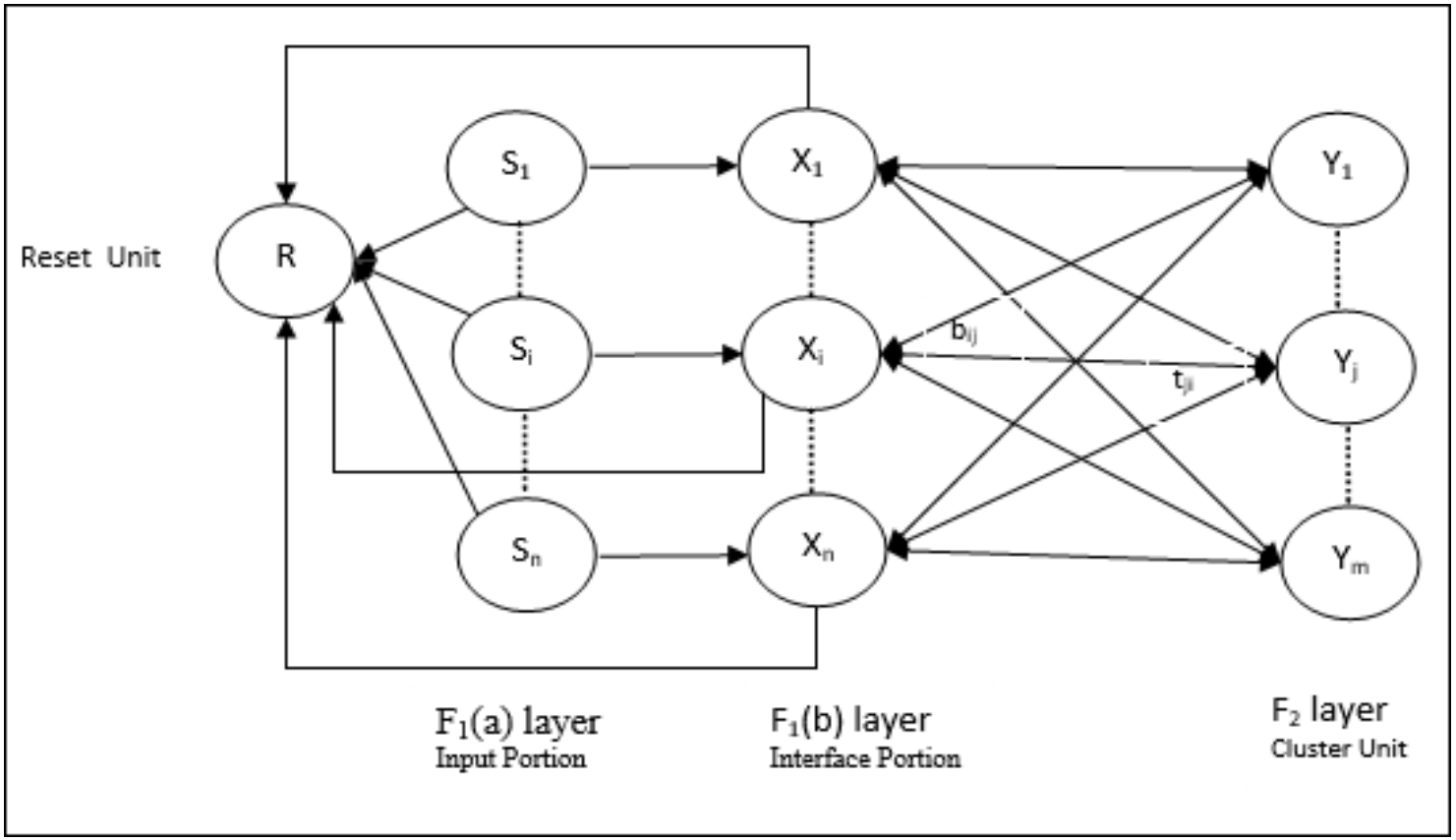
- **Cluster Unit ($F_2$ layer)** − This is a competitive layer. The unit having the largest net input is selected to learn the input pattern. The activation of all other cluster unit are set to 0.
- **Reset Mechanism** − The work of this mechanism is based upon the similarity between the top-down weight and the input vector. Now, if the degree of this similarity is less than the vigilance parameter, then the cluster is not allowed to learn the pattern and a rest would happen.

**Supplement Unit** − Actually the issue with Reset mechanism is that the layer $\mathbf{F_2}$ must have to be inhibited under certain conditions and must also be available when some learning happens. That is why two supplemental units namely, $\mathbf{G_1}$ and $\mathbf{G_2}$ is added along with reset unit, **R**. They are called **gain control units**. These units receive and send signals to the other units present in the network. **'+'** indicates an excitatory signal, while **'−'** indicates an inhibitory signal.

# Architecture of ART1



# Architecture of ART1



- Parameters Used
- Following parameters are used −
- **n** − Number of components in the input vector
- **m** − Maximum number of clusters that can be formed
- $\mathbf{b_{ij}}$ − Weight from $F_1$(b) to $F_2$ layer, i.e. bottom-up weights
- $\mathbf{t_{ji}}$ − Weight from $F_2$ to $F_1$(b) layer, i.e. top-down weights
- **ρ** − Vigilance parameter
- **||x||** − Norm of vector x

Algorithm

- **Step 1** − Initialize the learning rate, the vigilance parameter, and the weights as follows −

  $\alpha>1and0<\rho\leq1\alpha>1and0<\rho\leq1$

  $0<bij(0)<\alpha\alpha-1+nandtij(0)=10<bij(0)<\alpha\alpha-1+nandtij(0)=1$
- **Step 2** − Continue step 3-9, when the stopping condition is not true.
- **Step 3** − Continue step 4-6 for every training input.
- **Step 4** − Set activations of all $F_1(a)$ and $F_1$ units as follows
- **$F_2$ = 0 and $F_1(a)$ = input vectors**
- **Step 5** − Input signal from $F_1(a)$ to $F_1(b)$ layer must be sent like $si=xisi=xi$
- **Step 6** − For every inhibited $F_2$ node

  $yj=\sum ibijxiyj=\sum ibijxi$ the condition is **$y_j \neq -1$**

- **Step 7** − Perform step 8-10, when the reset is true.
- **Step 8** − Find **J** for $\mathbf{y_J \geq y_j}$ for all nodes **j**
- **Step 9** − Again calculate the activation on $F_1$(b) as follows xi=sitJixi=sitJi
- **Step 10** − Now, after calculating the norm of vector **x** and vector **s**, we need to check the reset condition as follows −If **||x||/ ||s||** < vigilance parameter **ρ**,theninhibit node **J** and go to step 7

  Else If **||x||/ ||s||** ≥ vigilance parameter **ρ**, then proceed further.

.

- **Step 11** − Weight updating for node **J** can be done as follows −bij(new)=αxiα−1+||x||bij(new)=αxiα−1+||x||

  tij(new)=xitij(new)=xi
- **Step 12** − The stopping condition for algorithm must be checked and it may be as follows −

  Do not have any change in weight.

  Reset is not performed for units.

  Maximum number of epochs reached

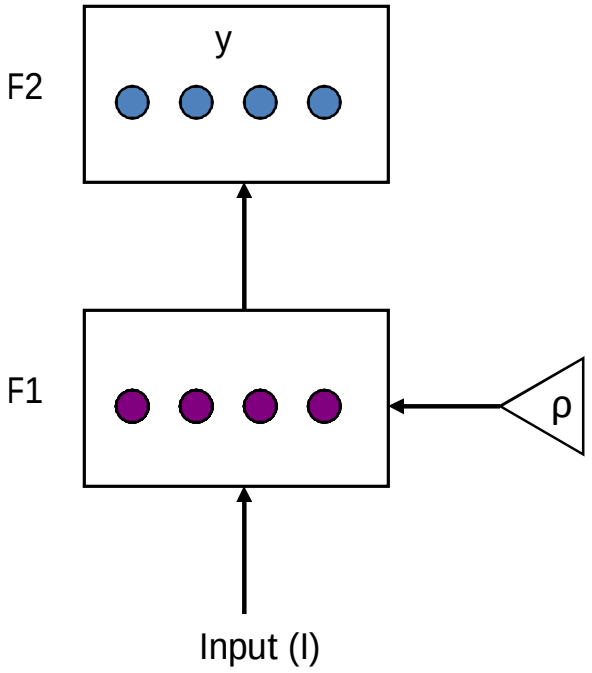
# Adaptive Resonance Model

The basic ART model, ART1, is comprised of the following components:

1. The short term memory layer: F1 – Short term memory.
2. The recognition layer: F2 – Contains the long term memory of the system.
3. Vigilance Parameter: $\rho$ – A parameter that controls the generality of the memory. Larger $\rho$ means more detailed memories, smaller $\rho$ produces more general memories.

Training an ART1 model basically consists of four steps.

# Adaptive Resonance Model




Step 1: Send input from the F1 layer to F2 layer for processing. The first node within the F2 layer is chosen as the closest match to the input and a hypothesis is formed. This hypothesis represents what the node will look like after learning has occurred, assuming it is the correct node to be updated.

# Application: Image-Text Associations

Querying data over the internet requires that noisy and/or junk data be discarded.

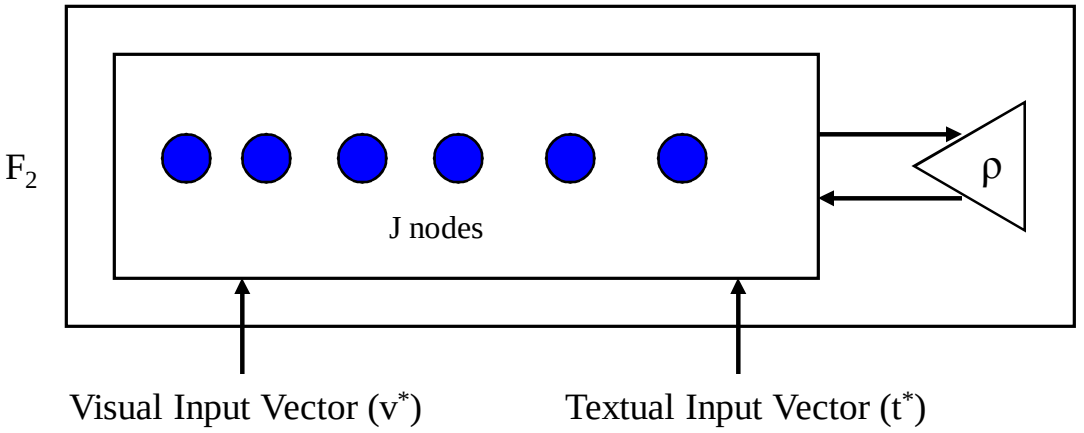
Goal: Filter out unnecessary data while keeping images and their captions.

Difficulties:

1. Large amounts of textual and multimedia data.
2. Captions can correspond to multiple images.
3. Training learning models require time and lots of training data.

# Fusion – ART Architecture

Fusion-ART uses two input vectors, one representing keywords of image data and the other representing textual data, to learn image-text associations.



## $F_2$ – Association ART

Learning such associations consists of four steps:

1. Choosing most relevant association.
2. Selecting association.
3. Determining if vectors are within vigilance.
4. Learning.

Internet Identity's Phishing Trends report for the second quarter of 2009 said that Avalanche "have detailed knowledge of commercial banking platforms, particularly treasury management systems and the Automated Clearing House (ACH) system. They are also performing successful real-time man-in-the-middle attacks that defeat two-factor security tokens."

Avalanche had many similarities to the previous group Rock Phish - the first phishing group which used automated techniques - but with greater in scale and volume.Avalanche hosted its domains on compromised computers (a botnet). There was no single hosting provider, makingis difficult to take down the domain and requiring the involvement of the responsible domain registrar.

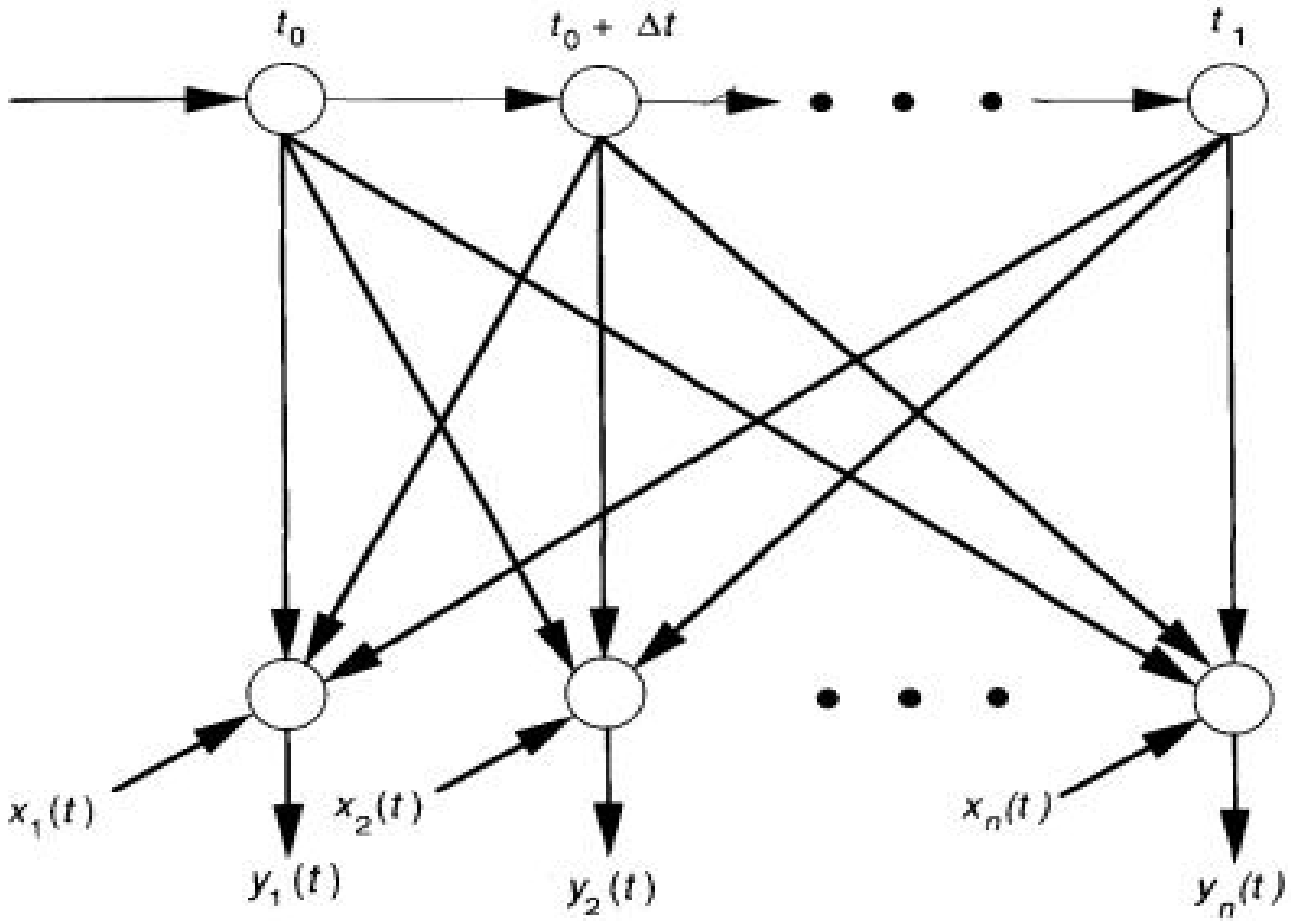



**Figure 9.3** This figure shows Grossberg's formal avalanche structure. The network consists of multiple outstars, all sharing their output units. The network learns $y(t) \approx \backslash(t)$. Once it is trained, the avalanche can replay $y(t)$ in the proper sequence by exciting the outstars sequentially.

# The Formal Avalanche

- Avalanche used spam email purporting to come from trusted organisations such as financial institutions or employment websites. Victims were deceived into entering personal information on websites made to appear as though they belong to these organisations. They were sometimes tricked into installing software attached to the emails or at a website.
  The malware logged keystrokes, stole passwords and credit card information, and allowed unauthorised remote access to the infected computer.

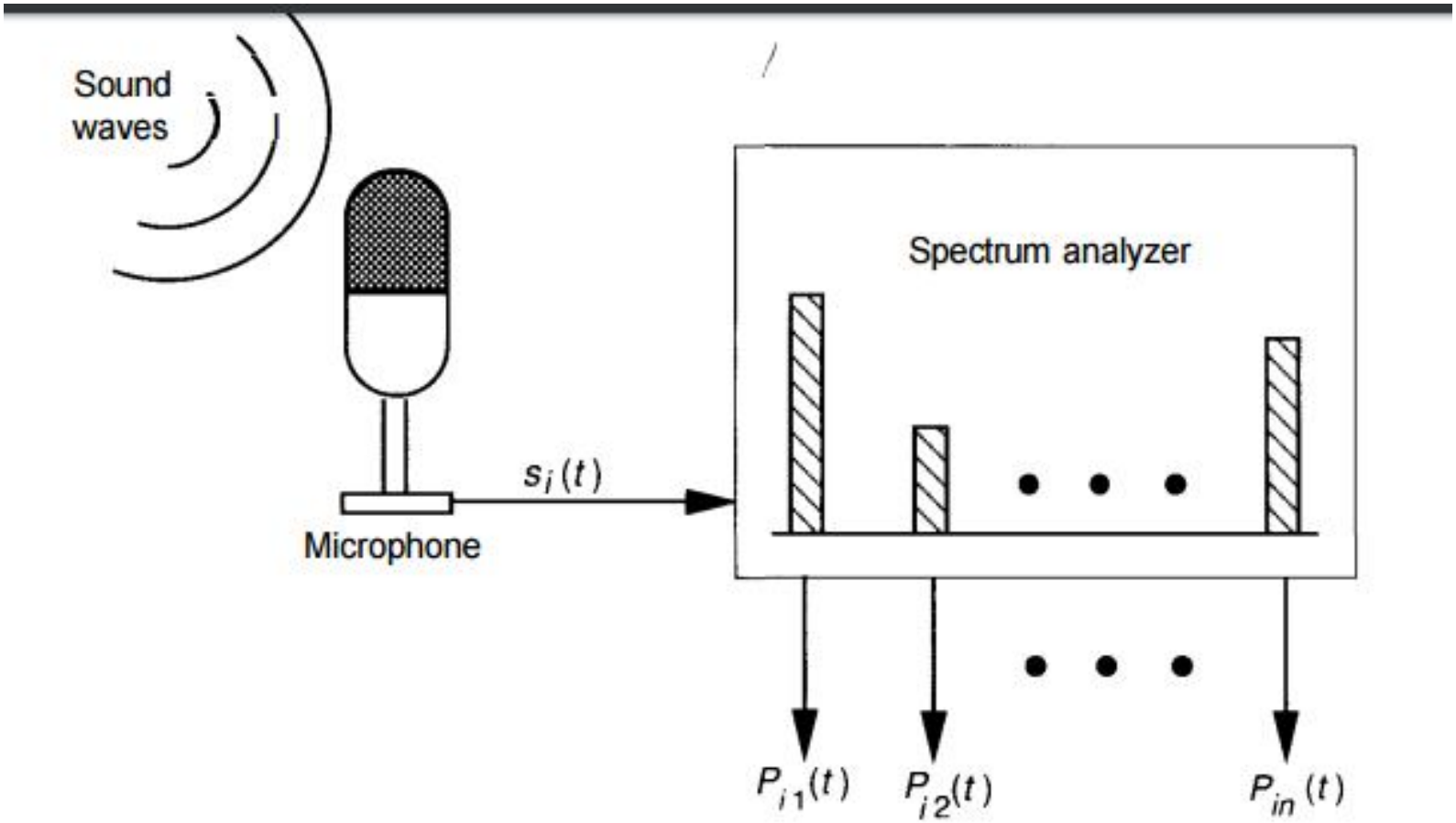



**Figure 9.4** This diagram shows the generation of power spectra from speech. The microphone acts as a transducer producing the electrical signal, $s_i(t)$, from the sound-pressure waves. The subscript *i* identifies a single word. The spectrum analyzer measures the power contained in each of *n* frequency bins. $P_{i1}(t)$ is the power in frequency bin 1 for the $i$th spoken word.

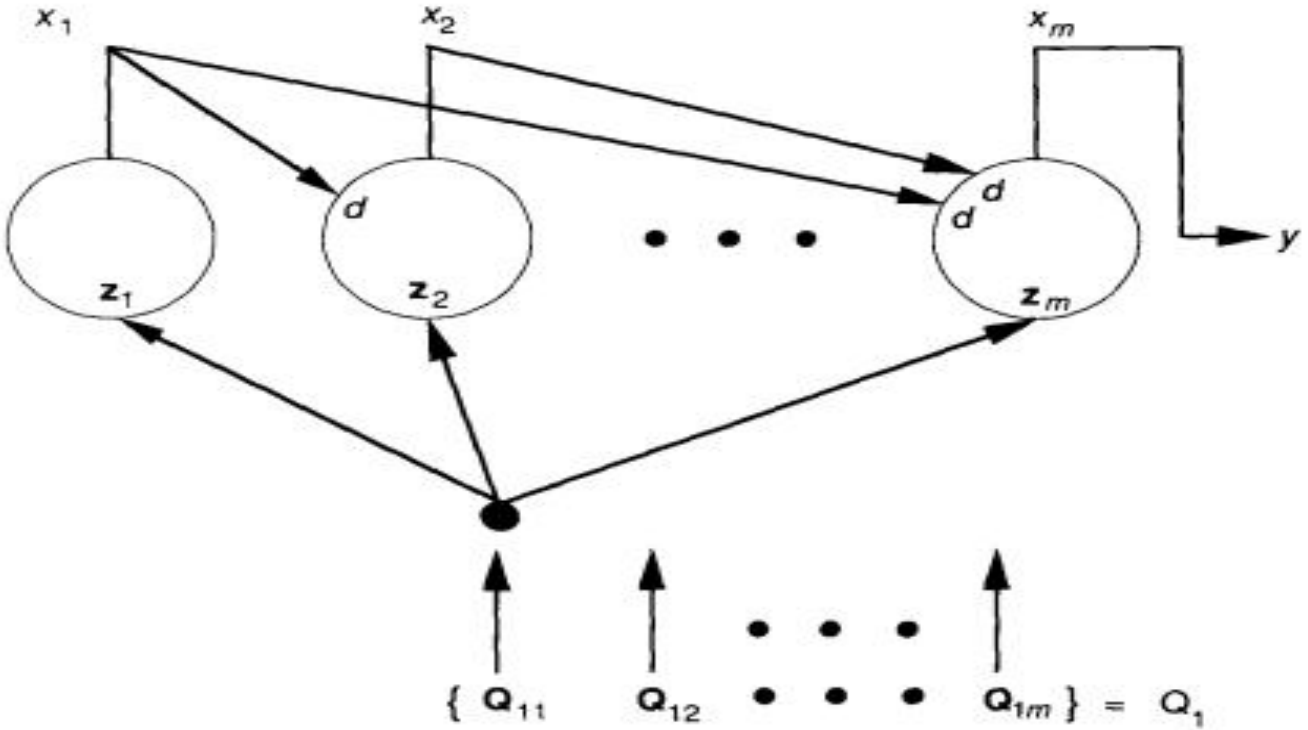



**Figure 9.5** The avalanche network for single-word recognition is shown. The output of each unit is connected to each successive unit, from left to right, with a connection strength of $d$, where $0 < d < 1$. The dimension of the input vector is equal to the number of channels in the spectrum analyzer, and the number of units is equal to the number of times the power spectrum is sampled. Weight vectors associated with each unit are denoted by the $z$ vectors. The network output is the single value, $y$. See the text for details of the operation of this network.

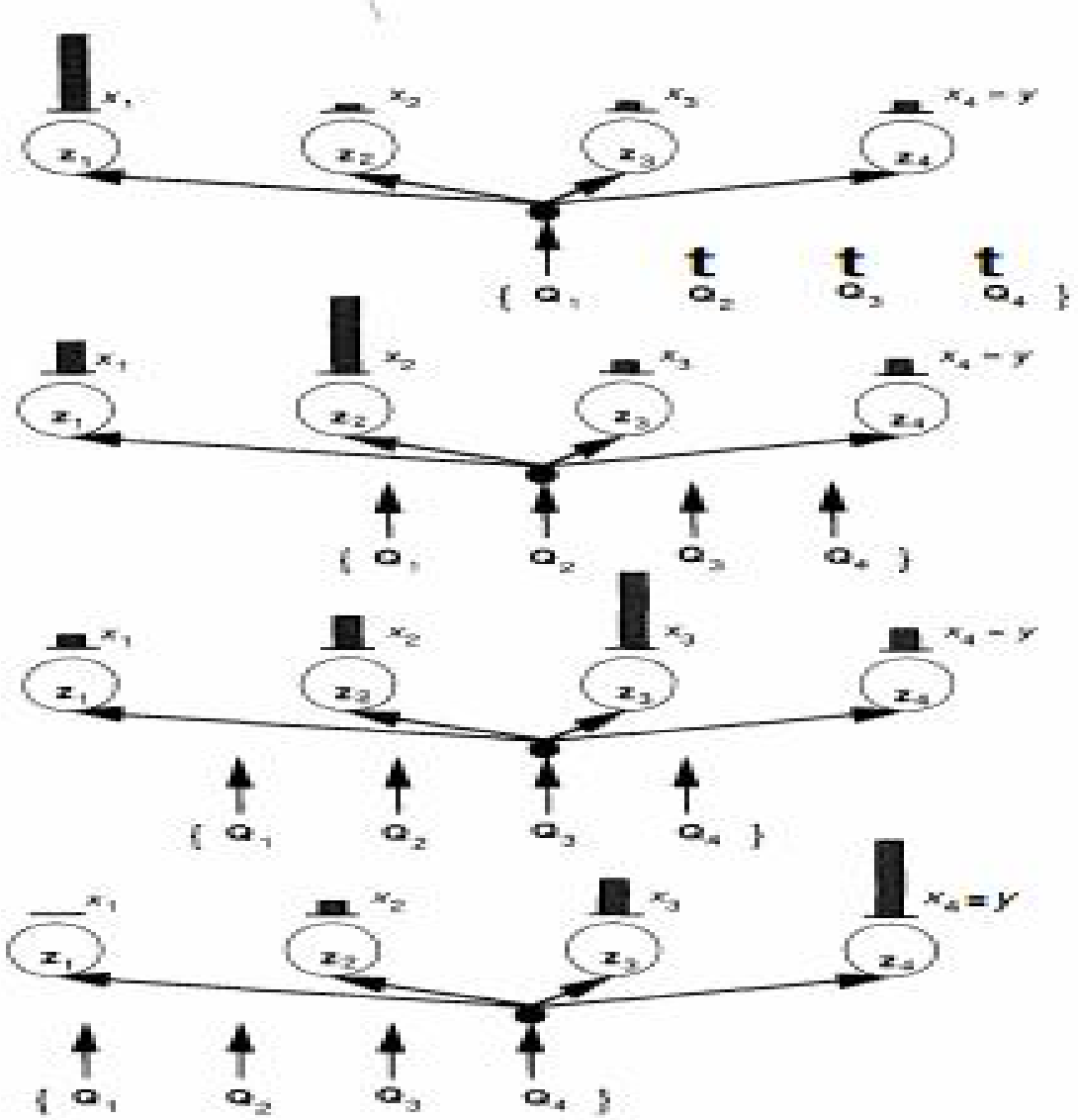



**Figure 9.7** This sequence (from top to bottom) shows the results from a small STN when the input patterns are presented in the order that matches the order of the weight vectors. Unit activities are represented by the bar graphs above each unit. Notice that, by the time the last input pattern is presented, the network output, represented by the output of the rightmost unit, is high, indicating that the STP is a close match to that pattern encoded by the sequence of weights in the network.

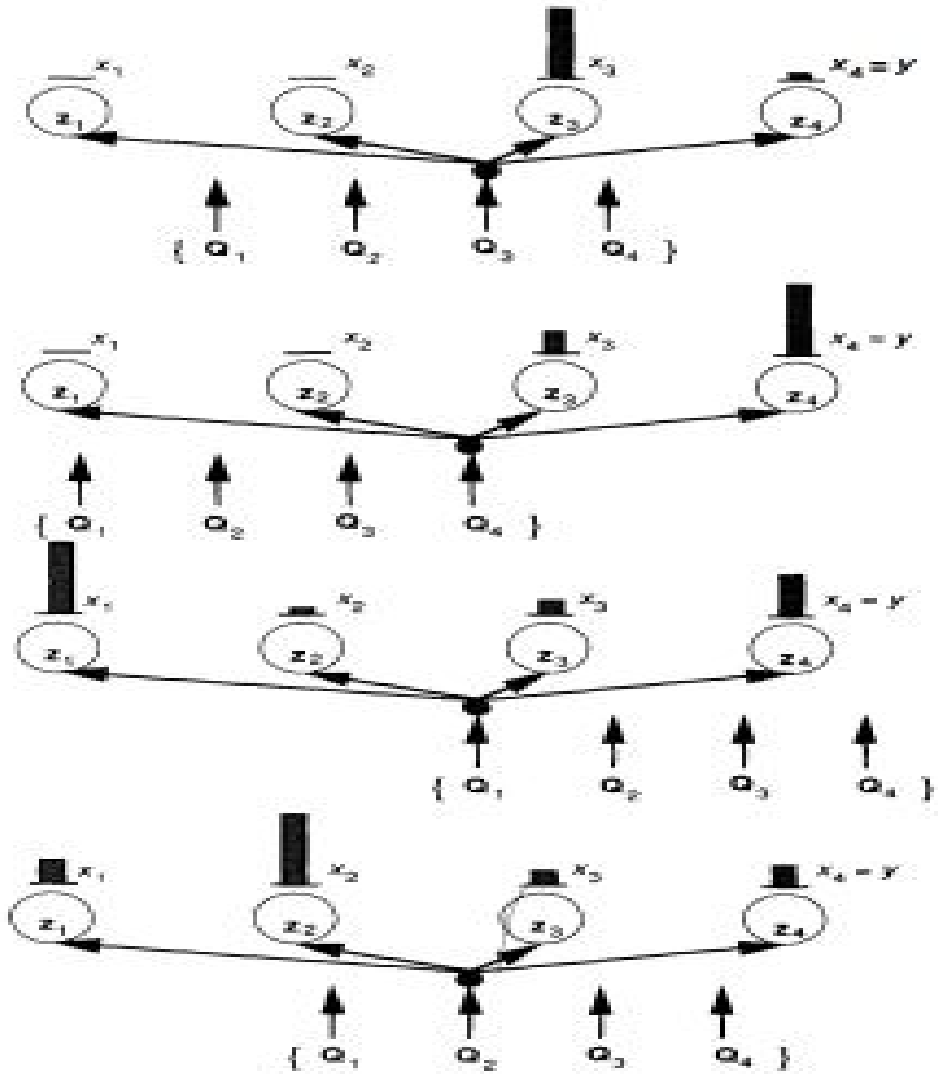



**Figure 9.8** In this sequence, the network is the same as that shown in Figure 9.7, as are the input patterns. In this case, however, the input patterns are presented to the network out of sequence. By the time the last input pattern is presented, the network output is low, indicating that the network has not recognized the sequence.

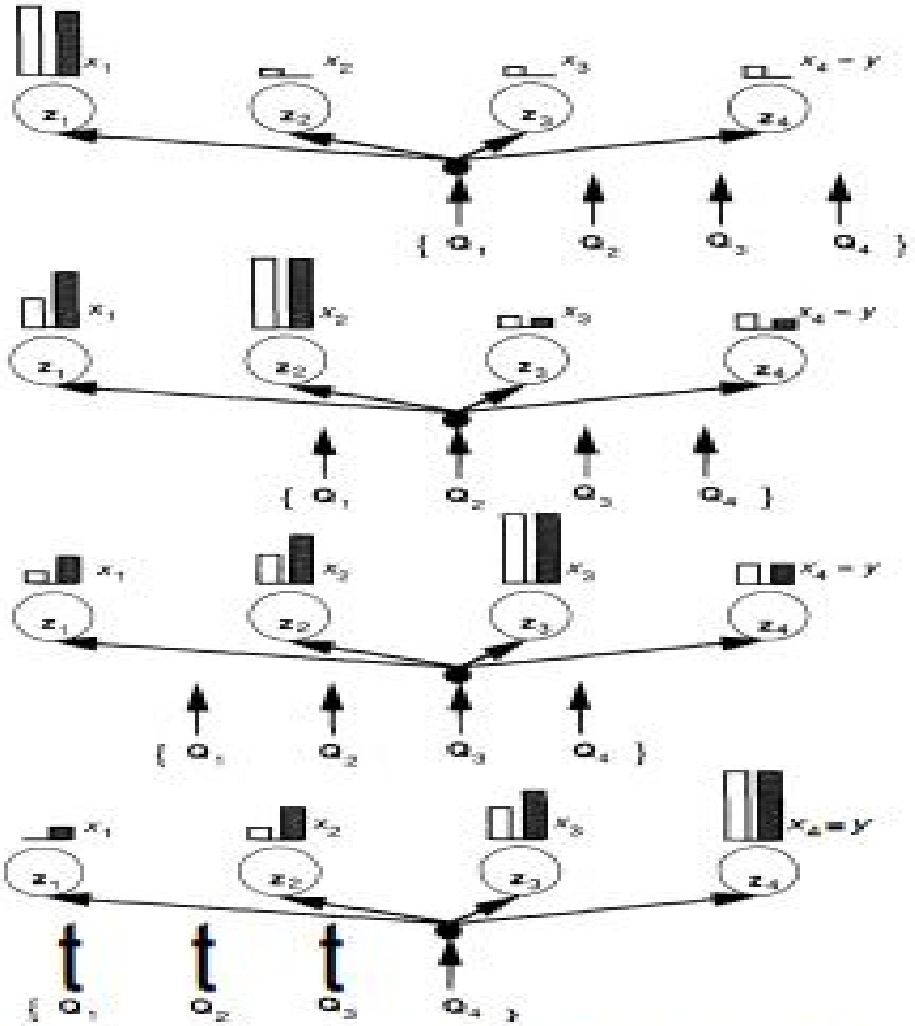



**Figure 9.9** This sequence shows the results from the presentation of an STP at two different rates. The activities represented by the unshaded bar graphs are for the case where each spatial pattern is presented and held for a certain time, $t$, before the next pattern is presented. The activities represented by the shaded bar graphs are for the case where each spatial pattern is presented for a time, $t/2$. Note that, in both cases, the network output is high.

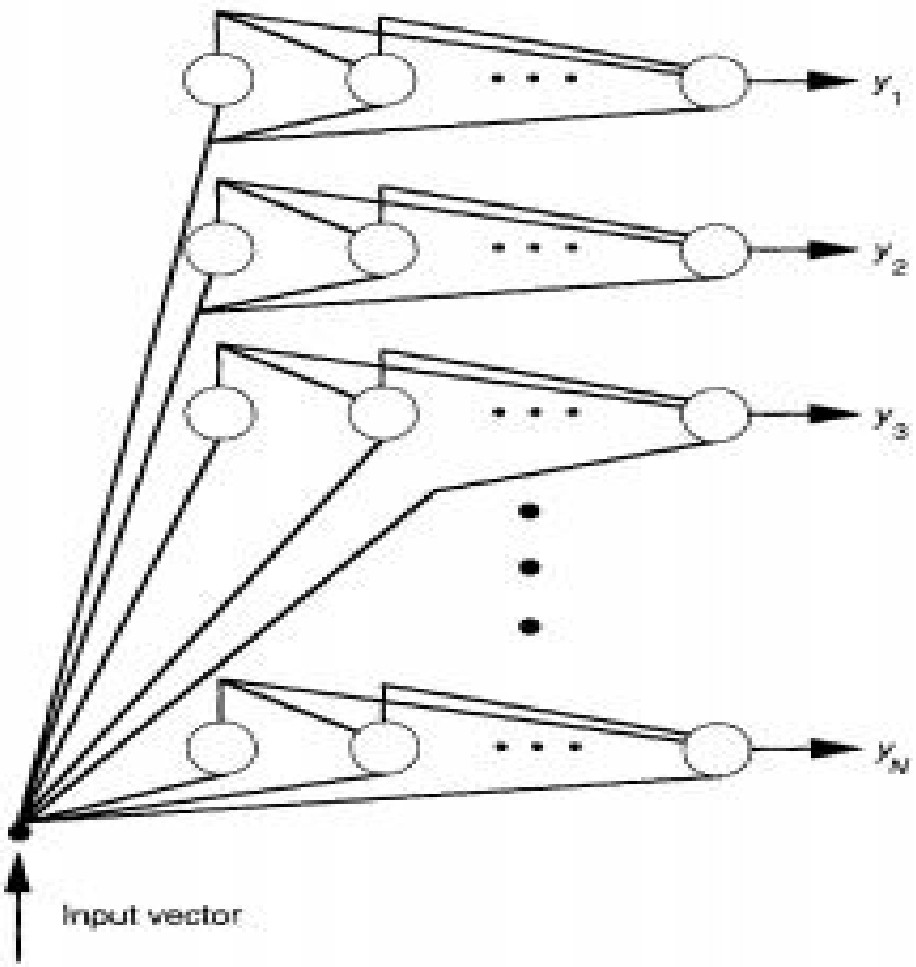



**Figure 9.10** A layered STN for word recognition is shown. Each layer is tuned to recognize a single word. Each unit in each layer receives the identical series of input vectors as a word is spoken and analyzed by the spectrum analyzer. After each word is presented, the largest $y$ value indicates the layer that most closely matched the STP.

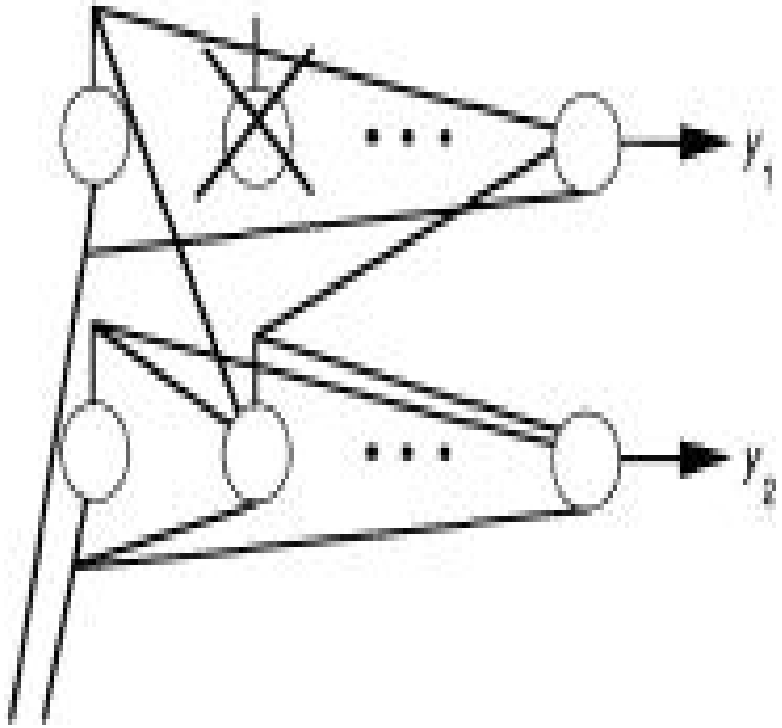



**Figure 9.11** This figure shows how to eliminate a redundant unit in a layered STN. Unit 2 on layers 1 and 2 have the same weight vector. By reconnecting the units as shown, we can eliminate one unit from the network.

You should note from the architecture diagram that there are no outputs corresponding to the $y$ values in the layered architecture described in the previous section. This fact complicates the interpretation of the network response to any particular STP. We shall first examine how the network responds to an STP, then shall discuss how the weight vectors are determined.

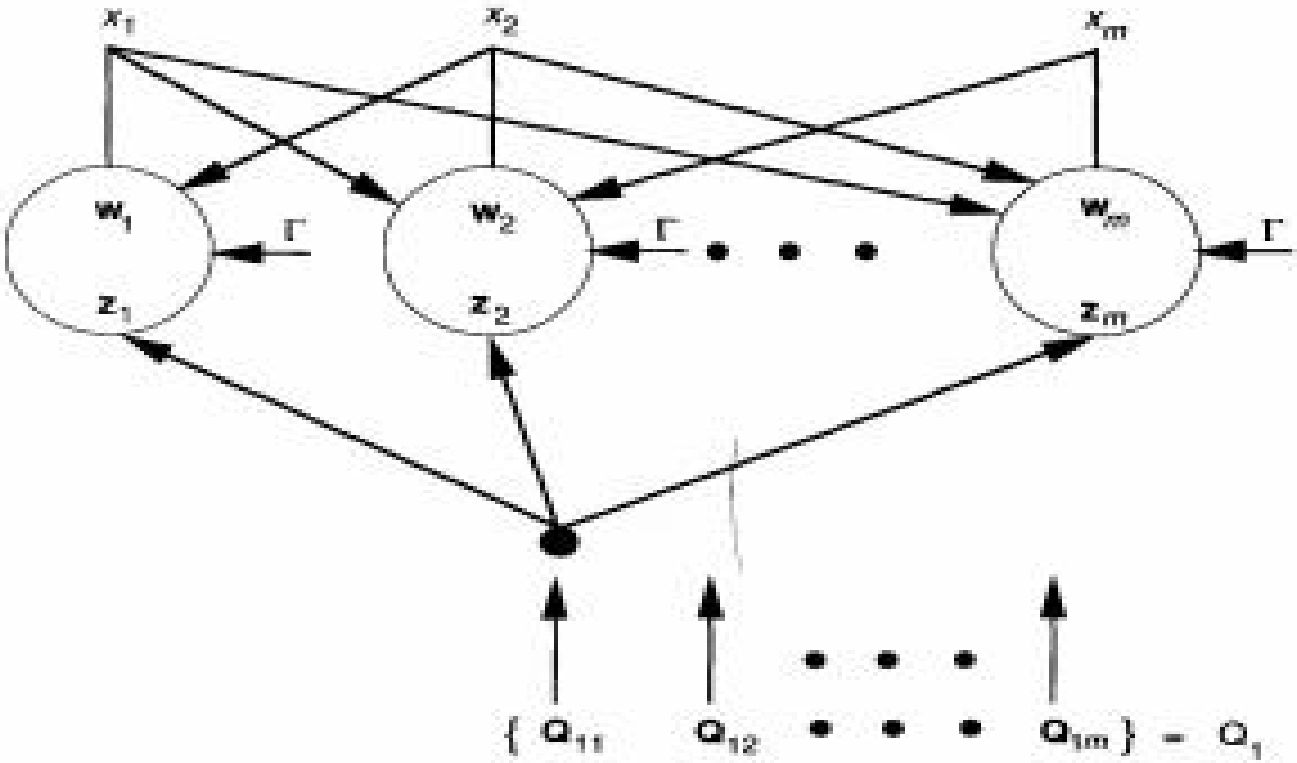



**Figure 9.12** The sequential, competitive avalanche-field architecture. The output of each unit is connected to all other units through the weight vectors $\mathbf{w}_i$. The $z_i$ are the weight vectors on the input connections. The global parameter $\Gamma$ mediates the competition in the network as described in the text.

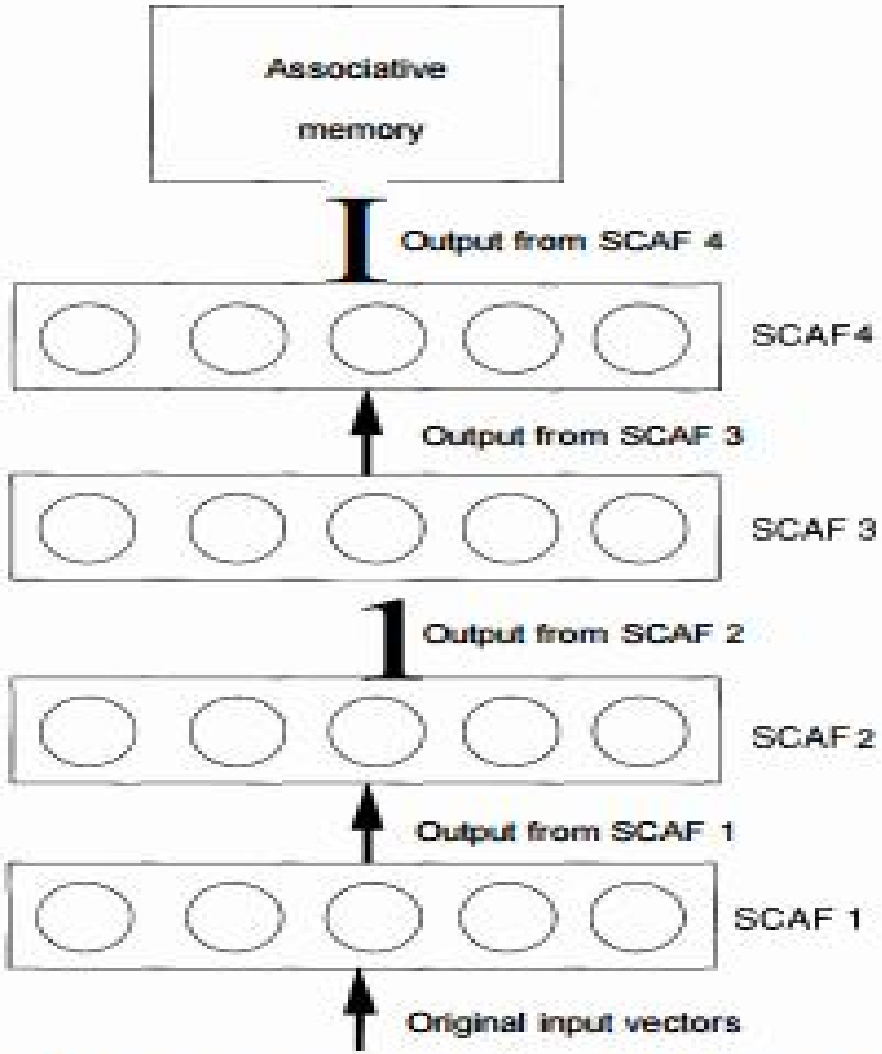



**Figure 9.18** This hierarchy of SCAF layers is used for spatiotemporal pattern classification. The outputs from each layer are sampled at a rate slower than the rate at which inputs to that layer change. The output from the top layer, essentially a spatial pattern, can be used as an input to an associative network that classifies the original STP.

# Spatio Temporal Networks

- Spatio Temporal Networks(STN) are spatial networks whose topology and/or attribute  change with time.These are encountered in many critical areas of everyday life such as transportation networks electric power distribution grids and social networks of mobile users.The model must meet the conflicting requirement of simplicity and adequate support for efficient algorithms.

# Spatio Temporal Networks

- Designing a STN database would require the data models,query languages and indexing methods tro efficiently represent query,store and manage time variant properties of the network.
- The properties of STN modeling and algorithm is to explore this design at the conceptual,logical and physical level.
- Models used to represent STNs are explored and analyzed.
- STN operations with their emphasis on their altered semantics with the addition of temporal dimensions are also analysed.

# Applications Of STN

- Speech Recognition
- Radar Analysis
- Sonar echo classification

# Unit-V
# NEO-CONGNITRON

- Cognitron - Structure & training
- The neocognitron architecture
- Data processing
- Performance
- Addition of lateral inhibition and feedback to the neocognitron.
- Optical neural networks
- Holographic correlators

# Cognitron

- Cognitron, as the name implies, is a network designed primarily with recognition Patterns in mind To do this, cognitive tissue uses inhibition and Neuron excitement in its various layers. It was originally designed by Fukushima (1975), and is an unattended network that resembles a biological neural network in Indonesia.

# Basic Principles of cognitron

- The cognitron basically consists of layers of inhibitory neurons and excitation.
  Interconnect neurons in certain layers only on the previous layer's
  neurons that surround the neurons. These settlements are referred to as
  Competition Areas of competition from given neurons.

# Basic Principles of cognitron

- For training efficiency, not all
  Training-trained neurons are limited to the most relevant elite group of Neurons, ie neurons previously trained for related tasks.
  While the connection area causes overlap of neurons, where the given neuron might belong to the connection region of more than one upstream neuron, Selection (for elite "selection) is introduced to overcome the overlapping difficulties.
  Competition will break neurons whose response is weaker.

# Basic Principles of cognitron

- Above Features provides a network with sufficient redundancy, to enable it to function Well in the presence of \ losing "neurons.
  The cognitron structure is based on a multi-layer architecture with progressive Reduction in the number of competitive areas. Alternatively, the two-layer group, LI and L-II may be repeated n times to produce a total 2n layer (L-I1, L-II1, L-I2, L-II2, etc.).

# Basic Principles of cognitron

- The cognitron structure is based on multi-layer architecture with progressive Reduction in the number of competitive areas. Alternatively, the two-layer group, LI and L-II may be repeated n times to produce a total 2n layer (L-I1, L-II1, L-I2, L-II2, etc.). The cognitron structure is based on multi-layer architecture with progressive Reduction in the number of competitive areas. Alternatively, the two-layer group, LI and L-II may be repeated n times to produce a total 2n layer (L-I1, L-II1, L-I2, L-II2, etc.).

# Network Operation

- Exclusive Neurons The output of excitatory neurons is calculated as follows:
- Let yk be the output of the excitation neuron in the previous layer and let vj be the output of the resistor neuron on the previous layer. De ne its output Components of excitatory neurons as:

$$x_i = \sum_k a_{ik} y_k \qquad \text{due to excitation inputs} \qquad (10.1)$$

$$z_i = \sum_k b_{ik} v_k \qquad \text{due to inhibition inputs} \qquad (10.2)$$

- Yi becomes the output of the excitatory cell. The ci weight is selected and not modulated during network practice.

Contd…

- Aik and bik become relevant weights, adjusted when neurons are considered
  More active than their neighbors, as discussed in 10.4 below. The total output of the
  above Neuron is given as:

$$y_i = f(N_i) \tag{10.3}$$

where

$$N_i = \frac{1+x_i}{1+z_i} - 1 = \frac{x_i - z_i}{1+z_i} \tag{10.4}$$

$$f(N_i) = \begin{cases} N_i \cdots & \text{for} \quad N_i \geq 0 \\ 0 \cdots & \text{for} \quad N_i < 0 \end{cases} \tag{10.5}$$

Hence, for small $z_i$

$$N_i \cong x_i - z_i \tag{10.6}$$

However, for very large $x$, $z$

$$N_i = \frac{x_i}{z_i} - 1 \tag{10.7}$$

Furthermore, if both $x$ and $z$ increase linearly with some $\gamma$ namely:

$$x = p\gamma \tag{10.8}$$

$$z = q\gamma \tag{10.9}$$

$p$ and $q$ being constants, then

$$y = \frac{p-q}{2q}\left[1 + \tanh\left(\log\frac{pq}{2}\right)\right] \tag{10.10}$$

Contd…

- Which is the legal form of Weber {Fechner (See: Guyton, 1971, p. 562 {563)
  Which approximates the response of biological sensory neurons.
- (B) Inhibitory neurons The
  output of the inhibitory neuron is given by:

$$v = \sum_i c_i y_i \qquad (10.11)$$

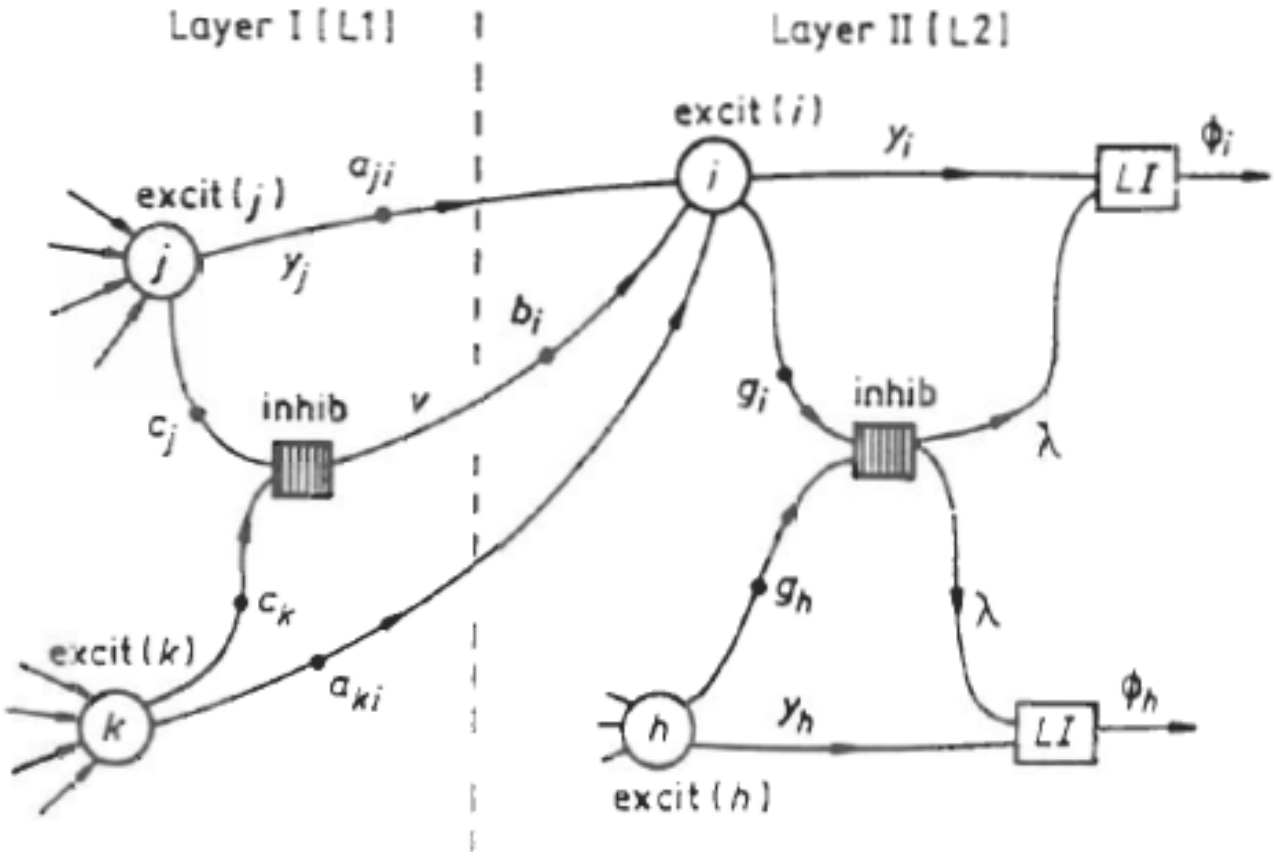
where

$$\sum_i c_i = 1 \qquad (10.12)$$

# Cognitron's Network Training

- The aji weight of excitation neurons in a two-tier cognitive structure is Iterated by? A as in the Press. (10.13) but only if the neuron is a winning neuron A region, where aji is the same as in the Press. (10.1) (ie, aji is the weight of a joy Input to a given excitation neuron), and cj becomes the weight at the input to the inhibitory neuron of this layer, whereas q is the preferred level of learning (training)

Contd..


Layer I [L1]
Layer II [L2]
excit (i)
excit (j)
excit (k)
excit (h)
inhib
inhib
LI
LI

Contd…

$$\partial\, a_{ji} = q\, c_j^*\, y_j^* \quad \text{(asterisk denoting previous layer)} \tag{10.13}$$

Note that there are several *excitory* neurons in each competition region of layer L1 and only one inhibitory layer.

The inhibitory weights $b_j$ to excitory neurons are iterated according to:

$$\partial b_i = \frac{q \sum_j a_{ji} y_j^z}{2v^*}; \quad \partial b_i = \text{change in } b_i \tag{10.14}$$

Where bi is the weight on the relationship between L1 layer inhibiting neurons and excitation neurons in L2, which denotes the sum of the weights of all L1 excitory neurons to the same neuron of L2, while v is the value of the Inhibitory Output as in Equation (10.11), q is Level of efficiency.

Contd..

- If no neurons are active in a particular area of competition, then Eq. (10.13), (10.14) Replaced with (10.15), (10.16), respectively:

$$\partial a_{ji} = q' c_j y_j \tag{10.15}$$

$$\partial b_i = q' v_i \tag{10.16}$$

where

$$q' < q \tag{10.17}$$

So now the higher the inhibition output, the higher the weight, sharp unlike the situation according to the Press. (10.13).

## The Neocognitron

- A more advanced version of cognitron was also developed by Fukushima etal. (1983), is a neocognitron. It is hierarchical and directed Simulation of human vision. The c algorithm for neocognitron is slight and very Complex, and therefore will not be discussed in this text. Recognition is arranged in the hierarchical structure of the 2-layer group, as in the case of cognitive. The two layers are now a layer (simple-cell) (S-layer) And the concentrating layer (C-layer), starting with the S-layer is denoted as S1 and ends with layer C (say, C4).

Contd..

- Each S-layer neuron responds to a given feature of its input layer (including the network input as a whole). Each array of Process layer C is in depth input from normally one layer S array.
  The number of neurons and arrangement generally goes from one layer to another. This structure allows the neocognitron to overcome the problem of recognition where the original Cognitive fails, such as the image below the position or distortion of the angle (Say the characters or digits are rotated somewhat in handwriting recognition problems).

Contd…

- Such as the image below the position or the distortion of the angle (Say the characters or digits are rotated somewhat in handwriting recognition problems). View image. 10.2. Such as the image below the position or the distortion of the angle (Say the characters or digits are rotated somewhat in handwriting recognition problems).

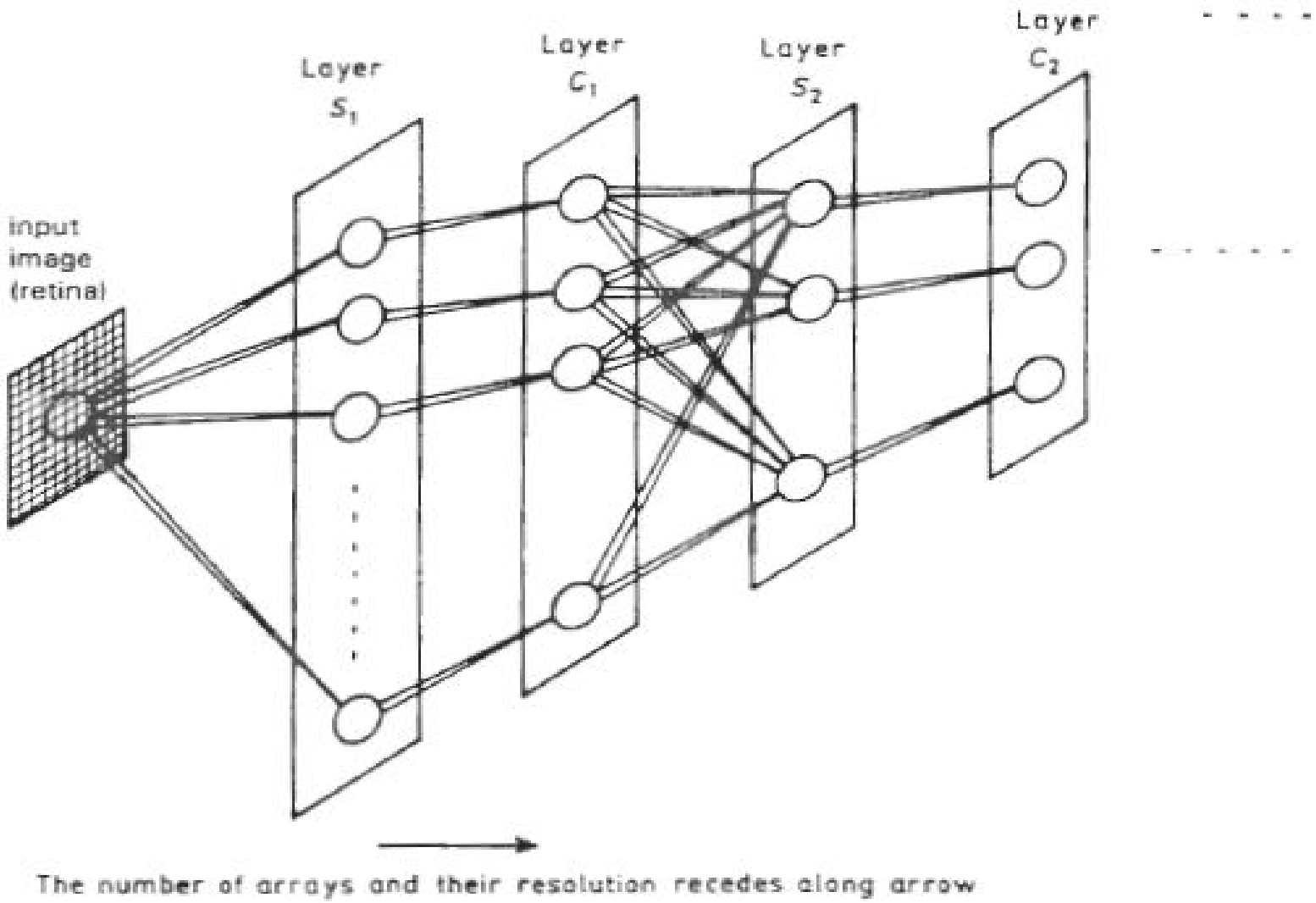



Fig. 10.2. Schematic of a neocognitron.

# Data Processing

- The lowest stage is the input layer consisting of two-dimensional array of cells, which correspond to photoreceptors of the retina. There are retinotopically ordered connections between cells of adjoining layers. Each cell receives input connections that lead from cells situated in a limited area on the preceding layer. Layers of "S-cells" and "C-cells" are arranged alternately in the hierarchical network.

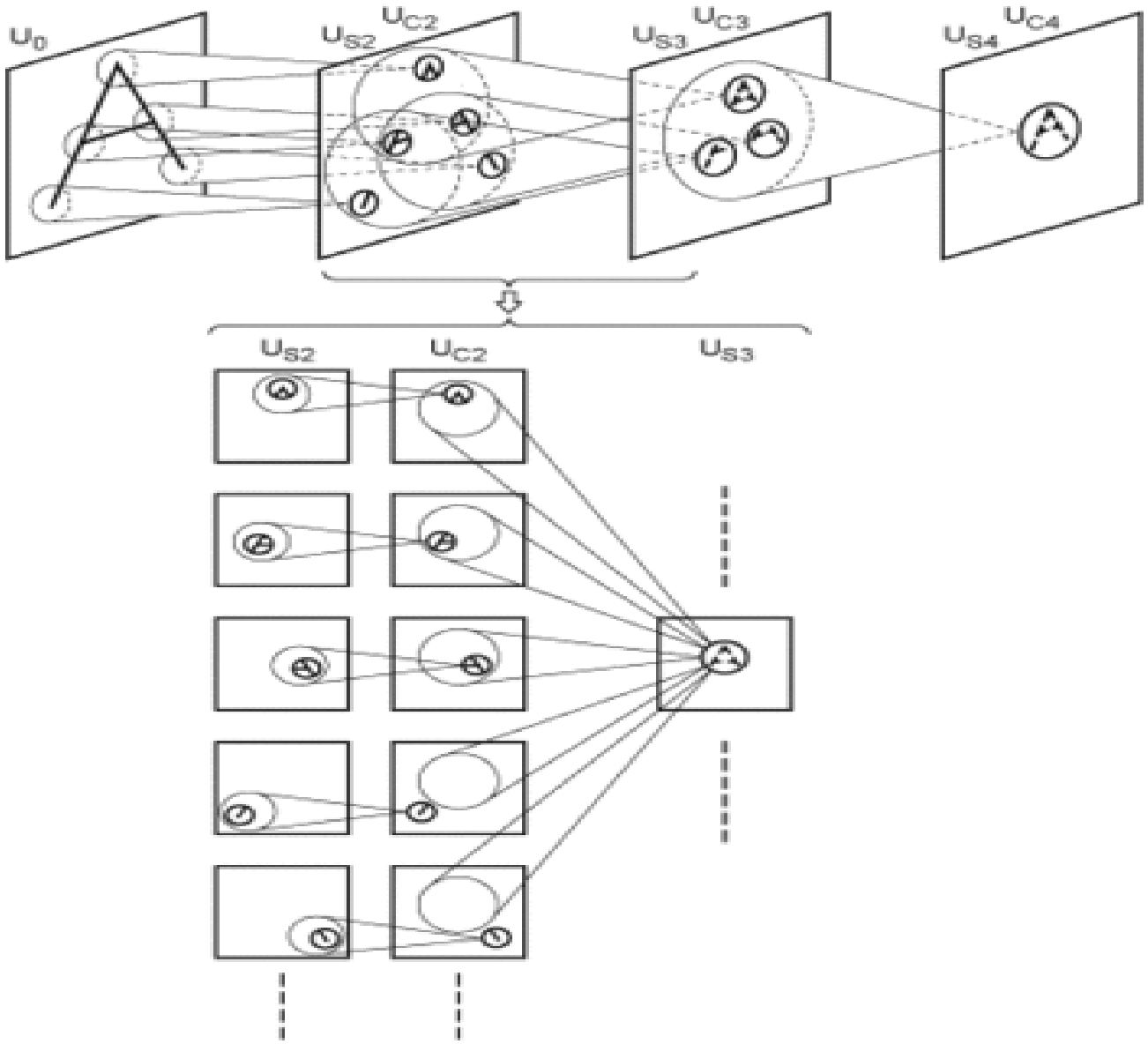
Contd..

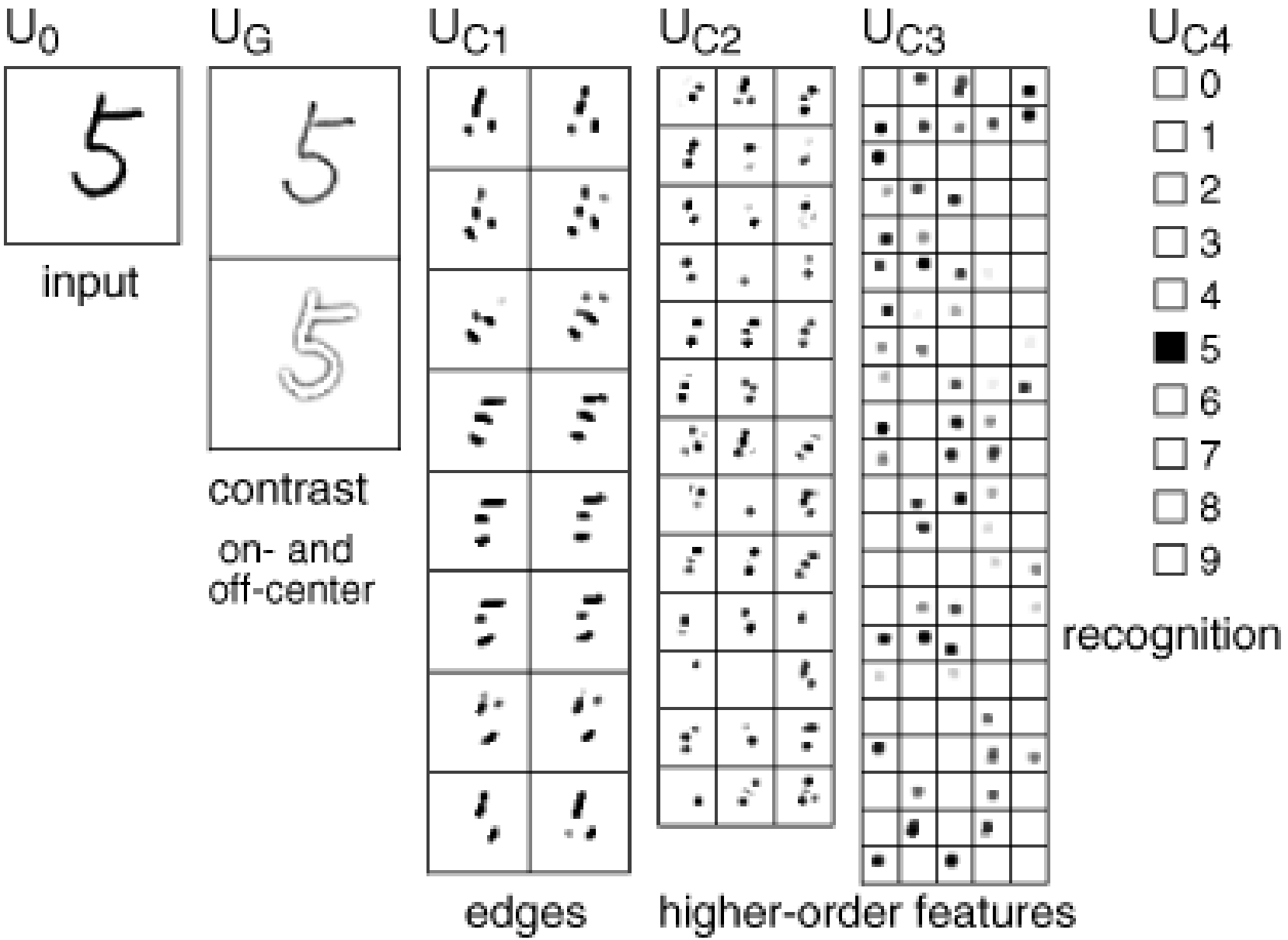
- **S-cells** work as feature-extracting cells. They resemble simple cells of the primary visual cortex in their response. Their input connections are variable and are modified through learning. After having finished learning, each S-cell come to respond selectively to a particular feature presented in its receptive field. The features extracted by S-cells are determined during the learning process. Generally speaking, *local* features, such as edges or lines in particular orientations, are extracted in lower stages. More *global*features, such as parts of learning patterns, are extracted in higher stages.

- **C-cells**, which resembles complex cells in the visual cortex, are inserted in the network to allow for positional errors in the features of the stimulus. The input connections of C-cells, which come from S-cells of the preceding layer, are fixed and invariable. Each C-cell receives excitatory input connections from a group of S-cells that extract the same feature, but from slightly different positions.

Contd..

- The C-cell responds if at least one of these S-cells yield an output. Even if the stimulus feature shifts in position and another S-cell comes to respond instead of the first one, the same C-cell keeps responding. Thus, the C-cell's response is less sensitive to shift in position of the input pattern. We can also express that C-cells make a blurring operation, because the response of a layer of S-cells is spatially blurred in the response of the succeeding layer of C-cells.


U0
UC2
US2
UC3
US3
UC4
US4
US2
UC2
US3


$U_0$
$U_G$
$U_{C1}$
$U_{C2}$
$U_{C3}$
$U_{C4}$
0
1
2
3
4
5
6
7
8
9
input
contrast
on- and
off-center
edges
higher-order features
recognition

- Each layer of S-cells or C-cells is divided into sub-layers, called "cell-planes", according to the features to which the cells responds. The cells in each cell-plane are arranged in a two-dimensional array. A cell-plane is a group of cells that are arranged retinotopically and share the same set of input connections. In other words, the connections to a cell-plane have a translational symmetry. As a result, all the cells in a cell-plane have receptive fields of an identical characteristic, but the locations of the receptive fields differ from cell to cell. The modification of variable connections during the learning progresses also under the restriction of shared connections.

# Optical Neural Networks

- The optical implementation of artificial neural networks is a subject that combines optics and neural networks. The notion that links the two fields is connectionism. In optical computers, photons are used instead of electrons as the carriers of information. The advantage of doing this derives from the fact that photons do not directly interact with one another. This makes it easier to establish a communication network connecting a large number of processing elements. Therefore, the design of optical computers is naturally guided toward architectures that require many connections.

# Holographic correlators

- In optical implementations using holograms,the interconnections are realized holographically using the third dimensions.
- The simplest form of holographic associative memory is realized by recording the hologram of an image using its associated pattern as the reference beam.
- The active planes are populated with neurons alone and holograms can be used to connect each neuron with same or adjacent processing planes.

In optical implementation using holograms, the interconnections are realized holographically using the third dimensions. The simplest form of holographic associative memory is realized by recording the hologram of an image using its associated pattern as the reference beam. The active planes can be populated with neurons alone and holograms can be used to connect each neuron with same, or adjacent, processing planes.

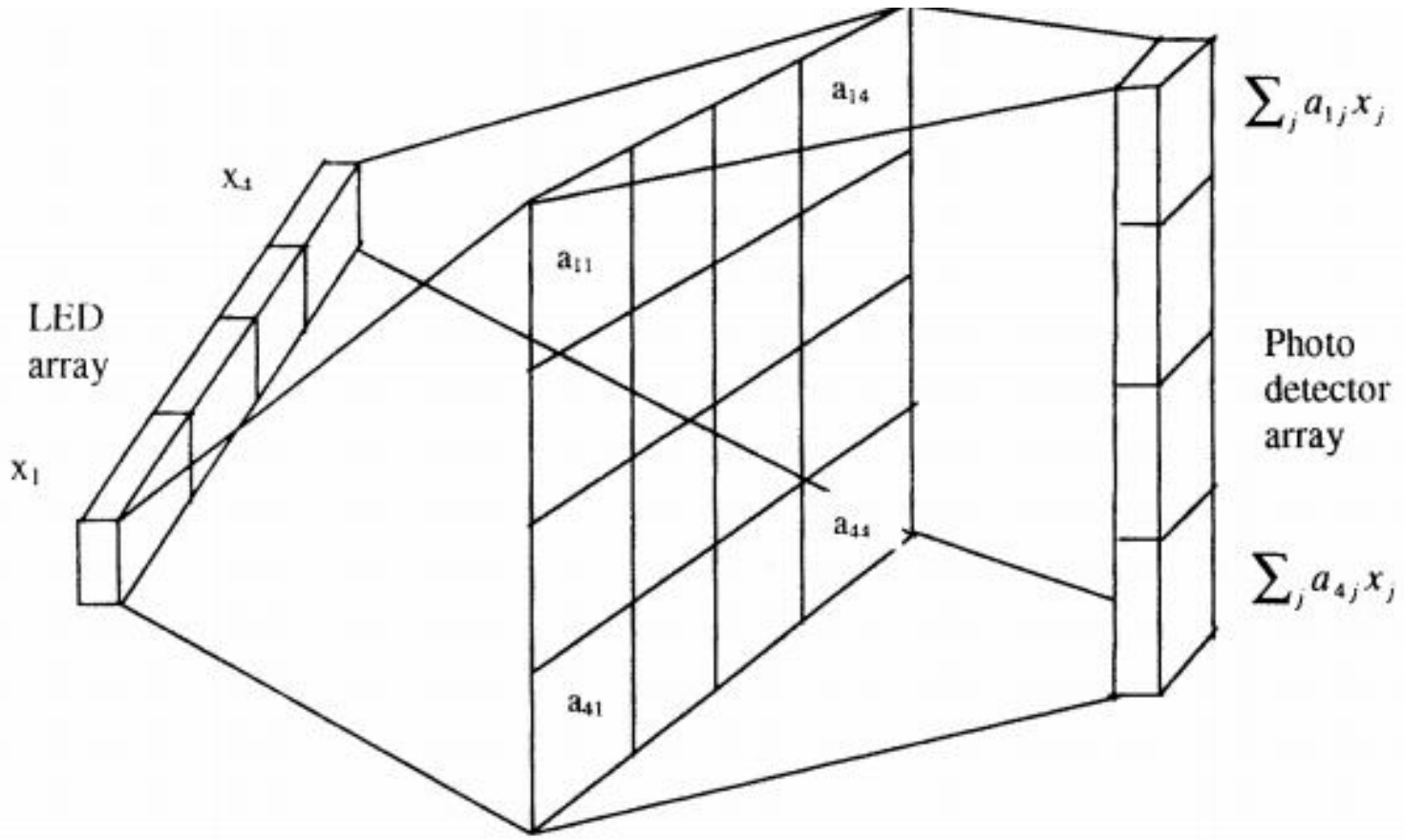



**Figure 1.5 Schematic diagram of an optical vector-matrix**